

Gradientenabstieg am Beispiel neuronaler Netze

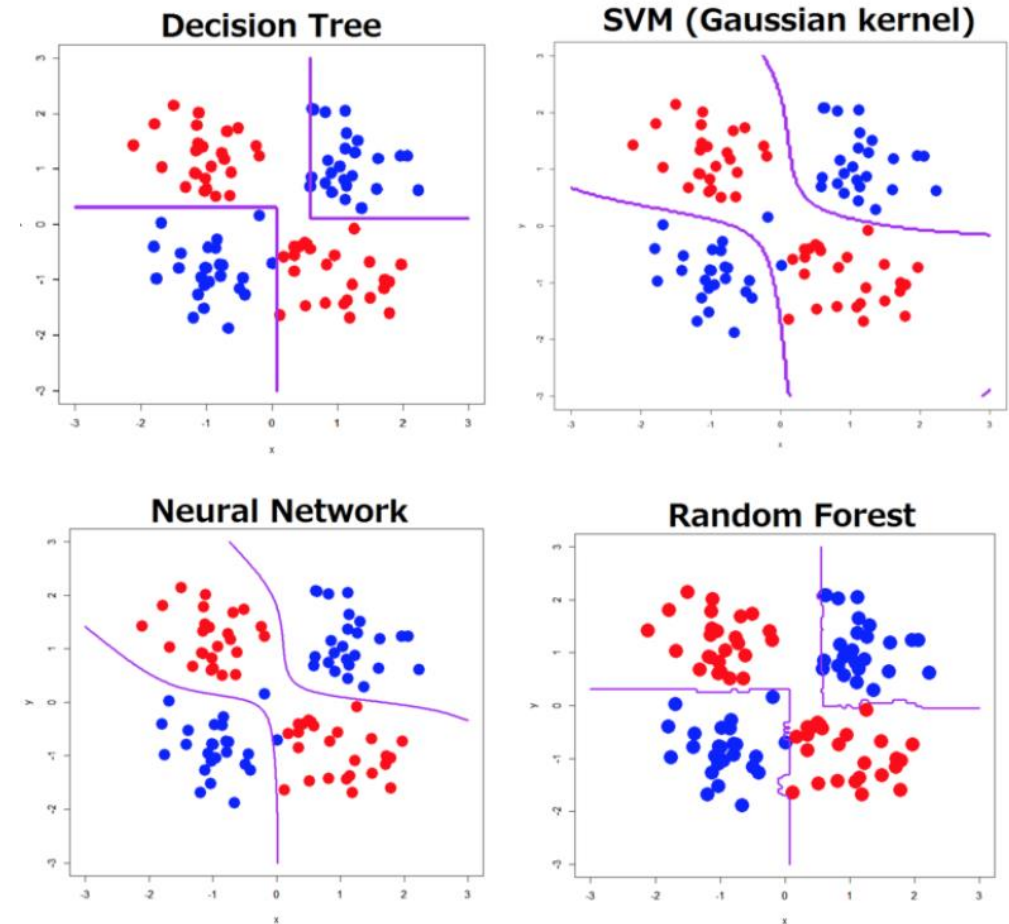
Orhan Konak
Hochschule Hannover
17.03.2026



Why Neural Networks?

- Many ML tasks require modeling **nonlinear relationships**
- Linear models often insufficient
- Neural networks = universal function approximators
- Used in vision, NLP, medicine, forecasting, etc.

Question: *How does it learn?*



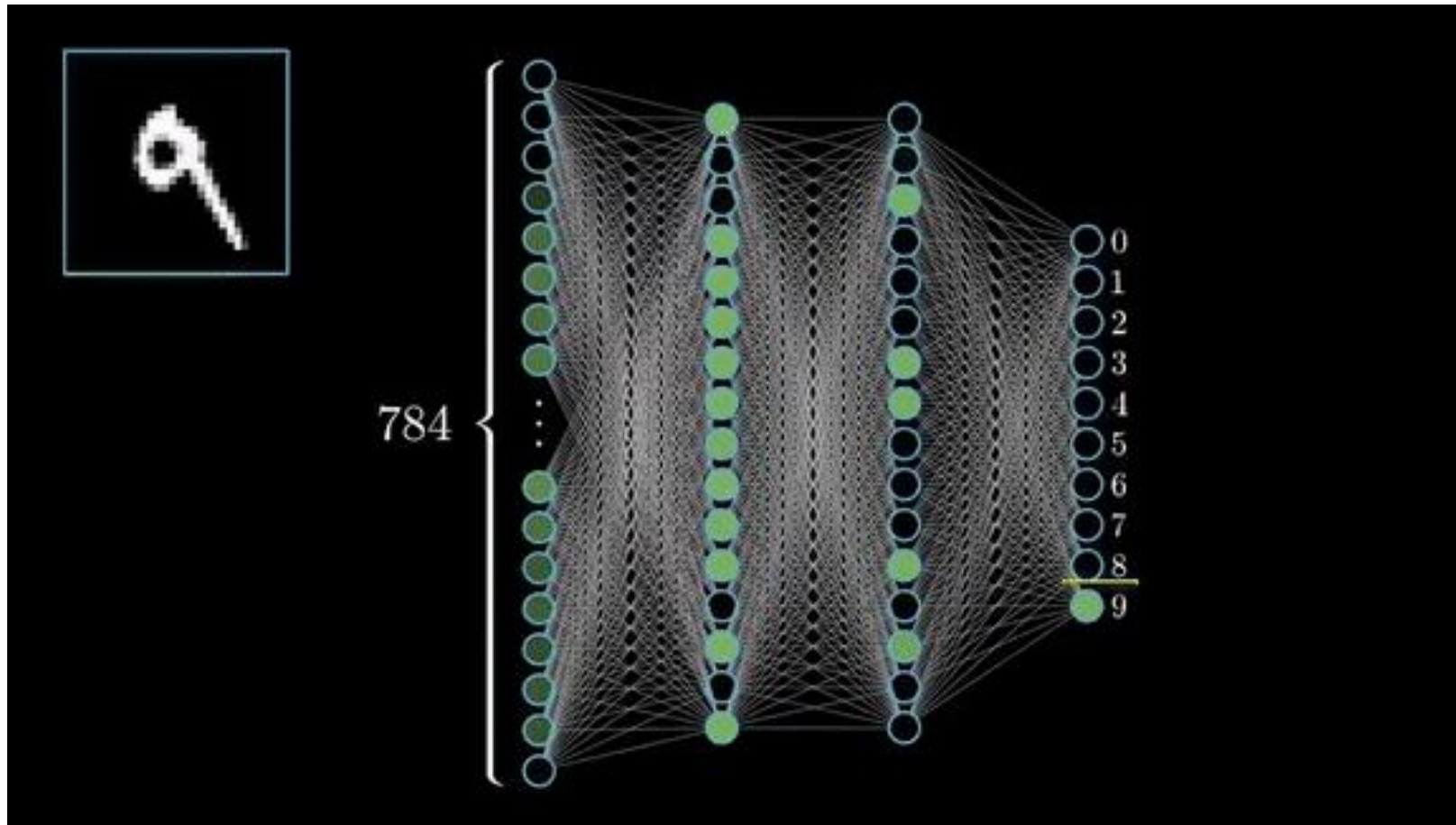
Agenda

- Neural Networks
- Gradient Descent
- Backpropagation

Agenda

- **Neural Networks**
- Gradient Descent
- Backpropagation

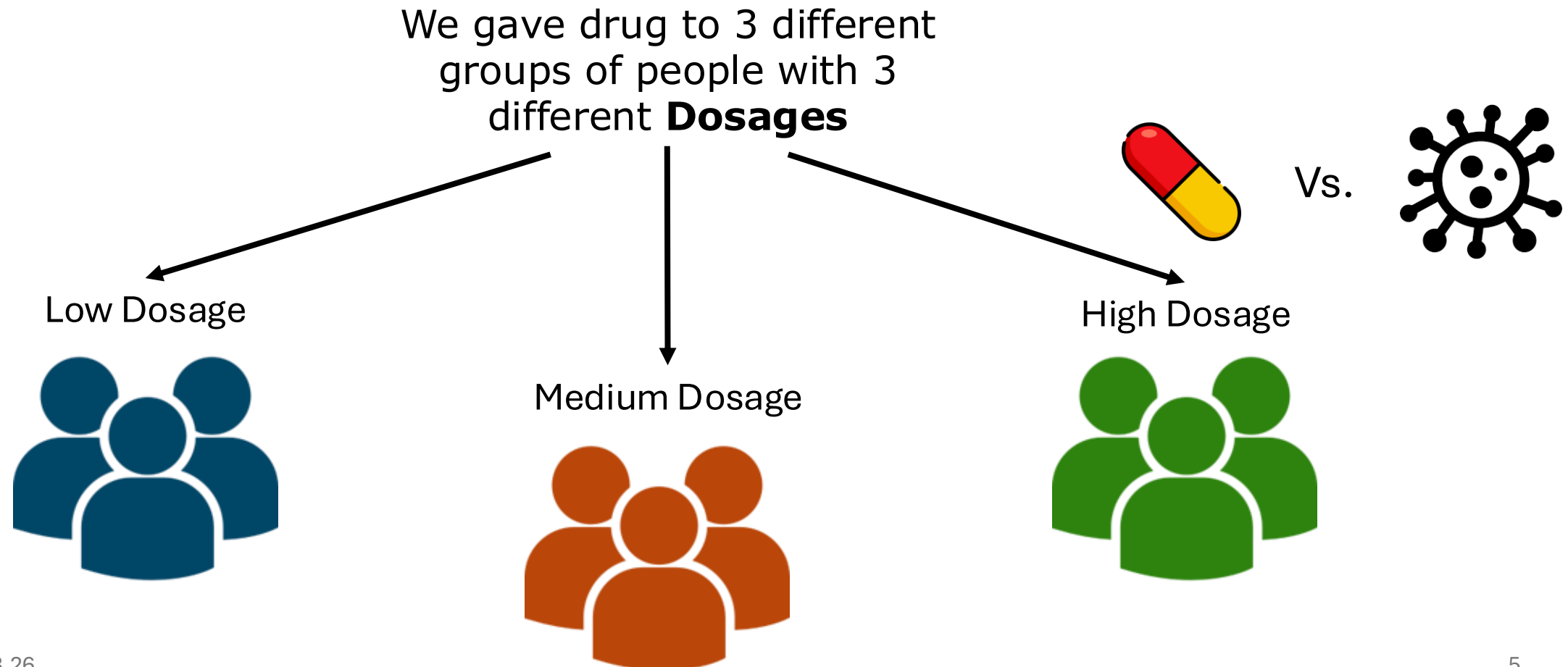
Neural Network Example



<https://medium.com/analytics-vidhya/applying-ann-digit-and-fashion-mnist-13accfc44660>

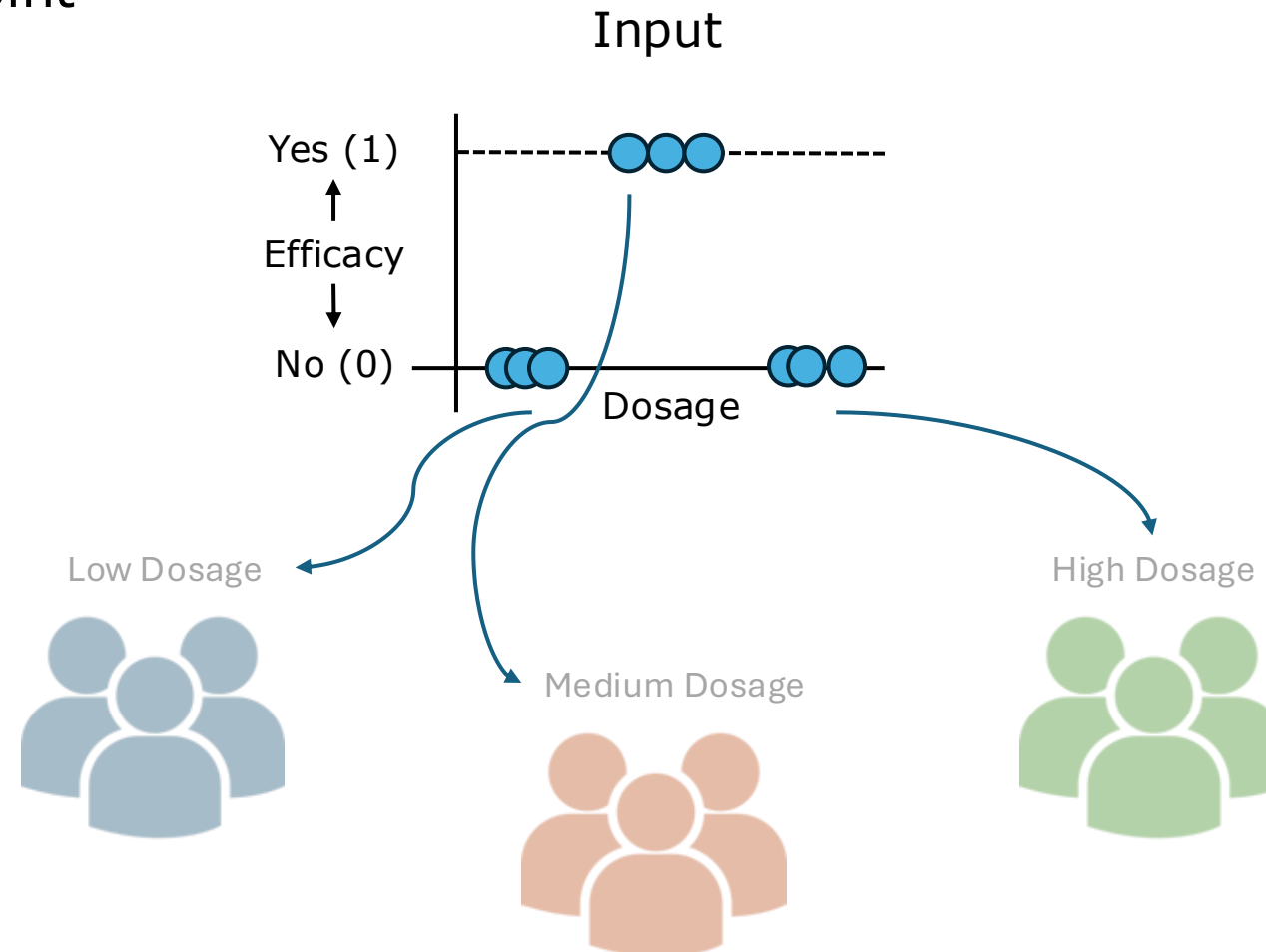
Neural Network Example

- E.g. Drug Dosage

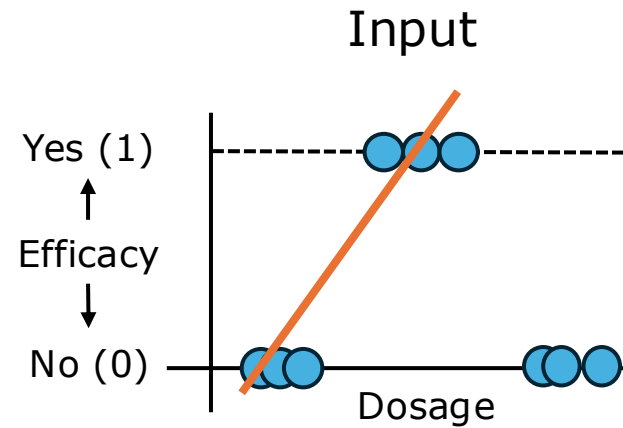


Neural Network Example

We can plot any point

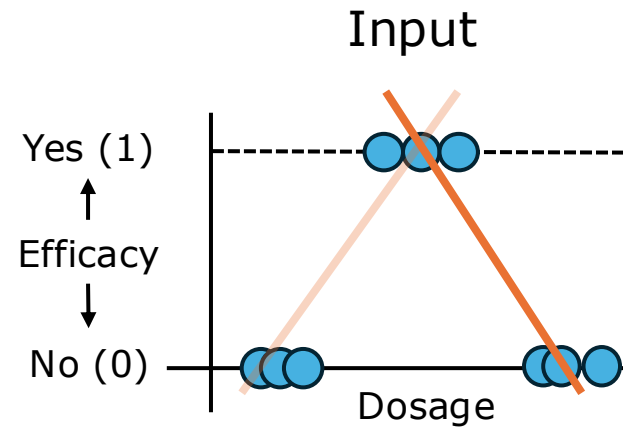


Neural Network Example



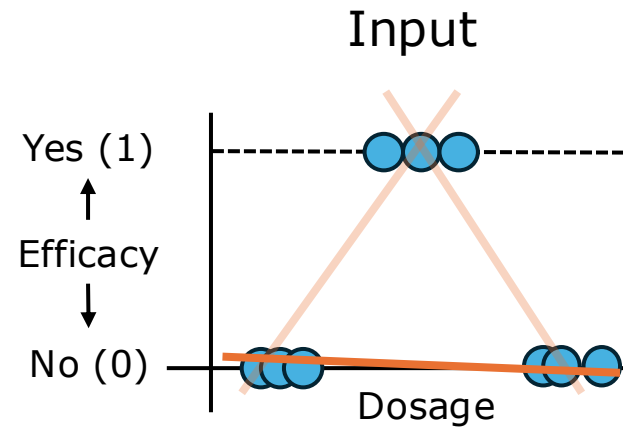
However, we can't just fit a **straight line** to the data to make predictions ...

Neural Network Example



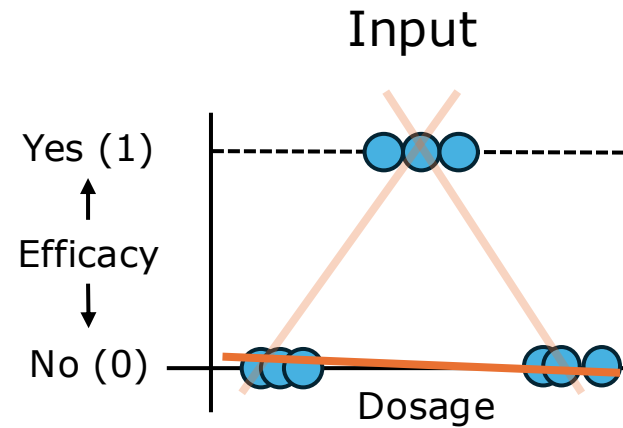
However, we can't just fit a **straight line** to the data to make predictions ...

Neural Network Example



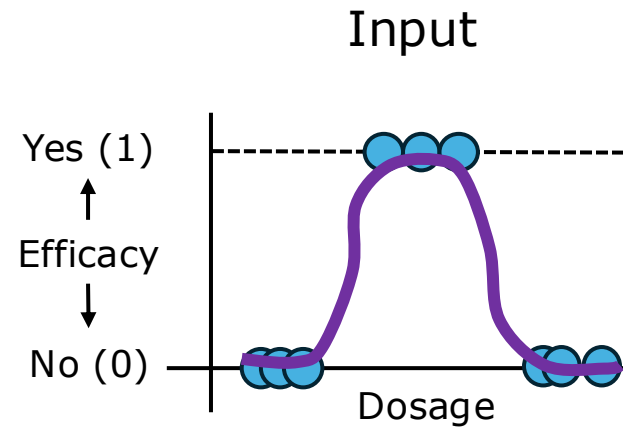
However, we can't just fit a **straight line** to the data to make predictions ...

Neural Network Example



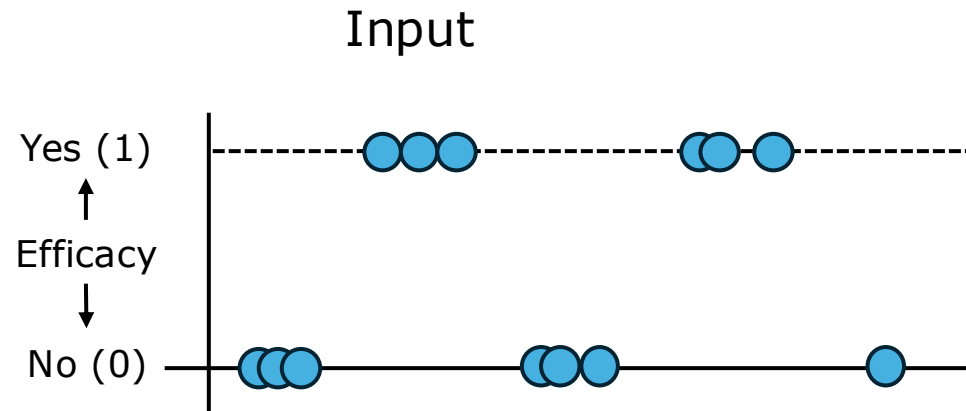
However, we can't just fit a **straight line** to the data to make predictions ...

Neural Network Example



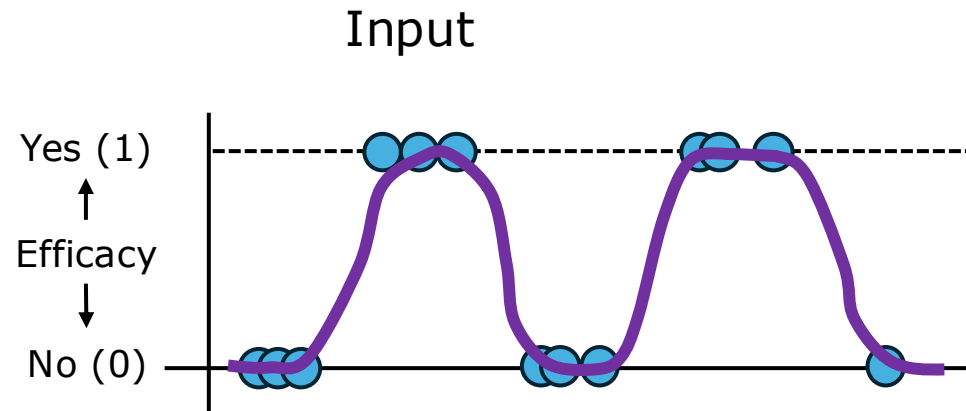
The good news is that
a **Neural Network**
can fit a **squiggle** to
the data

Neural Network Example



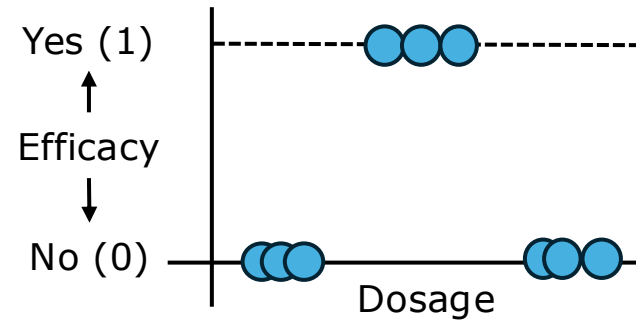
... even if we have a
really complicated
dataset like this

Neural Network Example



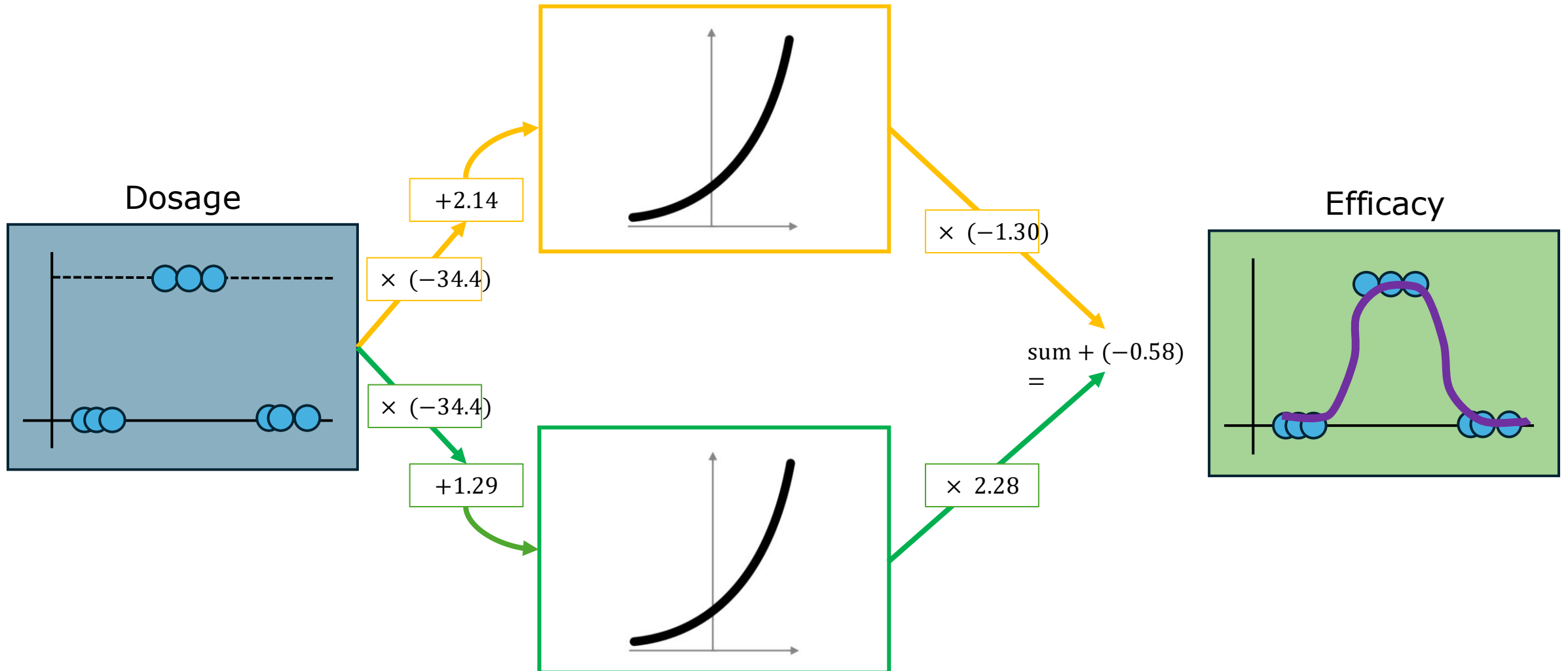
... even if we have a really complicated dataset like this

Neural Network Example

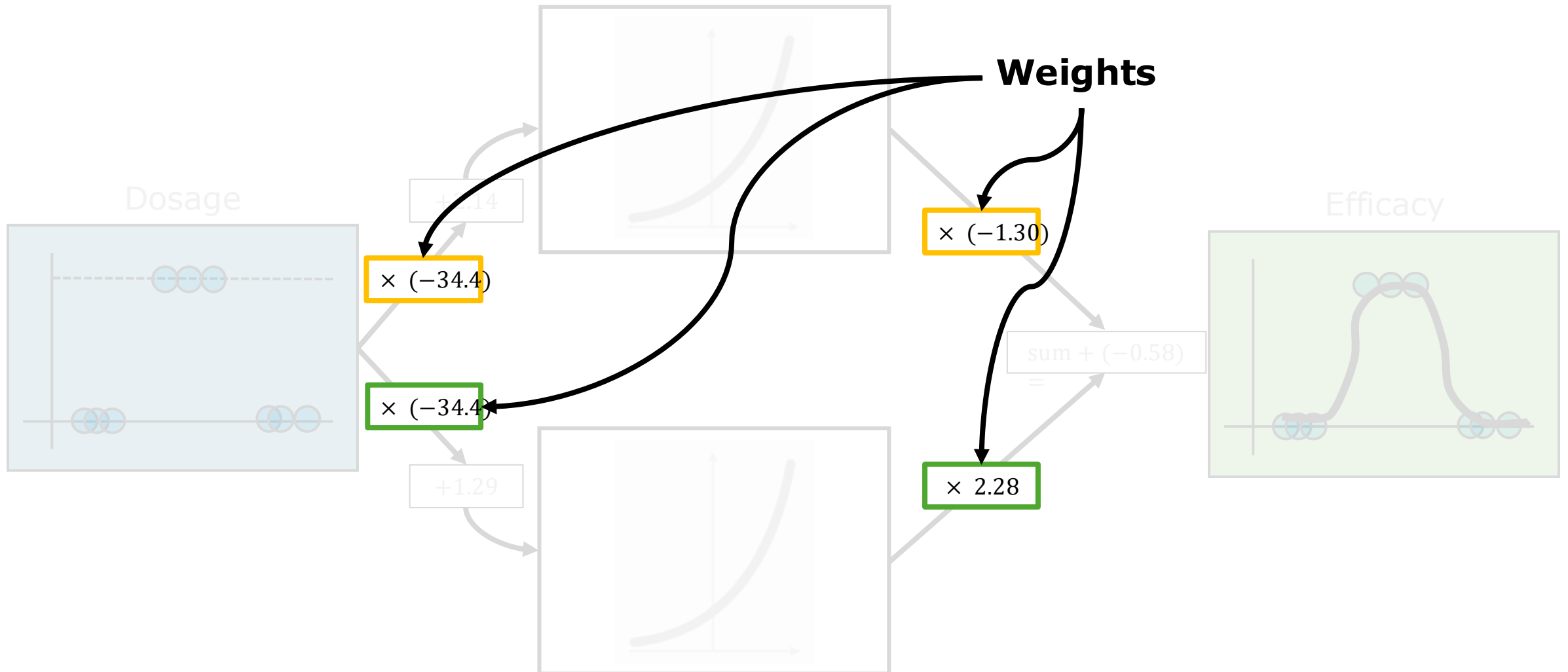


We can use this simple example dataset and turn it into a **Neural Network**

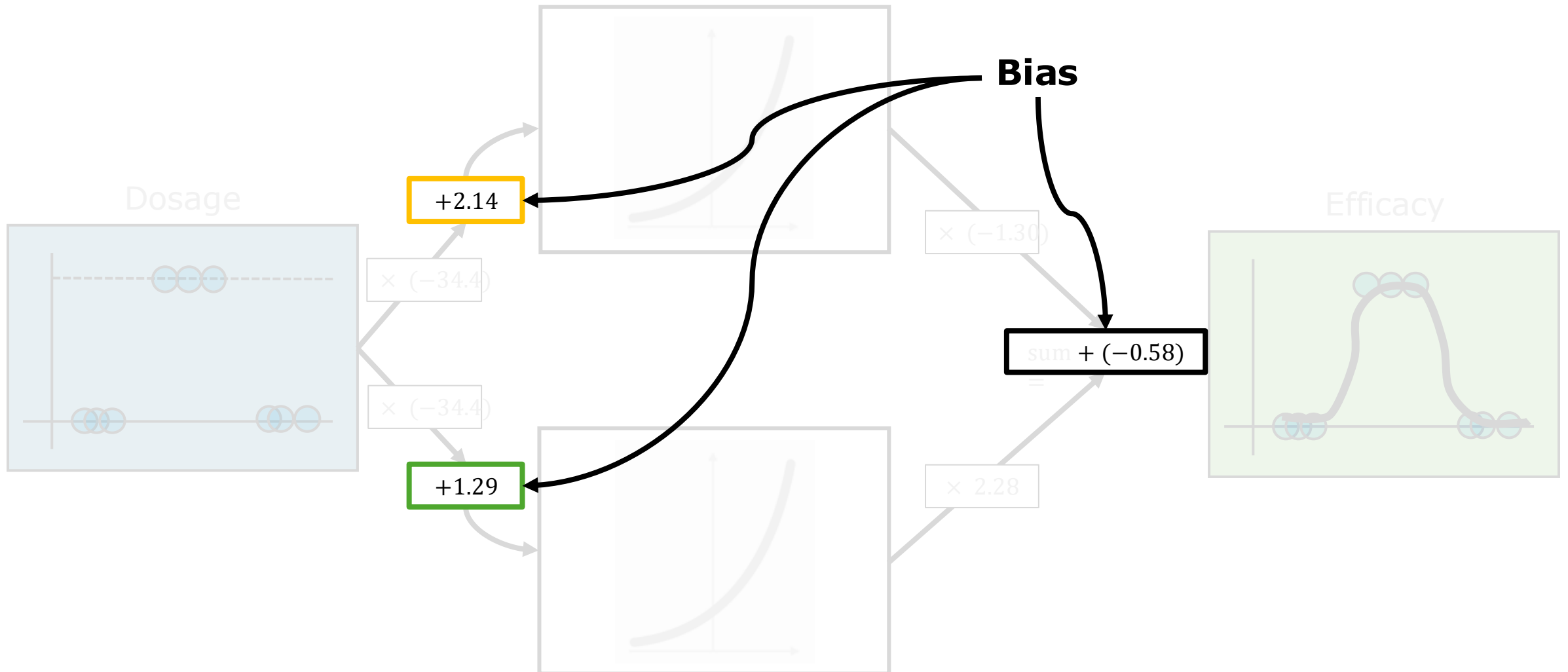
Neural Network Example



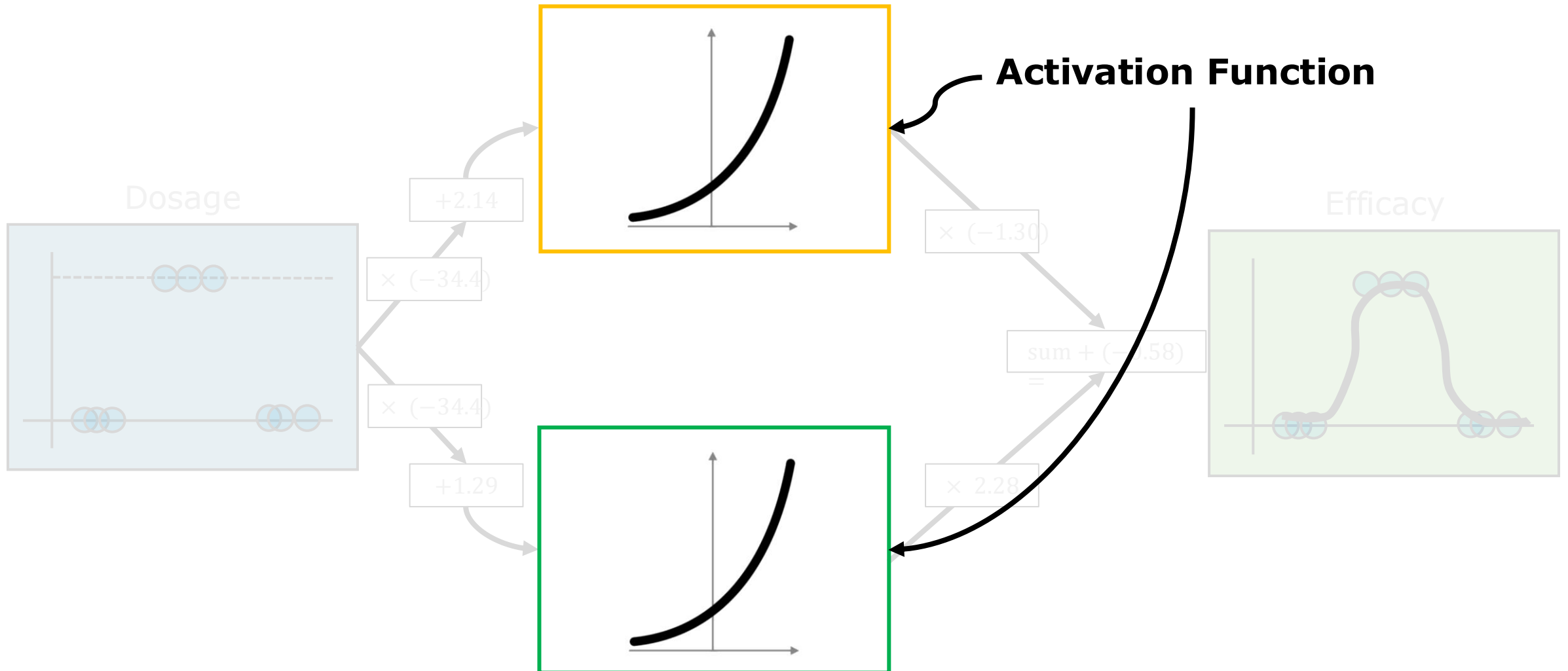
Neural Network Example



Neural Network Example



Neural Network Example



What Is a Neural Network?

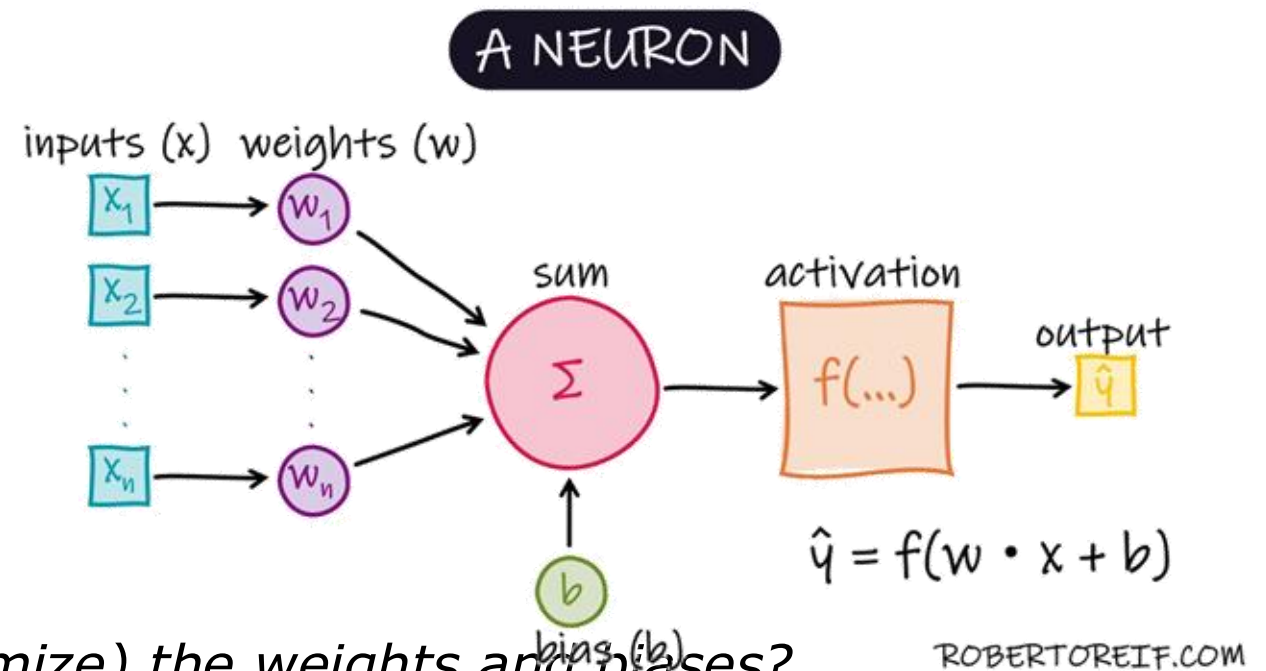
In mathematical notation:

- For one neuron:

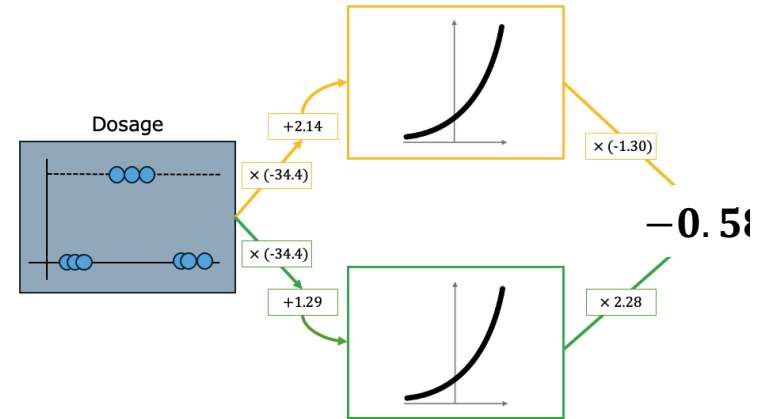
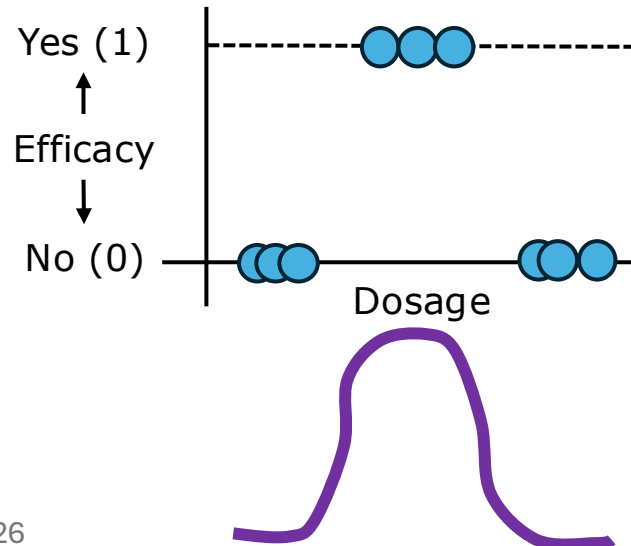
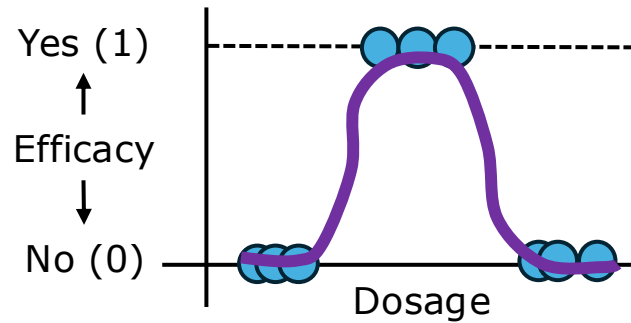
$$z = \mathbf{w}^T \mathbf{x} + b$$

- Stack layers → deep network

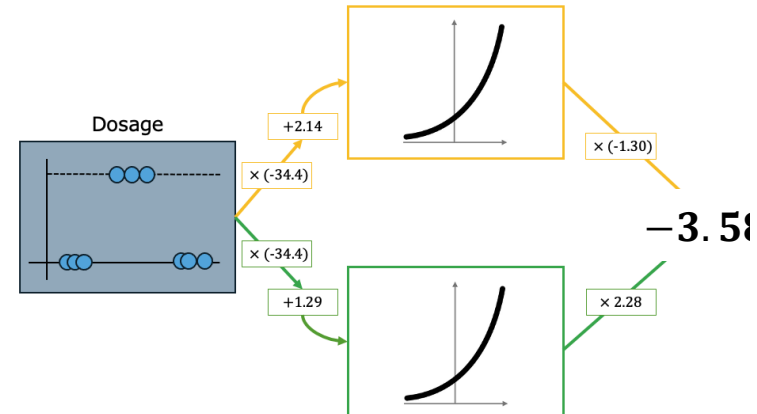
Question: *How do we calculate (optimize) the weights and biases?*



The Learning Goal

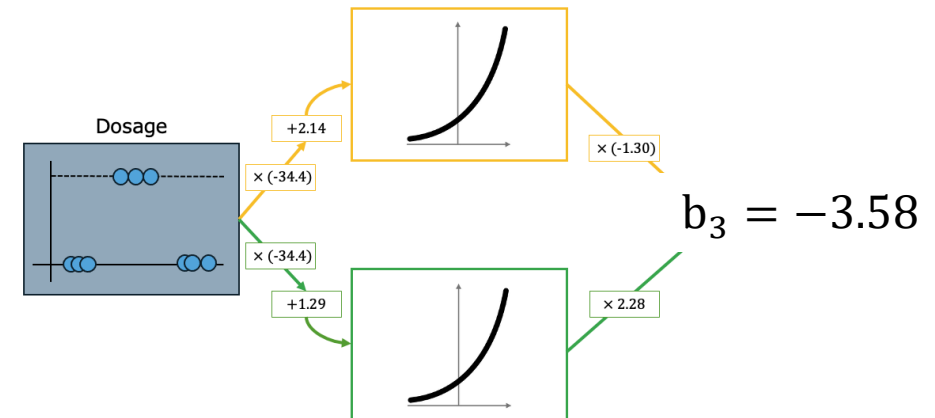
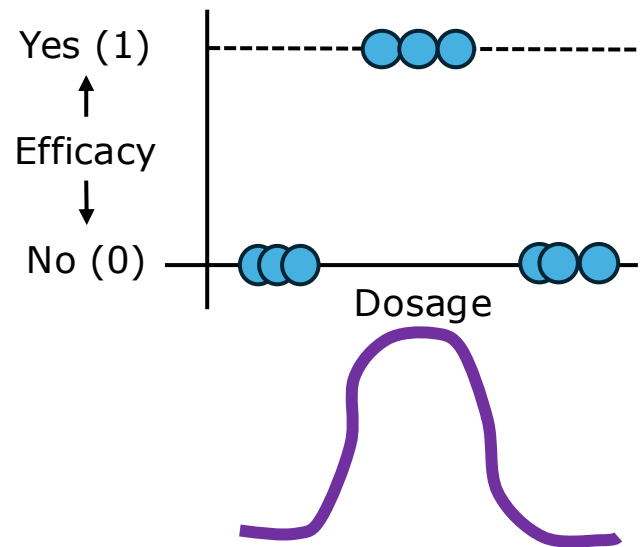


Let's imagine
what happens to
our outcome,
when changing
the last bias term
from this ...

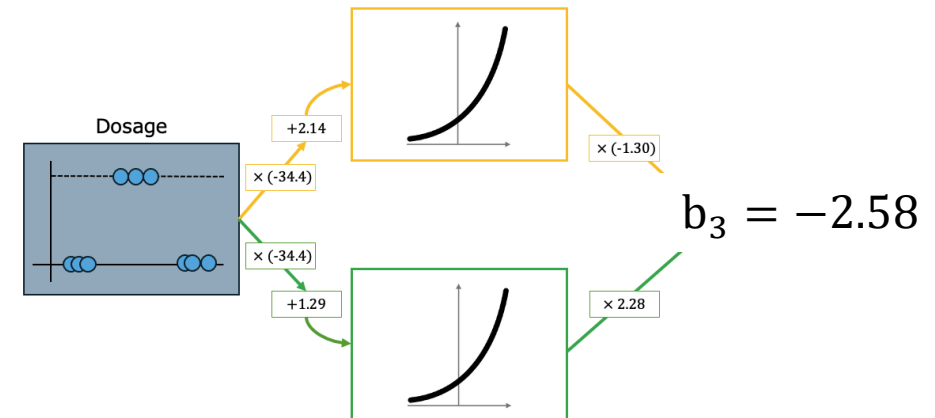
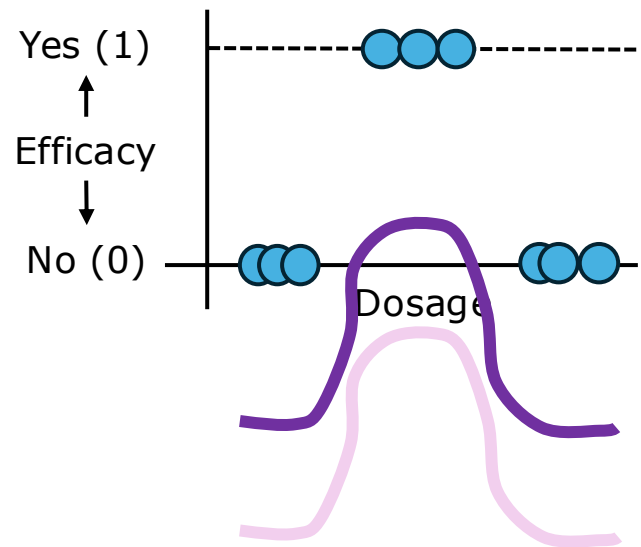


... to this

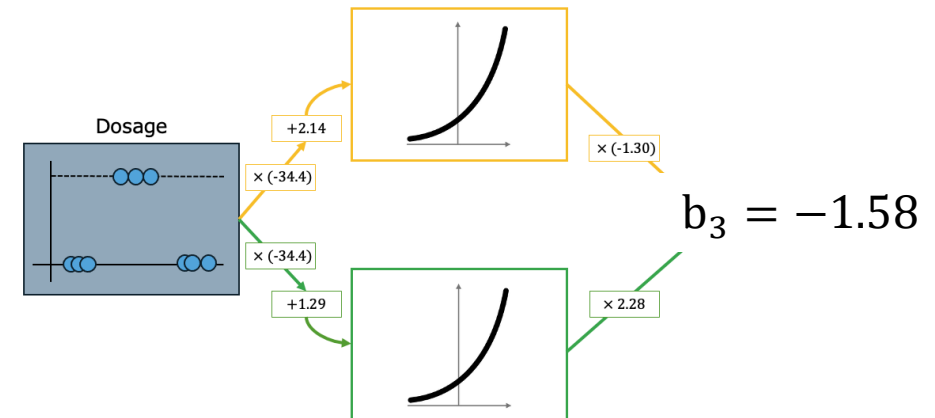
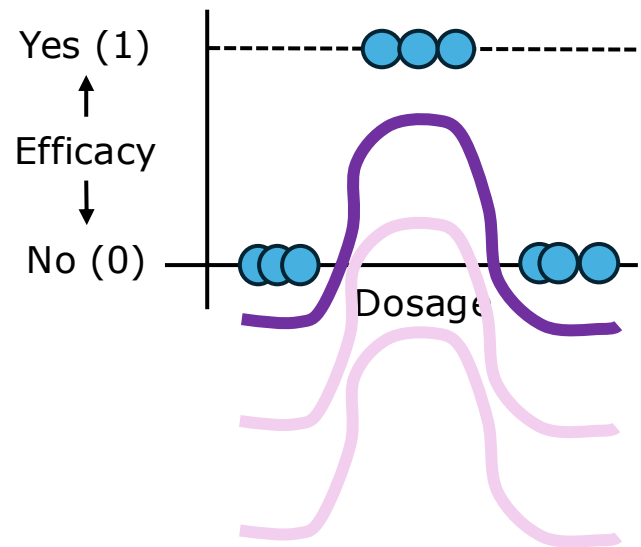
The Learning Goal



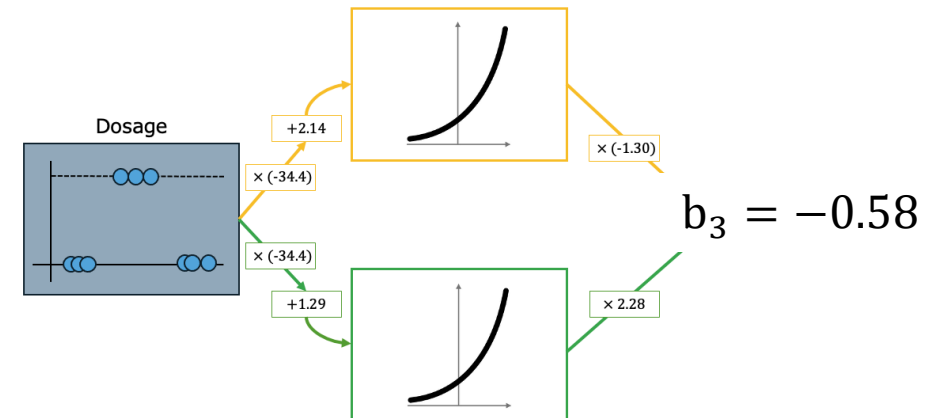
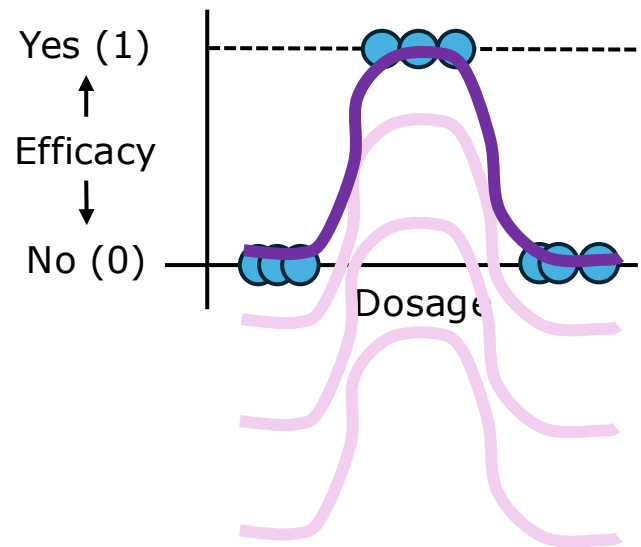
The Learning Goal



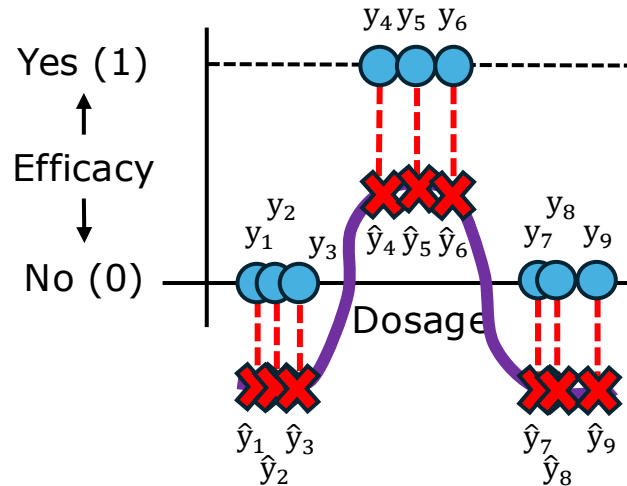
The Learning Goal



The Learning Goal



The Learning Goal



Goal: Find the best curve that represents our data

We can compute the error between our prediction $\hat{y}$ and the target y (Ground Truth):

$$L = \frac{1}{2} \sum_{i=1}^9 (y_i - \hat{y}_i)^2$$

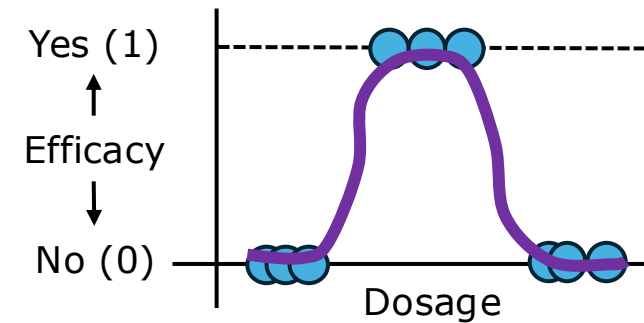
The Learning Goal

In almost every ML task:
Minimize Loss L

Example:

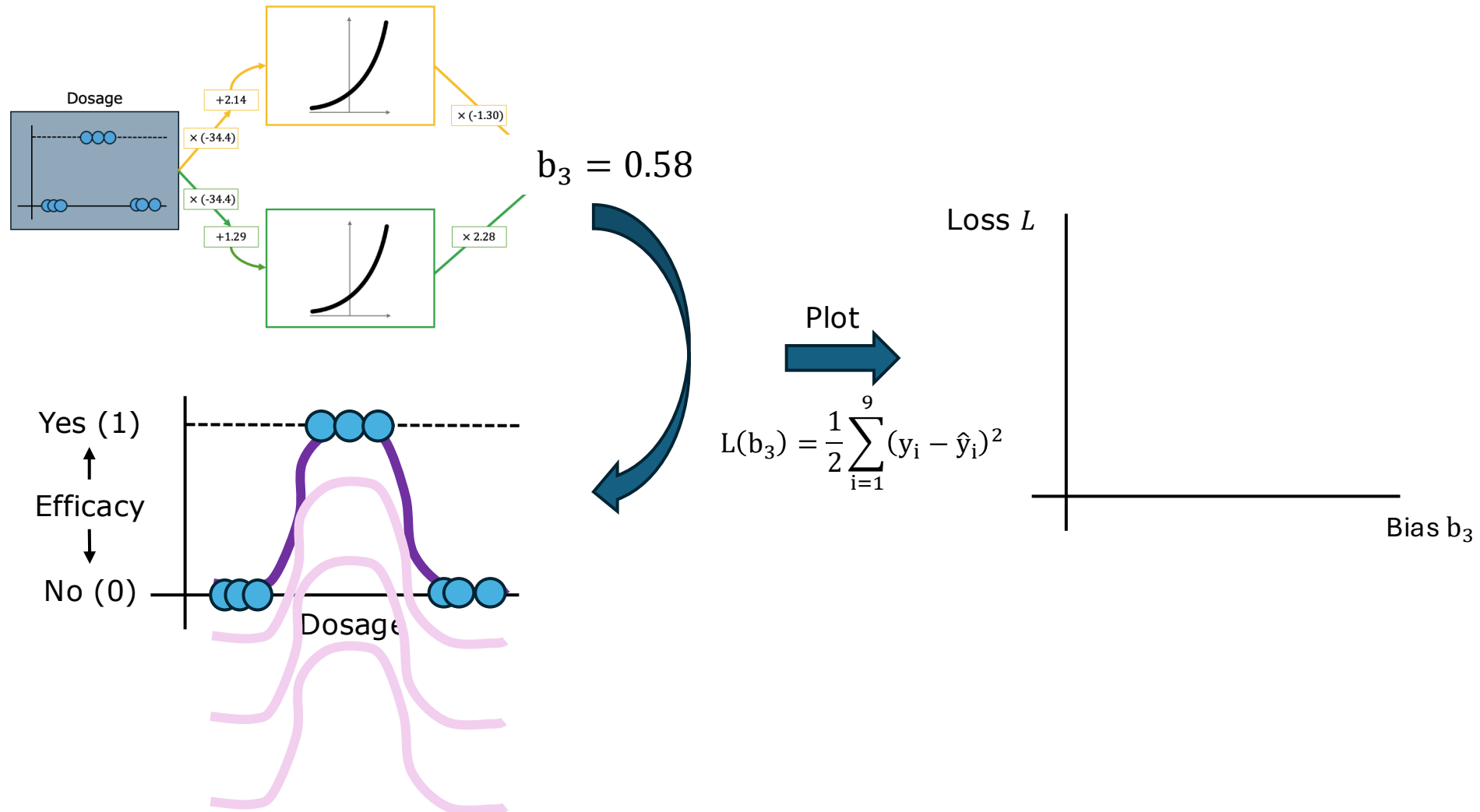
$$L = \frac{1}{2}(\mathbf{y} - \hat{\mathbf{y}})^2$$

- y = Ground Truth
- $\hat{y}$ = Prediction
- Goal: adjust w, b to minimize L

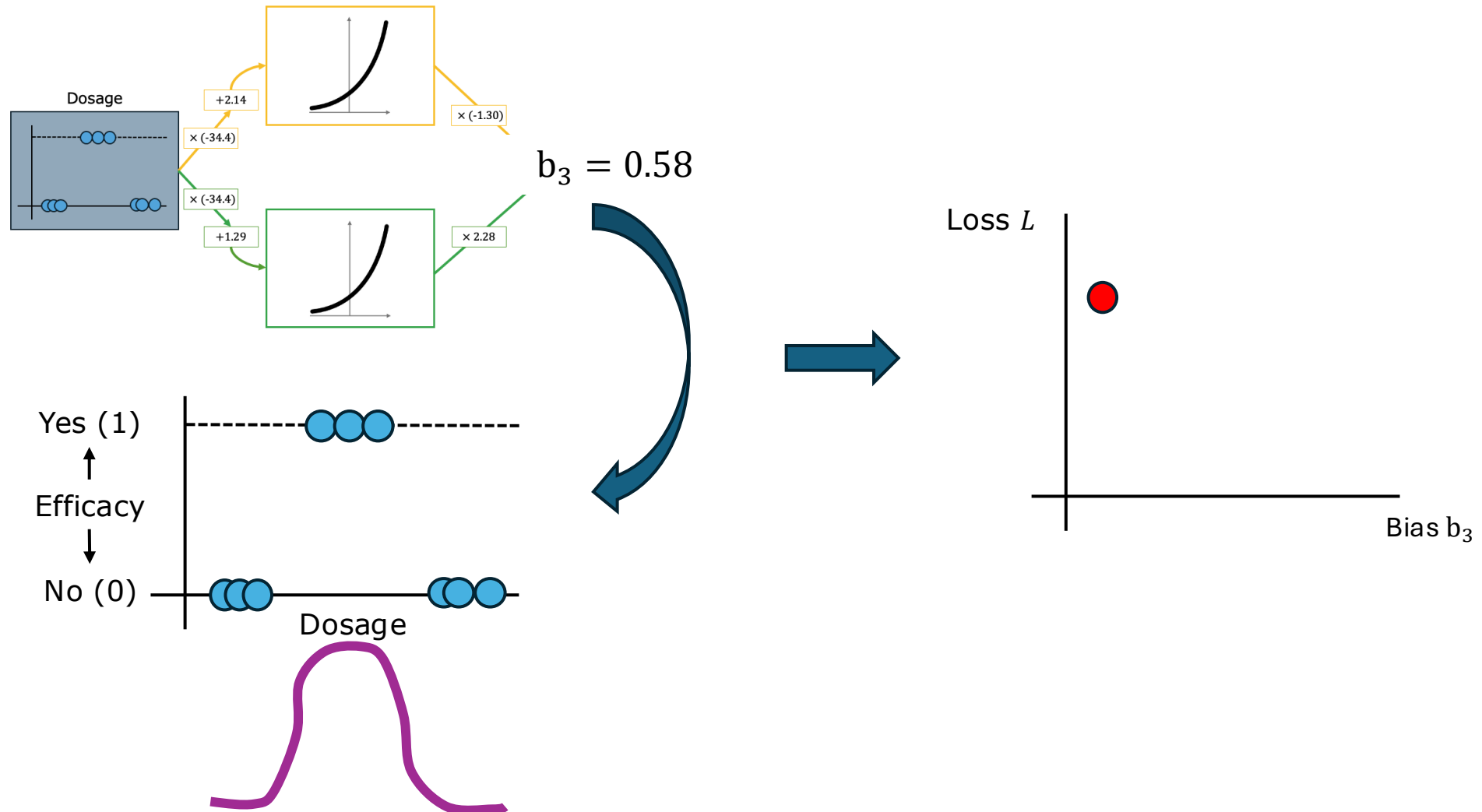


The good news is that
a **Neural Network**
can fit a **squiggle** to
the data

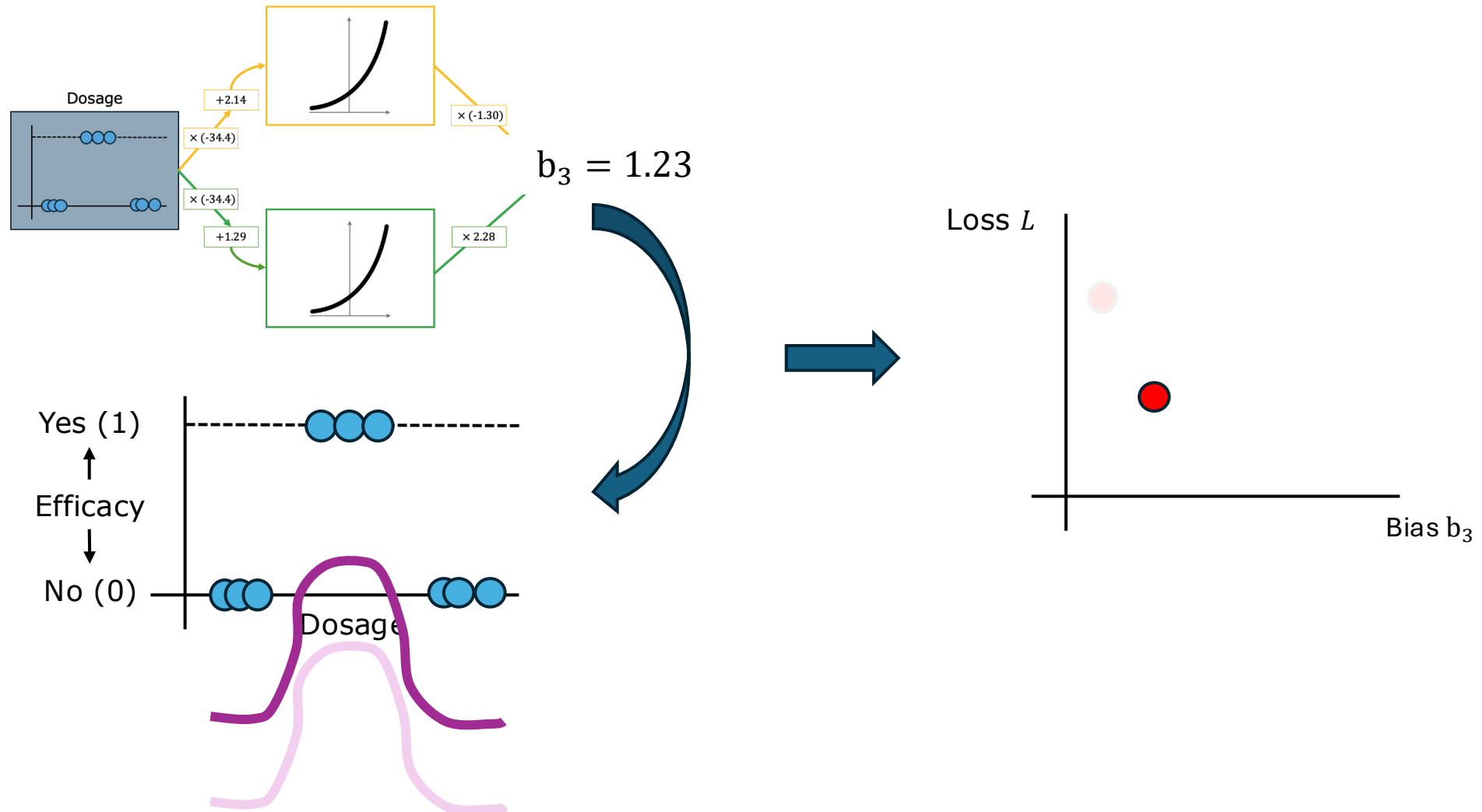
The Learning Goal



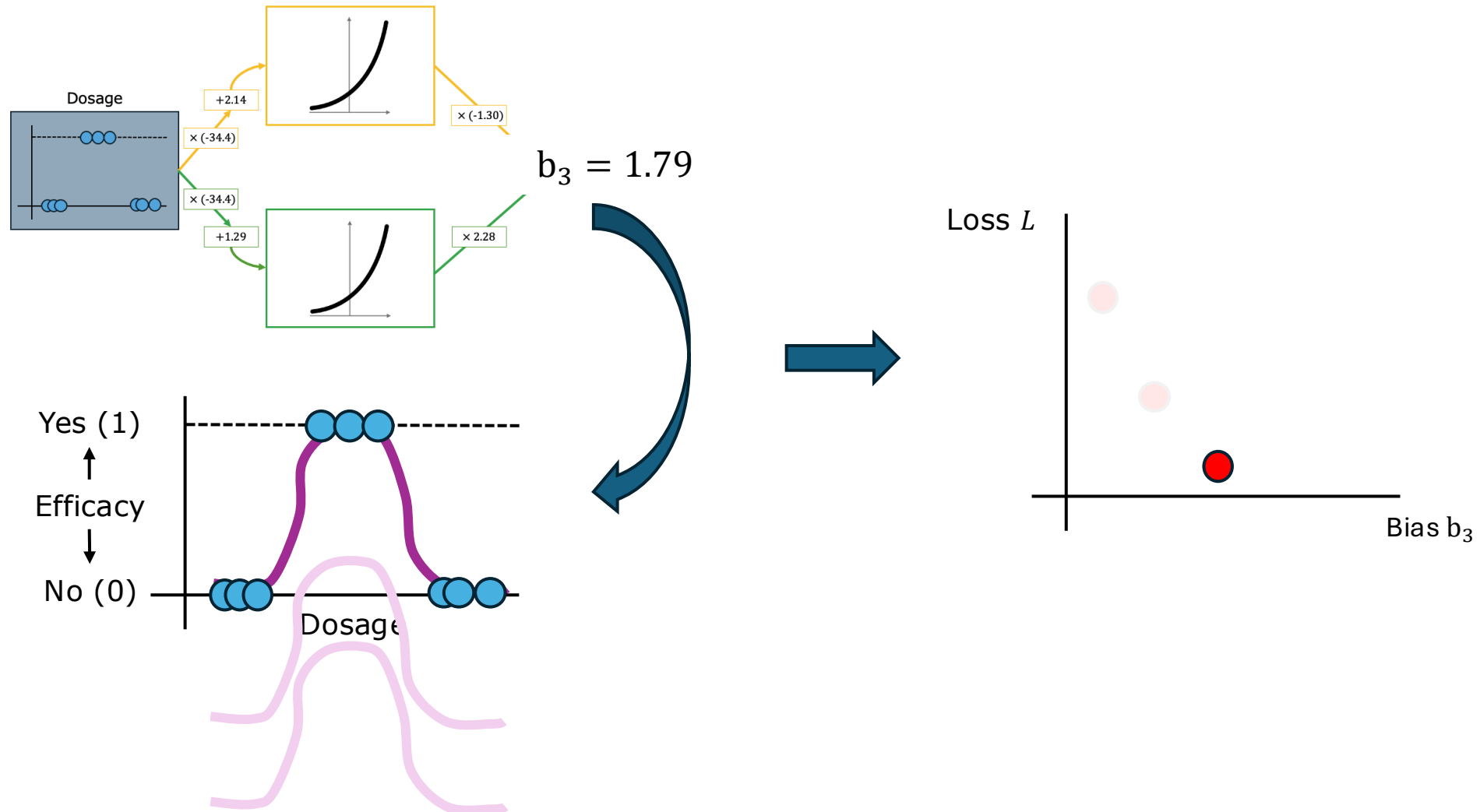
The Learning Goal



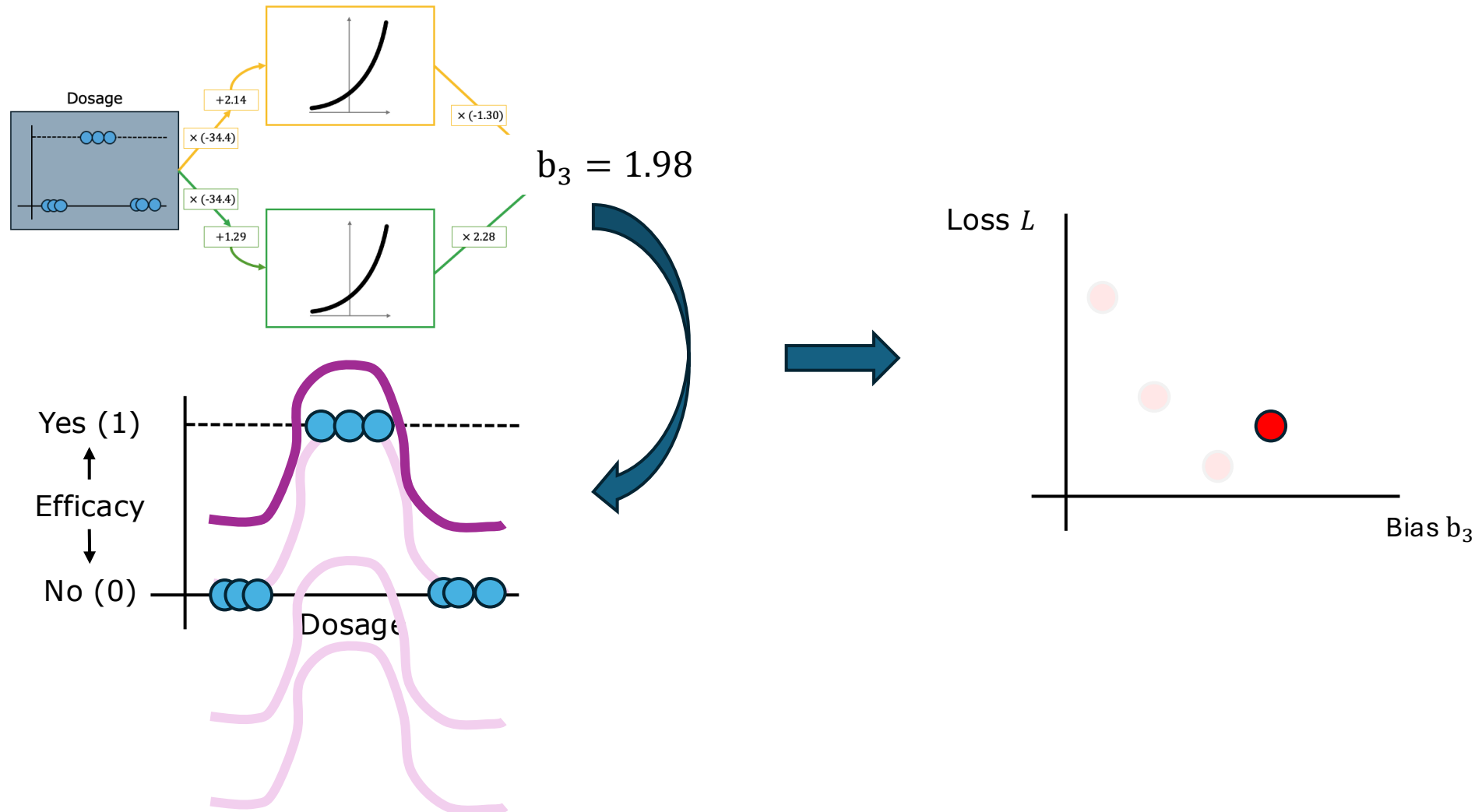
The Learning Goal



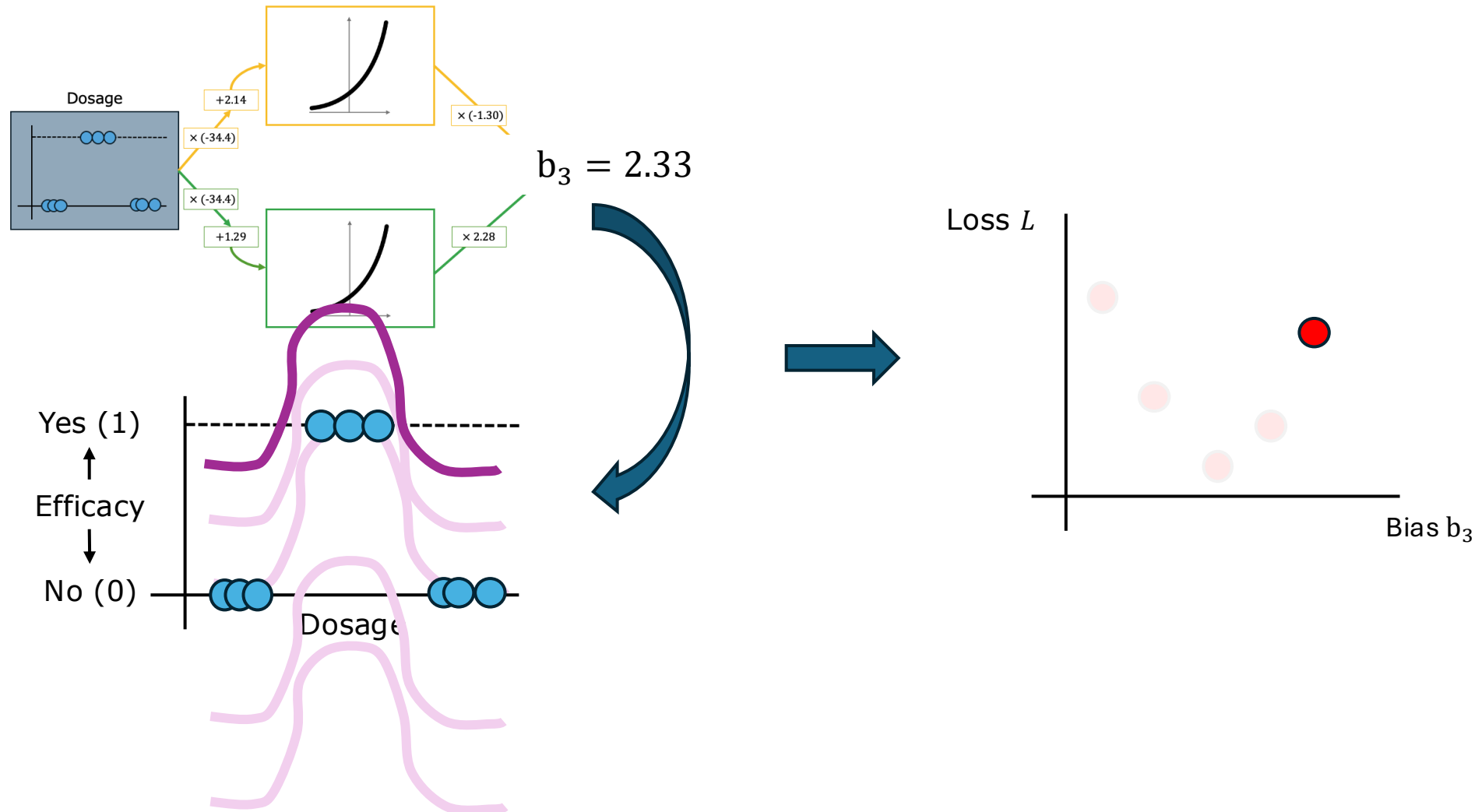
The Learning Goal



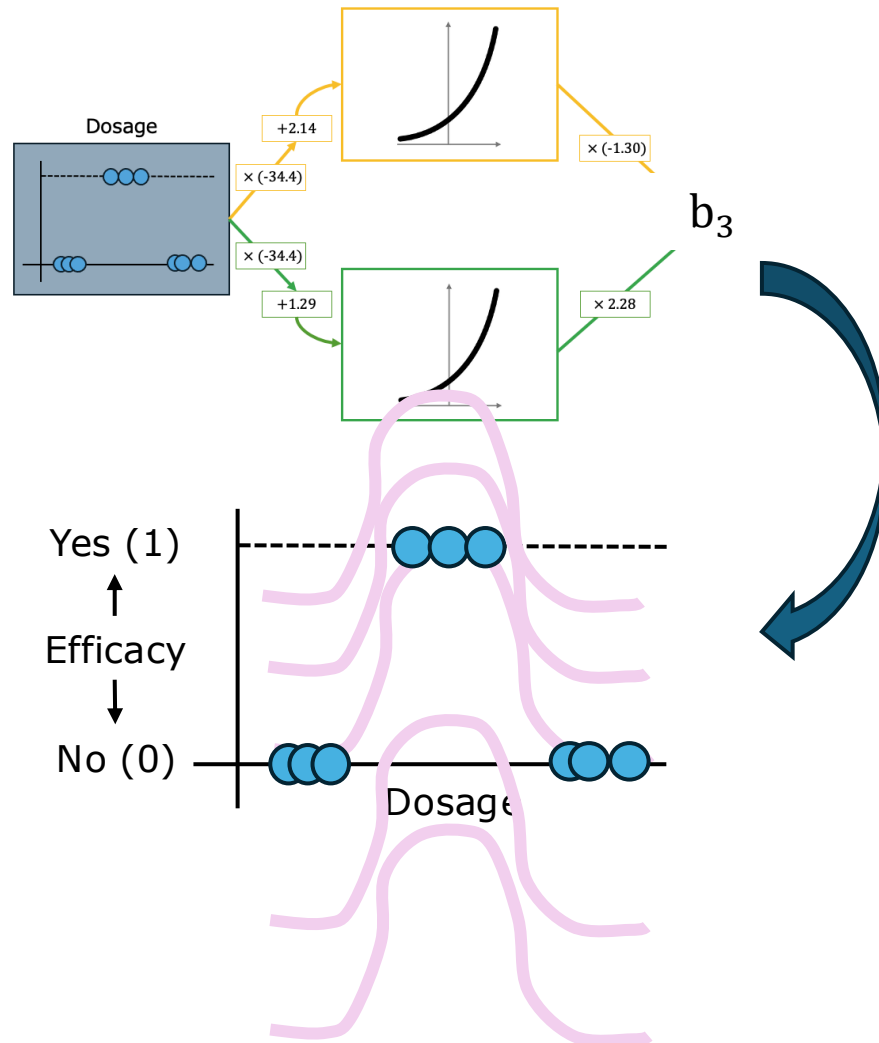
The Learning Goal



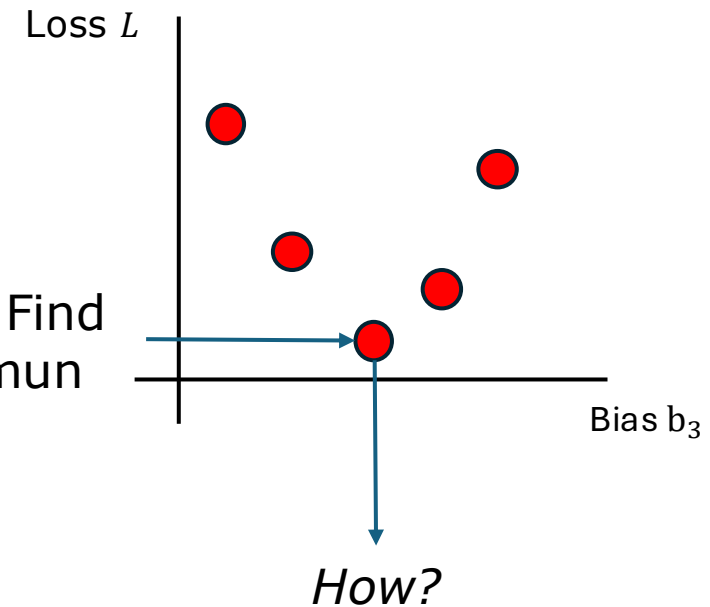
The Learning Goal



The Learning Goal



Goal: Find minimum



Agenda

- Neural Networks
- **Gradient Descent**
- Backpropagation

Gradient Descent

Update rule:

$$w := w - \eta \frac{\partial L}{\partial w}$$

- η = Learning Rate
- $\frac{\partial L}{\partial w}$ = Gradient
- Move in direction of steepest descent



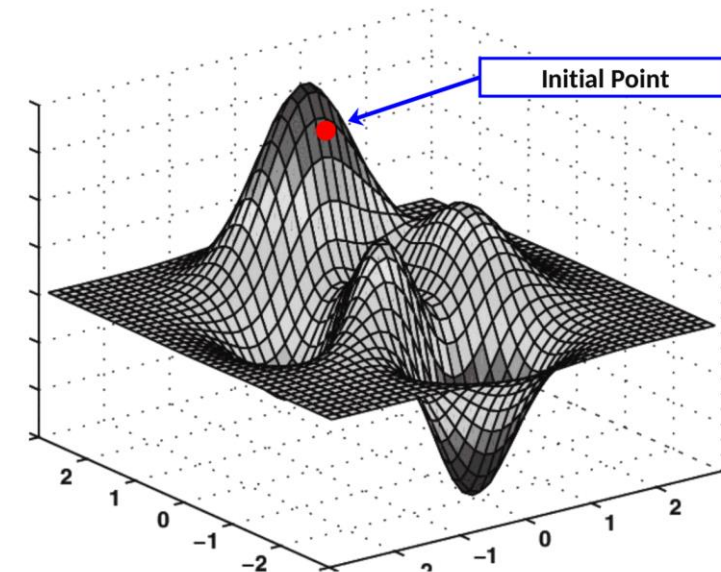
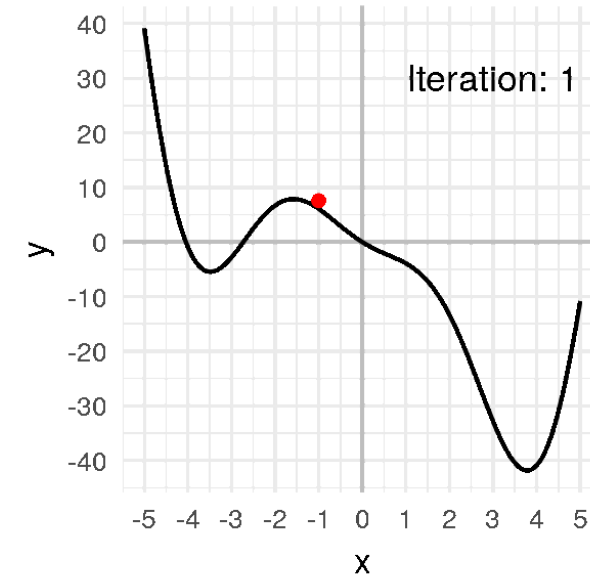
<https://www.geeksforgeeks.org/machine-learning/what-is-gradient-descent/>

Gradient Descent Illustration

Update rule:

$$w := w - \eta \frac{\partial L}{\partial w}$$

- η = Learning Rate
- $\frac{\partial L}{\partial w}$ = Gradient
- Move in direction of steepest descent



What is a Gradient?

- Gradient = slope of loss surface
- Points to direction of steepest increase
- We go in negative direction

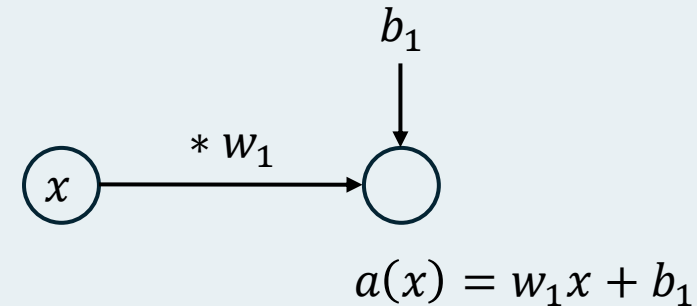
$$w := w - \eta \frac{\partial L}{\partial w}$$

But neural networks are:

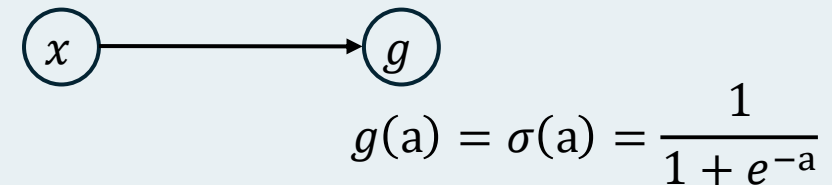
- Deep
- Nested
- Composed of many functions

How do we compute gradients efficiently?

E.g., simple neural network with input x :



This goes into an activation function $g(\cdot)$ (e.g. ReLU, Sigmoid, ...):



What is a Gradient?

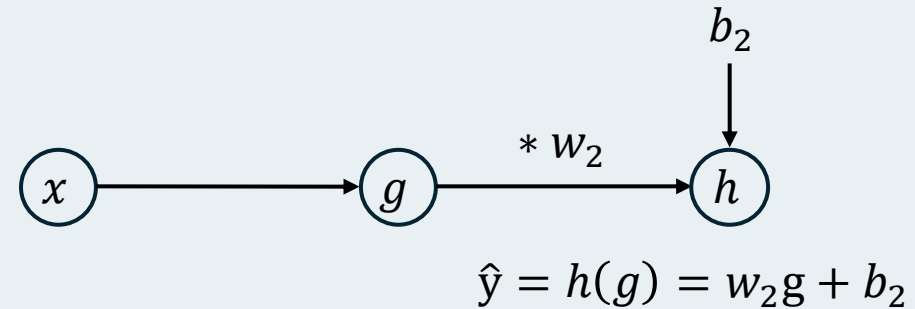
- Gradient = slope of loss surface
- Points to direction of steepest increase
- We go in negative direction

$$w := w - \eta \frac{\partial L}{\partial w}$$

But neural networks are:

- Deep
- Nested
- Composed of many functions

How do we compute gradients efficiently?



If we wrap this up, we'll get this function for **prediction**:

$$\hat{y} = h(s) = h(g(a(x))) = w_2 g(w_1 x + b_1) + b_2$$

Nested:

$$\hat{y}(x) = w_2 g(w_1 x + b_1) + b_2$$

Composed:

$$\hat{y}(x) = (h \circ g \circ a)(x)$$

What is a Gradient?

We are still interested in this:

$$w := w - \eta \frac{\partial L}{\partial w}$$

and have this now as a prediction:

$$\hat{y}(x) = (h \circ g \circ a)(x)$$

We also know:

$$L(\hat{y}, y) = \frac{1}{2} (y - \hat{y})^2$$

Which becomes to:

$$L(x, y) = \frac{1}{2} (y - (w_2, g(w_1 x + b_1) + b_2))^2$$

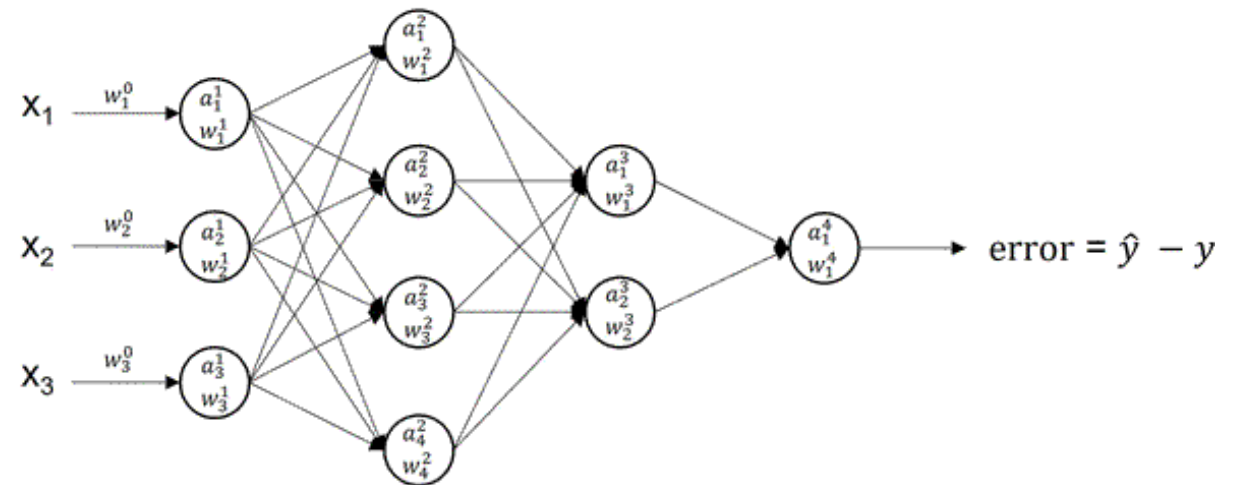
Chain Rule

How do we compute the gradient?

By using the **Chain Rule!**

$$\begin{aligned}
 & \left[\mathbf{f}(\mathbf{g}(\mathbf{h}(\mathbf{i}(\mathbf{j}(\mathbf{x})))) \right]' \\
 &= \\
 & \mathbf{f}'(\mathbf{g}(\mathbf{h}(\mathbf{i}(\mathbf{j}(\mathbf{x})))) \\
 & \quad * \mathbf{g}'(\mathbf{h}(\mathbf{i}(\mathbf{j}(\mathbf{x})))) \\
 & \quad \quad * \mathbf{h}'(\mathbf{i}(\mathbf{j}(\mathbf{x}))) \\
 & \quad \quad \quad * \mathbf{i}'(\mathbf{j}(\mathbf{x})) \\
 & \quad \quad \quad \quad * \mathbf{j}'(\mathbf{x}) \\
 & \quad \quad \quad \quad \quad * 1
 \end{aligned}$$


https://en.wikipedia.org/wiki/Chain_rule



Agenda

- Neural Networks
- Gradient Descent
- **Backpropagation**

Simple Chain Rule Example

Let:

$$\begin{aligned}g(x) &= 3x + 1 \\f(u) &= u^2\end{aligned}$$

Then our full function is:

$$y = f(g(x)) = (3x + 1)^2$$

→ **Key idea: multiply local derivatives**

So we have two derivatives:

$$\begin{aligned}\frac{dy}{du} &= 2u \\ \frac{du}{dx} &= 3\end{aligned}$$

Chain Rule Formula

$$\boxed{\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}}$$

Now substitute:

$$\frac{dy}{dx} = (2u) \cdot 3$$

Replace $u = 3x + 1$:

$$\begin{aligned}\frac{dy}{dx} &= 2(3x + 1) \cdot 3 \\ &= 6(3x + 1)\end{aligned}$$

Important

- The outer function changes with respect to the inner function
- The inner function changes with respect to x

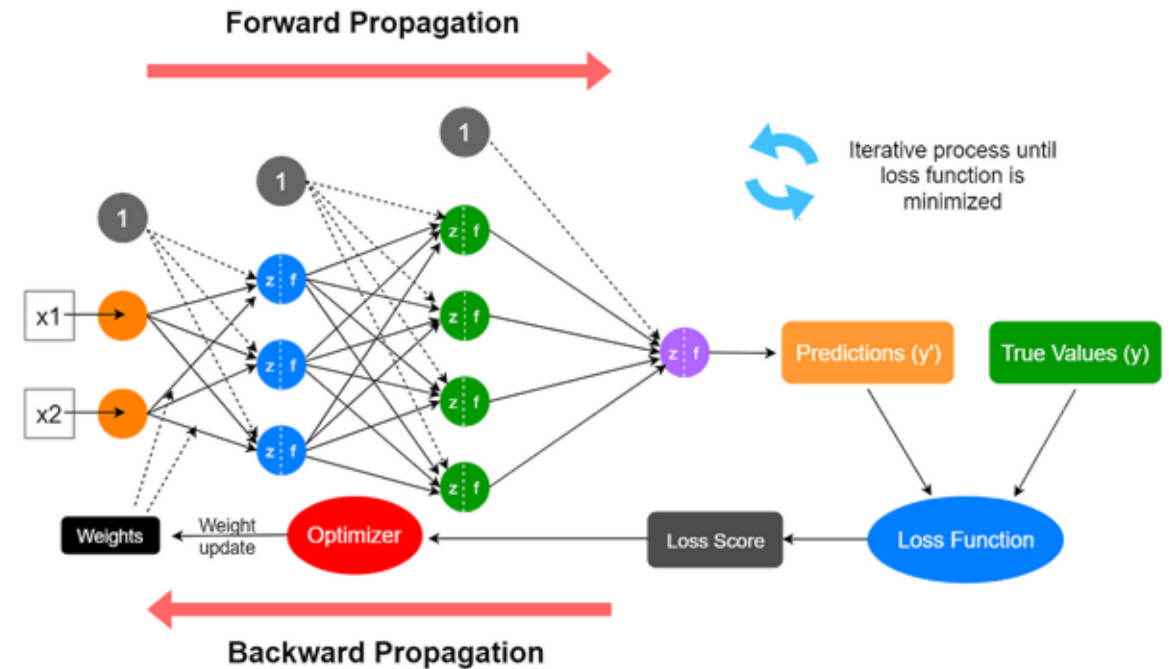
The total change = *outer change* $\times$ *inner change*

→ You multiply the sensitivities

Forward & Backward Pass

Each node:

- Stores value (forward pass)
- Stores derivative (backward pass)

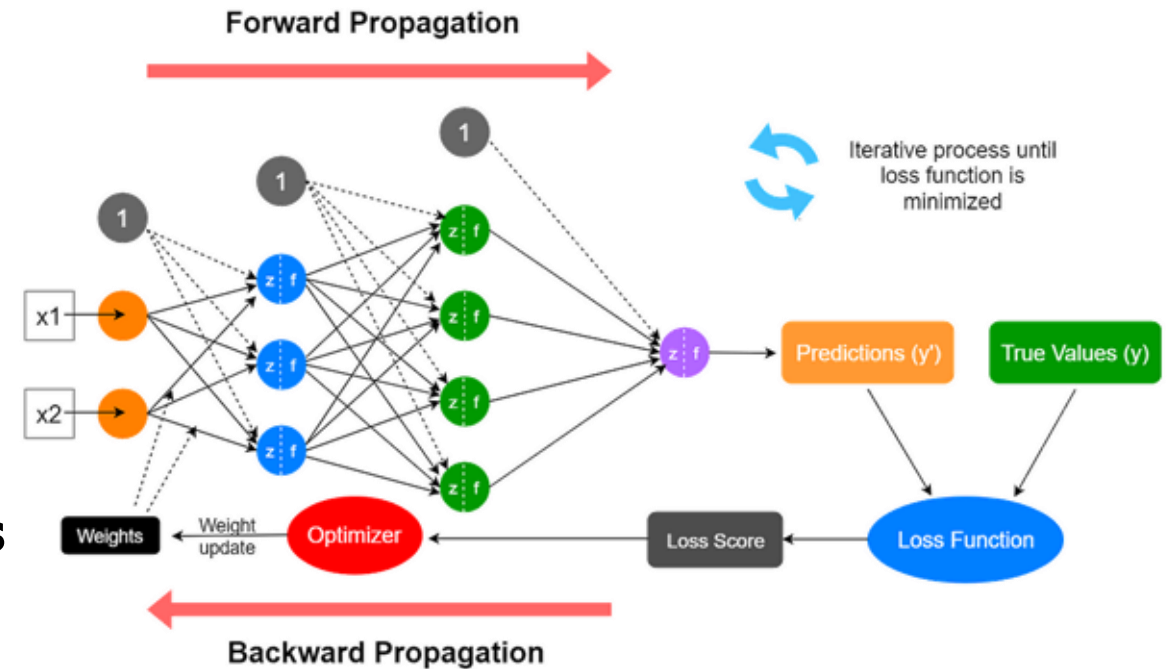


<https://wandb.ai/mostafaibrahim17/ml-articles/reports/Decoding-Backpropagation-and-Its-Role-in-Neural-Network-Learning--Vmlldzo3MTY5MzM1>

Forward Pass

1. Initialize random w, b
2. Compute:
 - $z = w^T x + b$
 - $a = \sigma(z)$
 - $\hat{y}$
3. Compute loss

We now have **all intermediate values**



<https://wandb.ai/mostafaibrahim17/ml-articles/reports/Decoding-Backpropagation-and-Its-Role-in-Neural-Network-Learning--Vmlldzo3MTY5MzM1>

Backward Pass

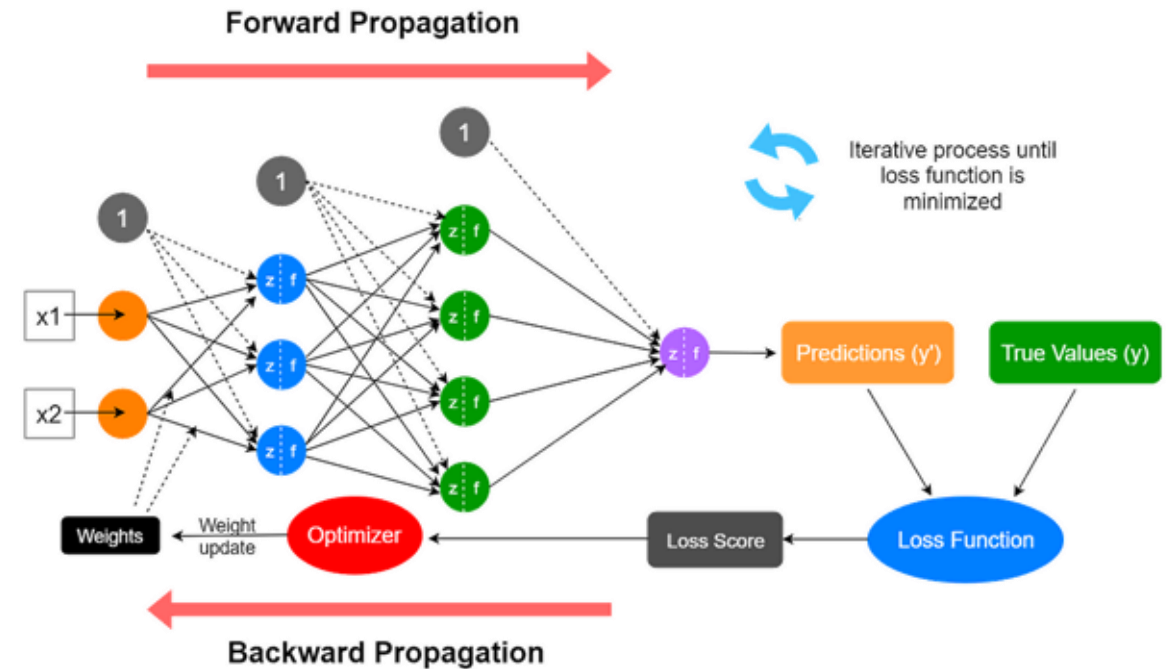
We compute:

$$\frac{\partial L}{\partial w}$$

Using chain rule:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w}$$

→ We move backwards layer by layer



<https://wandb.ai/mostafaibrahim17/ml-articles/reports/Decoding-Backpropagation-and-Its-Role-in-Neural-Network-Learning--Vmlldzo3MTY5MzM1>

Why Backprop Is Efficient

Without backprop:

- Recompute many partial derivatives repeatedly

With backprop:

- Reuse intermediate derivatives
- Dynamic programming
- Complexity proportional to forward pass

→ Cost $\approx 2 \times$ forward pass

Matrix Form

For one layer:

$$Z = W X + b$$

$$A = \sigma(Z)$$

Backward:

$$dW = \delta X^T$$

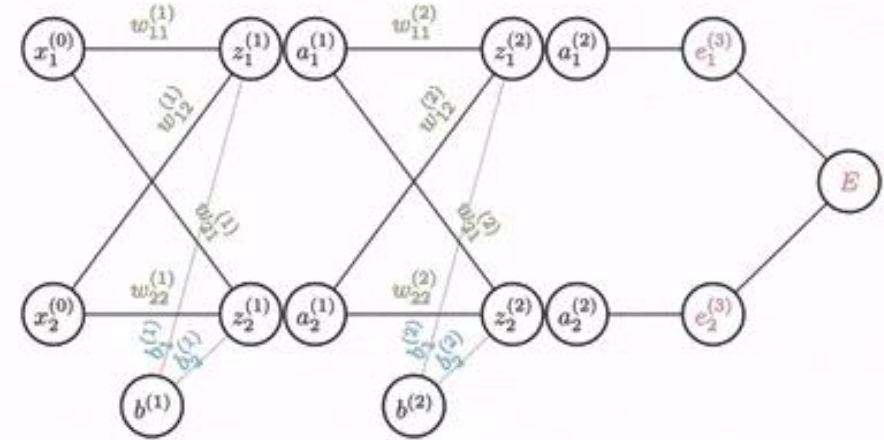
$$\delta = dA \odot \sigma'(Z)$$

Where:

- $\odot$ = elementwise product
- Efficient vectorized implementation

$$\frac{\partial E}{\partial w_{11}^{(2)}} = \frac{\partial e_1^{(3)}}{\partial a_1^{(2)}} \frac{\partial a_1^{(2)}}{\partial z_1^{(2)}} \frac{\partial z_1^{(2)}}{\partial w_{11}^{(2)}} \quad \frac{\partial E}{\partial w_{12}^{(2)}} = \frac{\partial e_1^{(3)}}{\partial a_1^{(2)}} \frac{\partial a_1^{(2)}}{\partial z_1^{(2)}} \frac{\partial z_1^{(2)}}{\partial w_{12}^{(2)}}$$

$$\frac{\partial E}{\partial w_{21}^{(2)}} = \frac{\partial e_2^{(3)}}{\partial a_2^{(2)}} \frac{\partial a_2^{(2)}}{\partial z_2^{(2)}} \frac{\partial z_2^{(2)}}{\partial w_{21}^{(2)}} \quad \frac{\partial E}{\partial w_{22}^{(2)}} = \frac{\partial e_2^{(3)}}{\partial a_2^{(2)}} \frac{\partial a_2^{(2)}}{\partial z_2^{(2)}} \frac{\partial z_2^{(2)}}{\partial w_{22}^{(2)}}$$

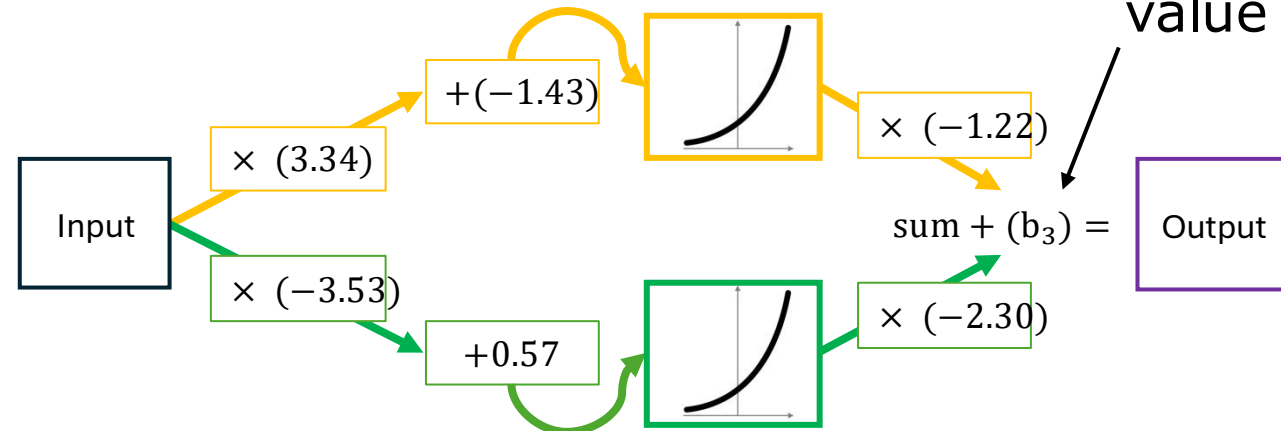
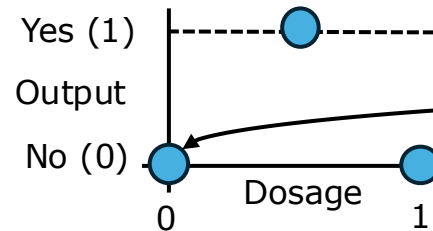


$$\begin{bmatrix} \frac{\partial E}{\partial w_{11}^{(2)}} & \frac{\partial E}{\partial w_{12}^{(2)}} \\ \frac{\partial E}{\partial w_{21}^{(2)}} & \frac{\partial E}{\partial w_{22}^{(2)}} \end{bmatrix} = \underbrace{\begin{bmatrix} \frac{\partial e_1^{(3)}}{\partial a_1^{(2)}} \\ \frac{\partial e_2^{(3)}}{\partial a_2^{(2)}} \end{bmatrix} \odot \begin{bmatrix} \frac{\partial a_1^{(2)}}{\partial z_1^{(2)}} \\ \frac{\partial a_2^{(2)}}{\partial z_2^{(2)}} \end{bmatrix}}_{\delta^L} \odot \begin{bmatrix} \frac{\partial z_1^{(2)}}{\partial w_{11}^{(2)}} & \frac{\partial z_1^{(2)}}{\partial w_{12}^{(2)}} \\ \frac{\partial z_2^{(2)}}{\partial w_{21}^{(2)}} & \frac{\partial z_2^{(2)}}{\partial w_{22}^{(2)}} \end{bmatrix}$$

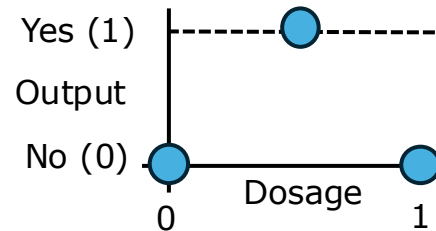
$$\frac{\partial E}{\partial W^{(2)}} = \delta^L \odot \frac{\partial Z^{(2)}}{\partial W^{(2)}}$$

Backpropagation Example

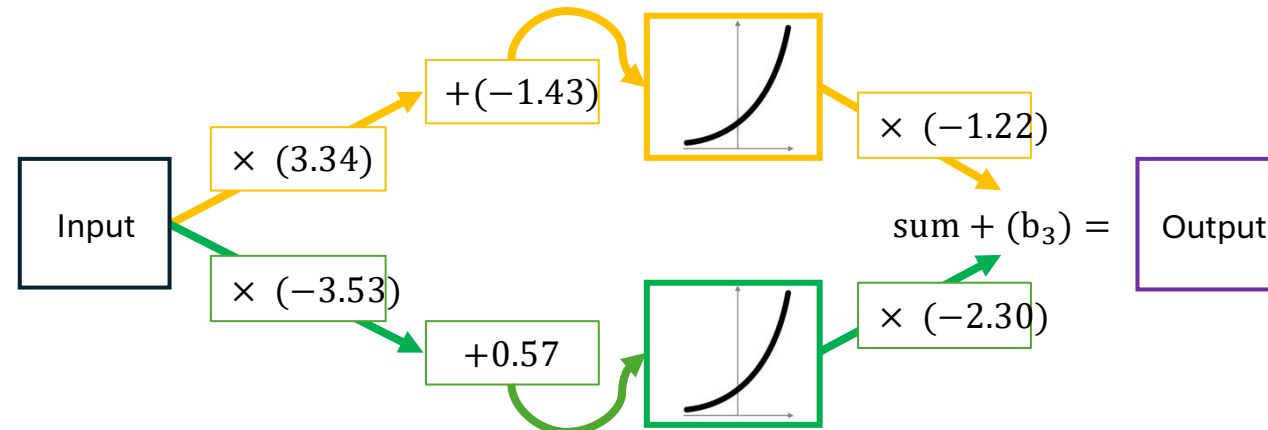
Our **Dosage** example again: Let's keep it simple with 3 data points and let's assume Dosages go from **0** (low) to **1** (high)



Backpropagation Example

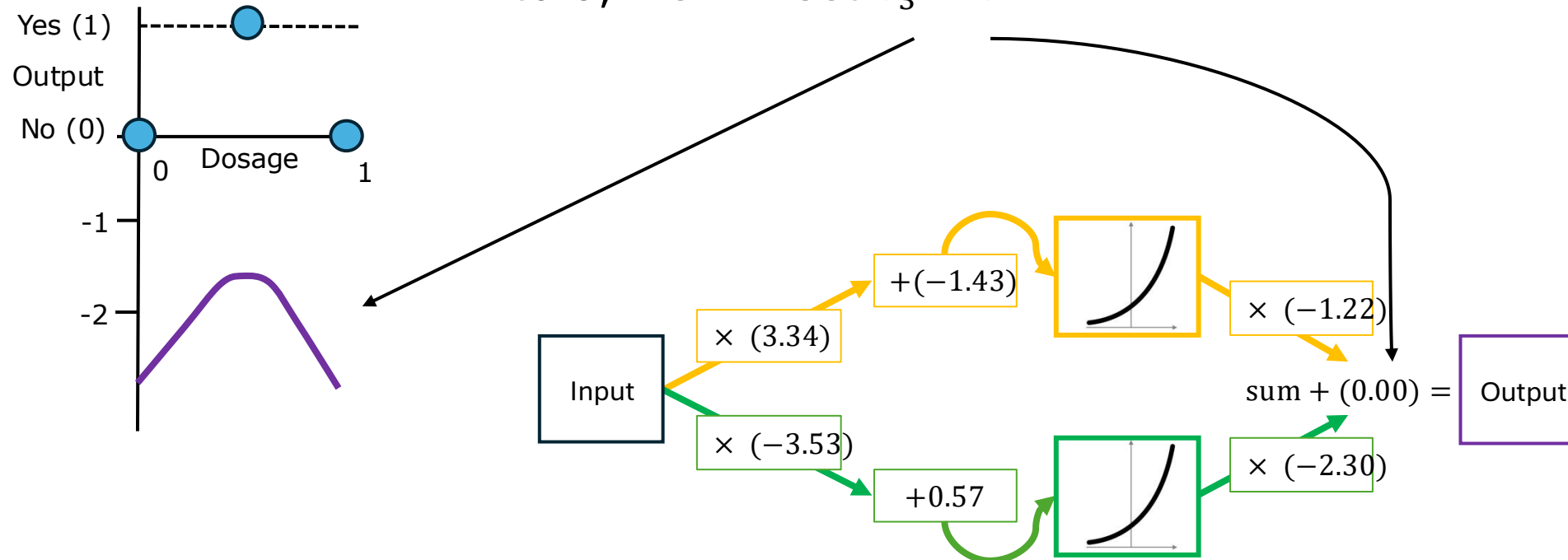


Because Bias terms are frequently initialized to 0, we will set $b_3 = 0$



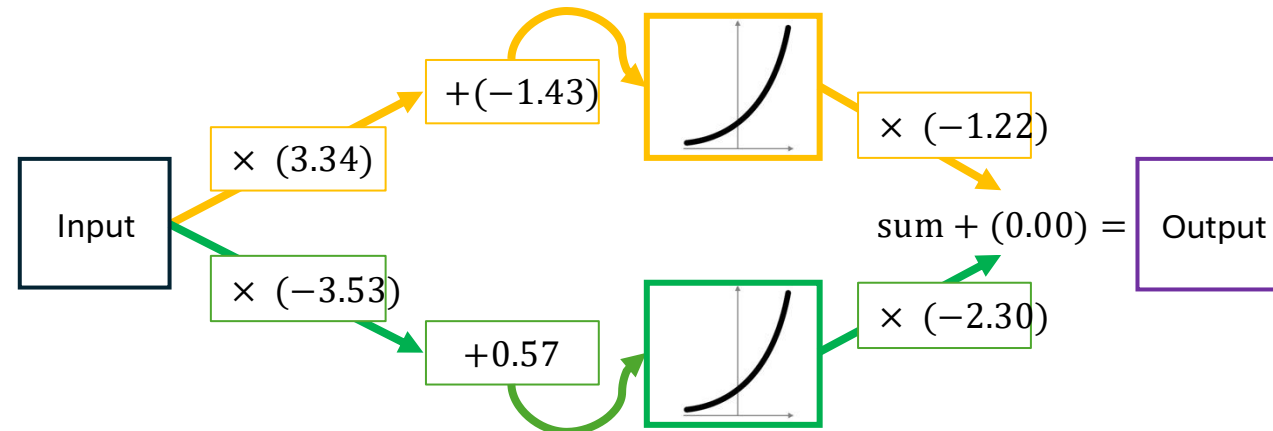
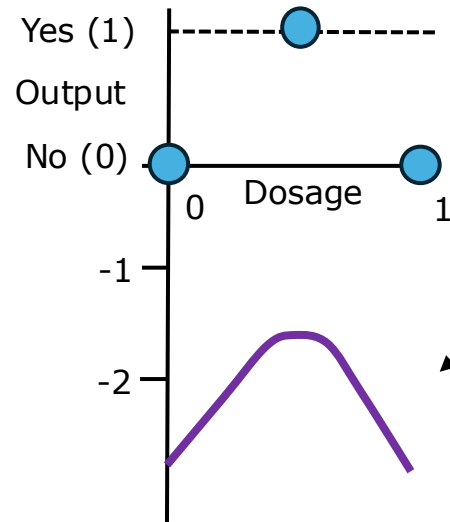
Backpropagation Example

Because Bias terms are frequently initialized to 0, we will set $b_3 = 0$



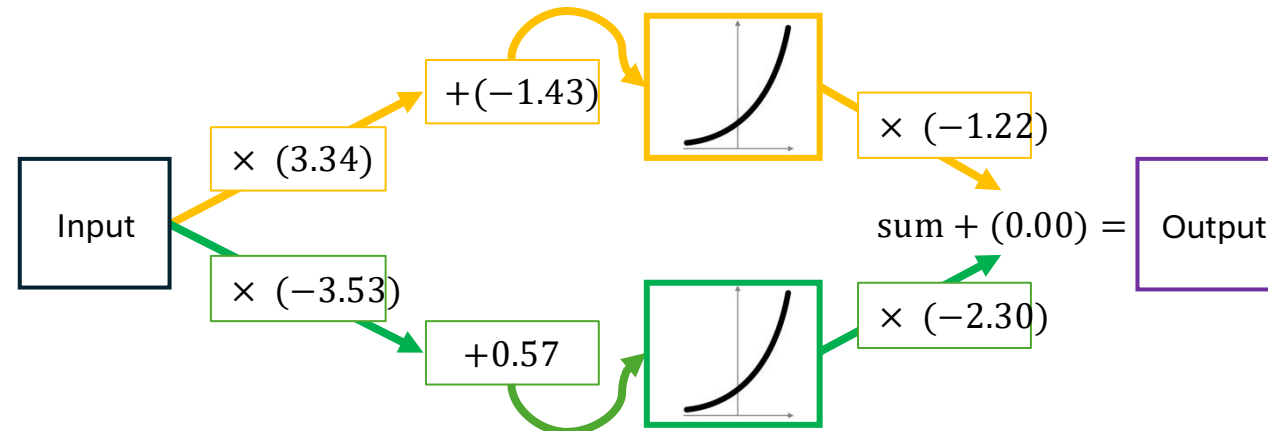
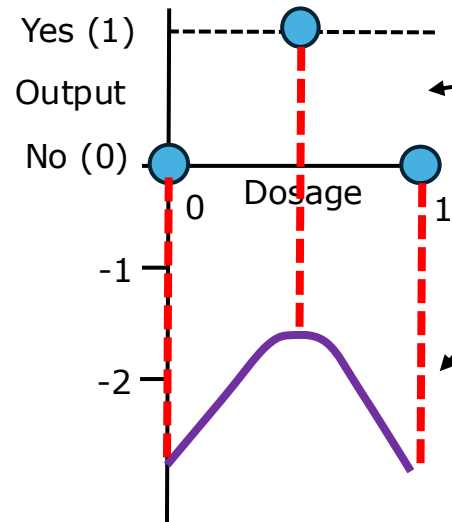
Backpropagation Example

... the **purple squiggle** is pretty far from the data that we observed

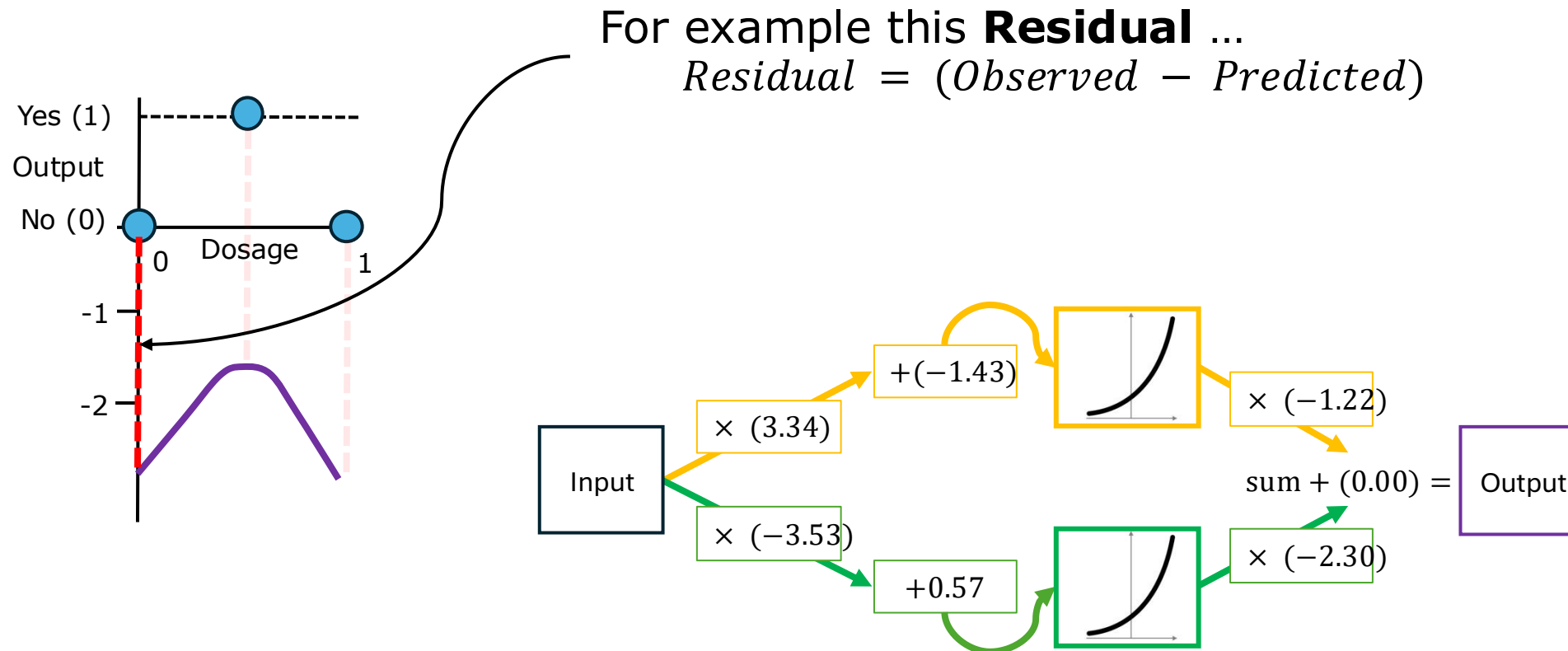


Backpropagation Example

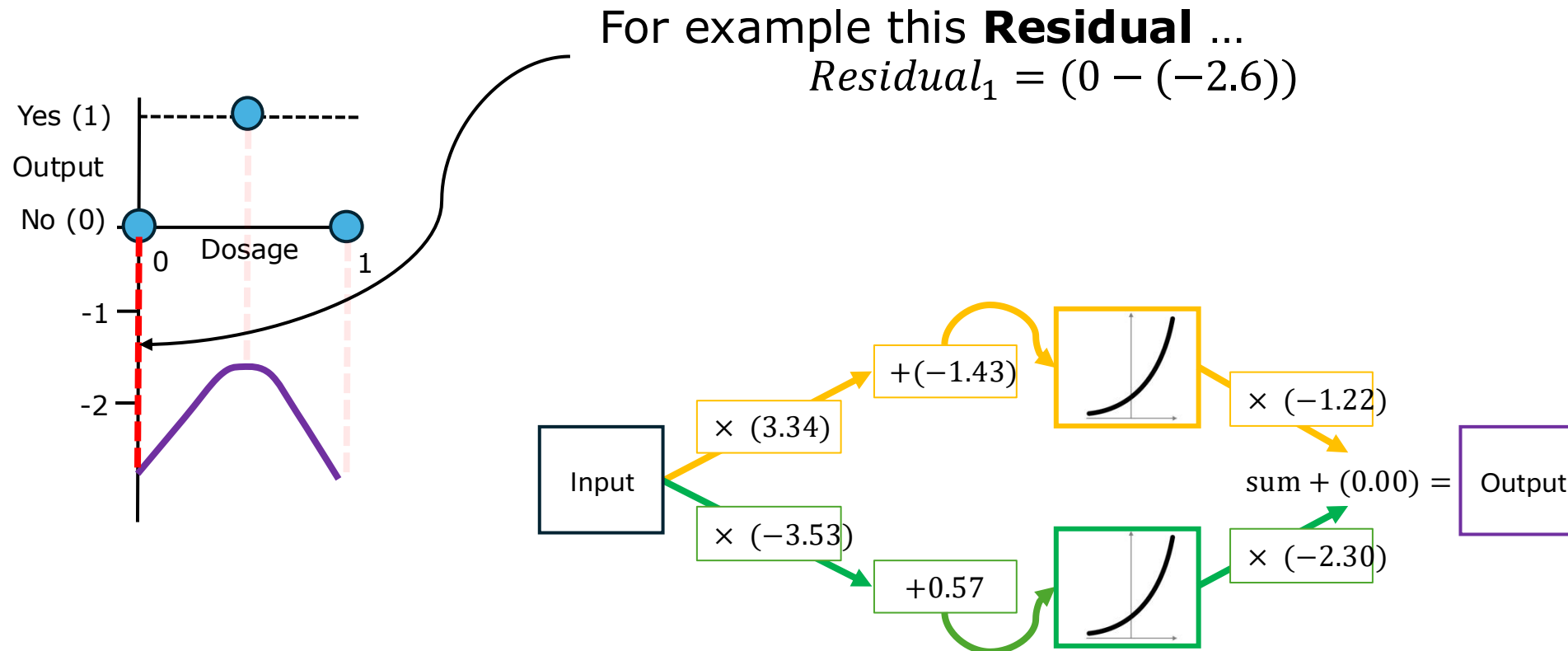
We can quantify how good the **purple squiggle** fits the data by calculating the **Sum of the Squared Residuals**



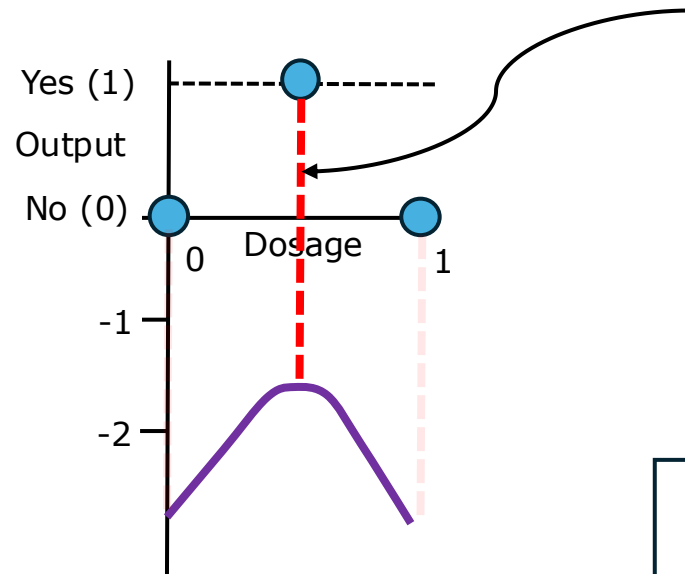
Backpropagation Example



Backpropagation Example



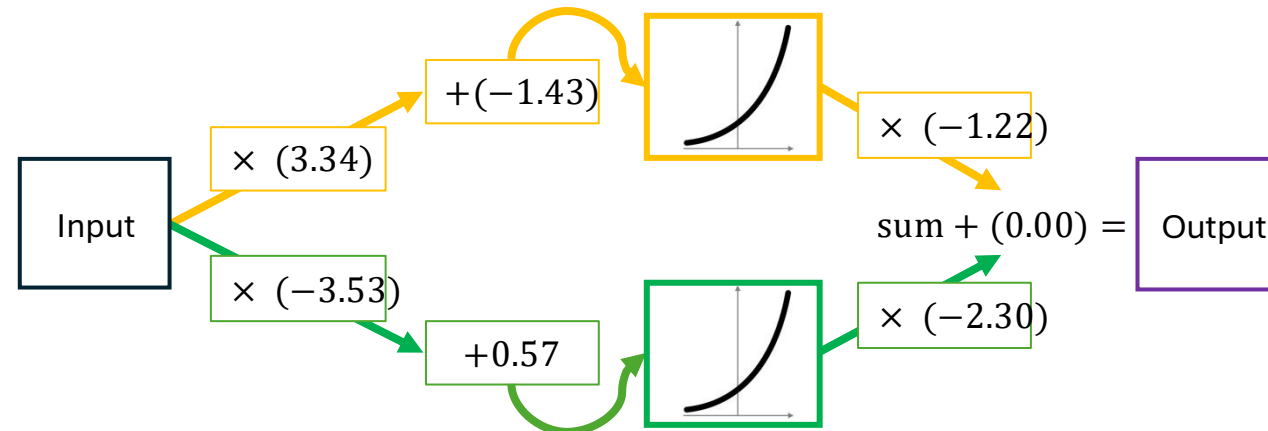
Backpropagation Example



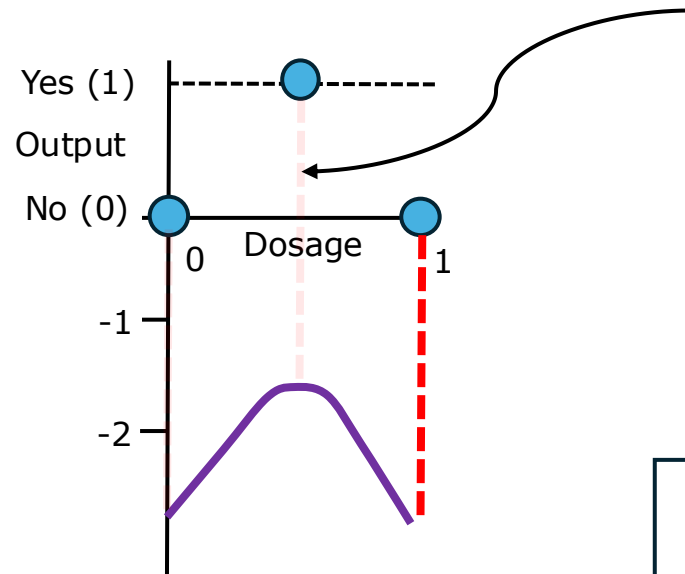
For example this **Residual** ...

$$Residual_1 = (0 - (-2.6))$$

$$Residual_2 = (1 - (-1.6))$$



Backpropagation Example

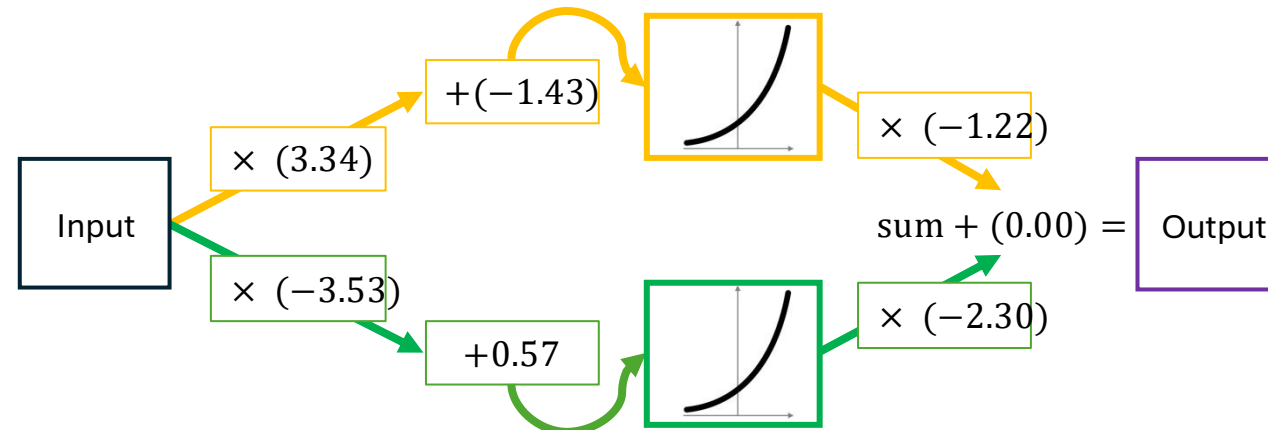


For example this **Residual** ...

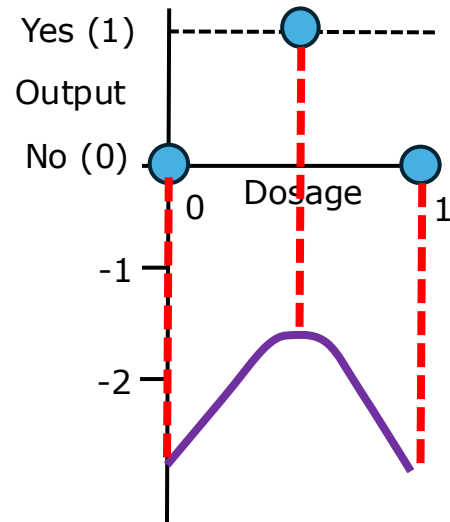
$$Residual_1 = (0 - (-2.6))$$

$$Residual_2 = (1 - (-1.6))$$

$$Residual_3 = (0 - (-2.6))$$

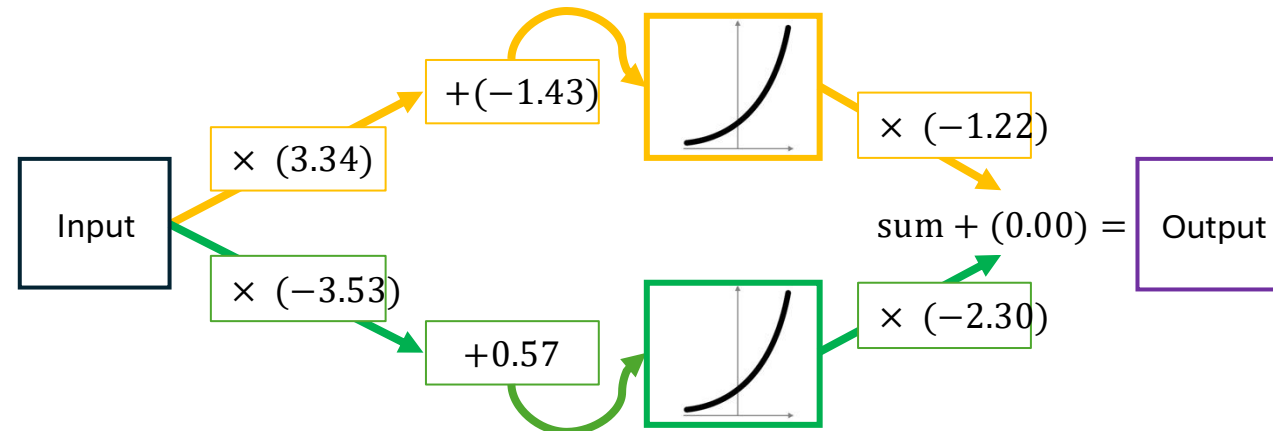


Backpropagation Example

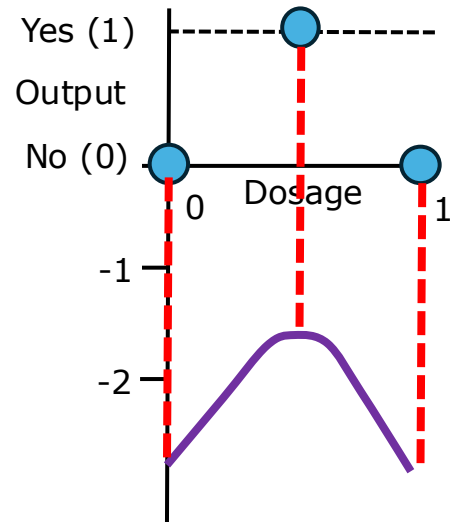


Now we square each **Residual** ...

$$Residual = (0 - (-2.6))^2 + (1 - (-1.6))^2 + (0 - (-2.6))^2$$

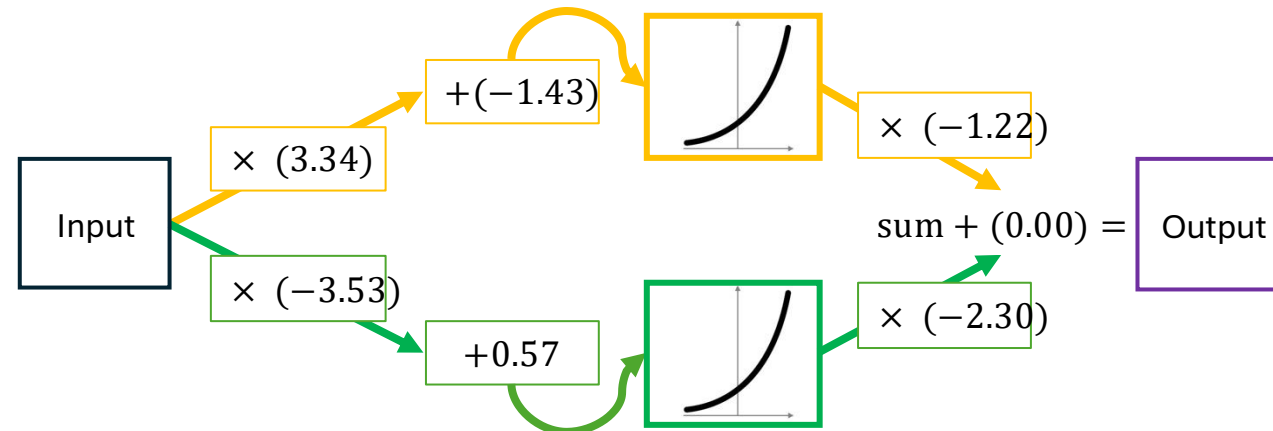


Backpropagation Example



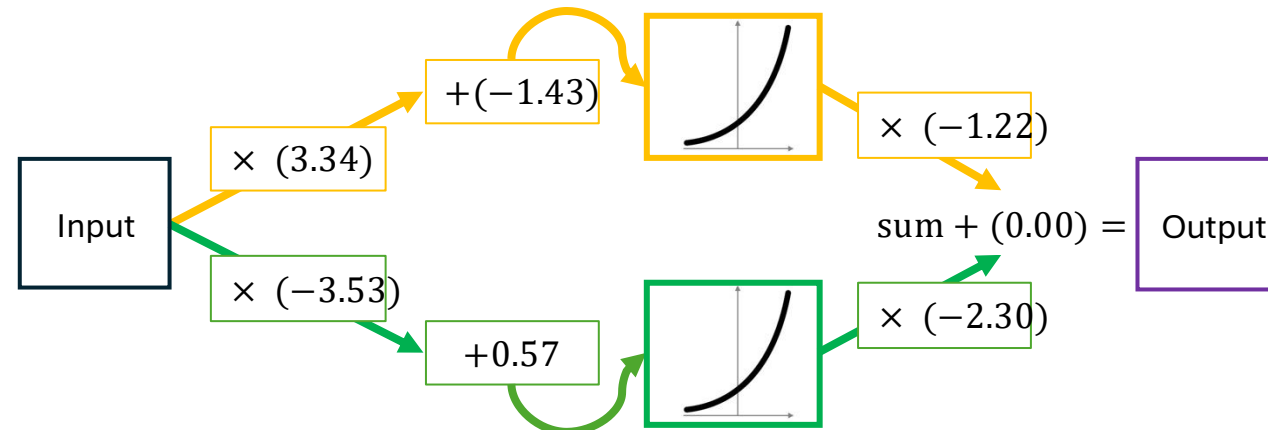
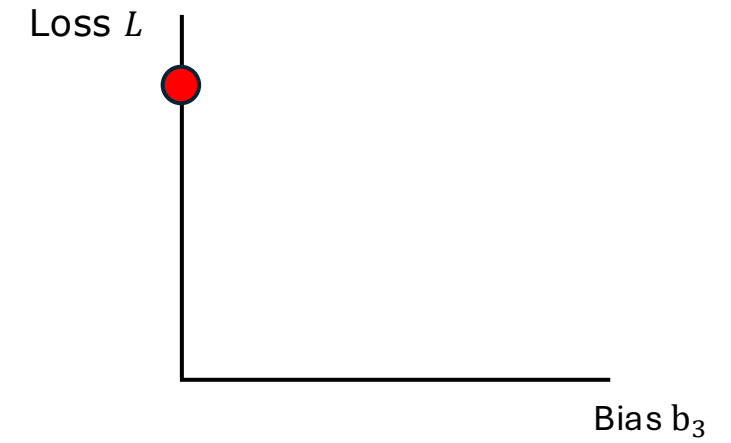
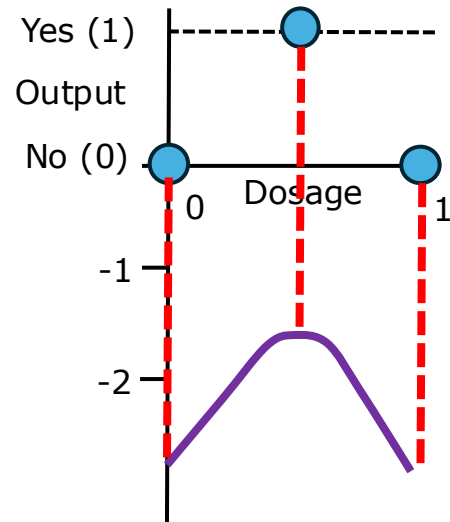
... to get **20.4** for the **Sum of the Squared Residuals (L)**:

$$\text{Residual} = (0 - (-2.6))^2 + (1 - (-1.6))^2 + (0 - (-2.6))^2 = 20.4$$

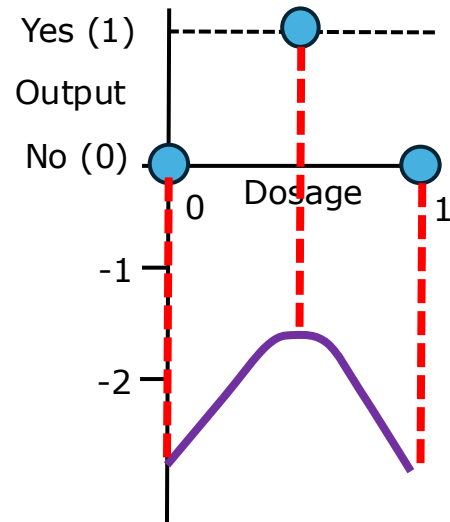


Backpropagation Example

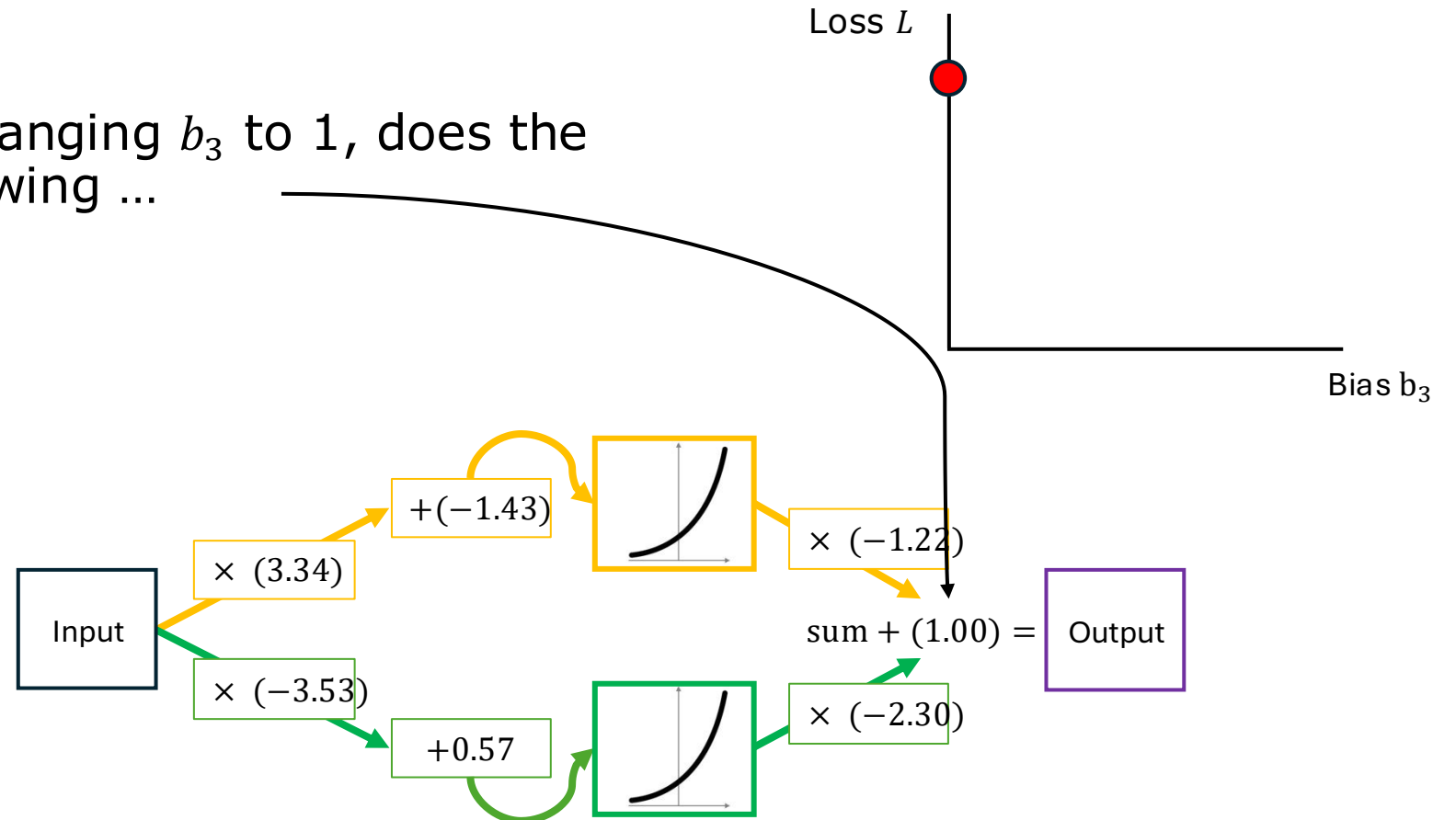
... plot the **20.4** for L on the y-axis



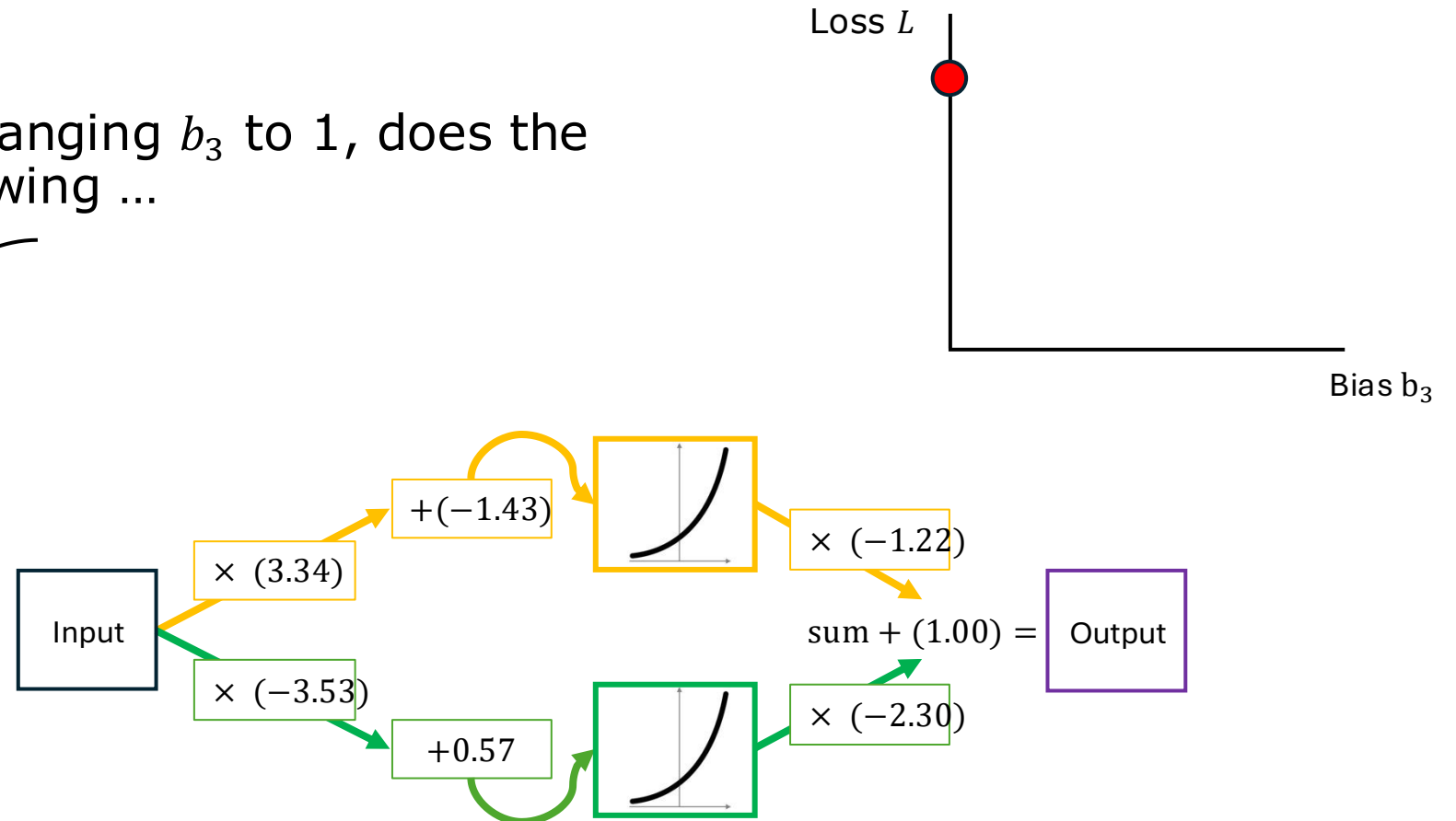
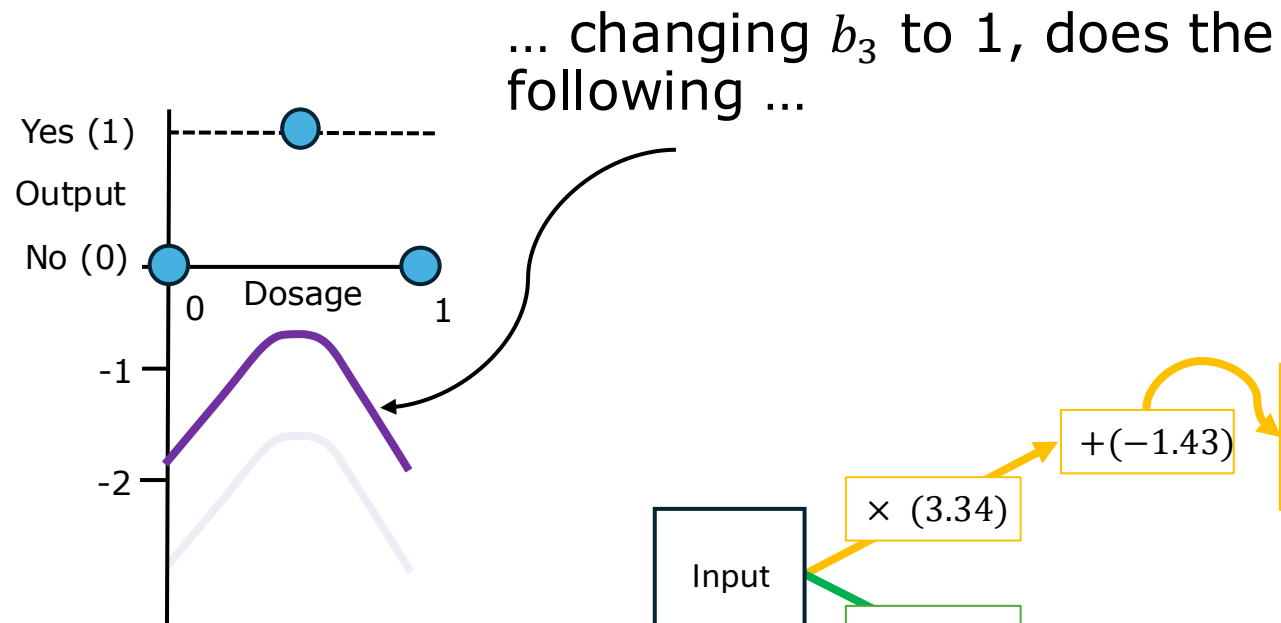
Backpropagation Example



... changing b_3 to 1, does the following ...



Backpropagation Example

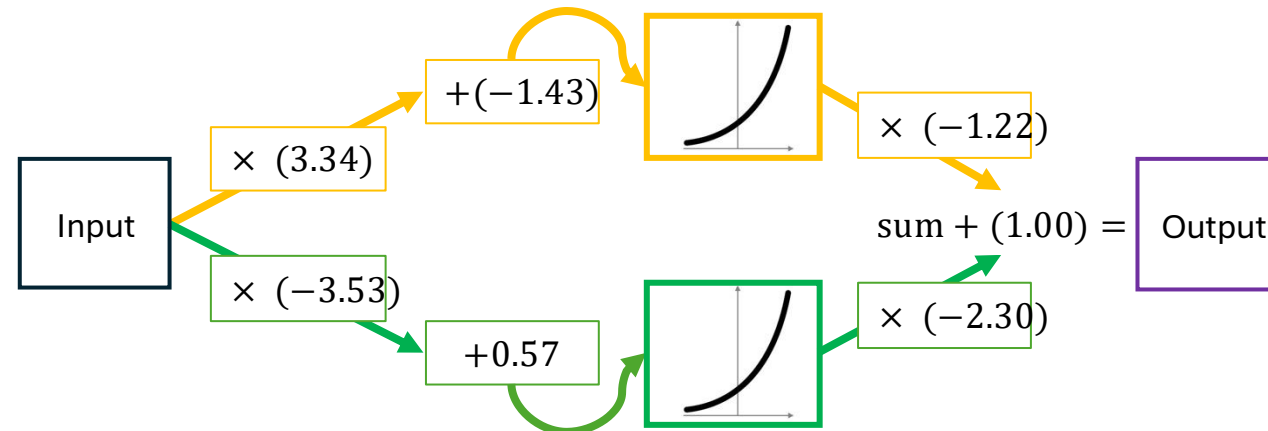
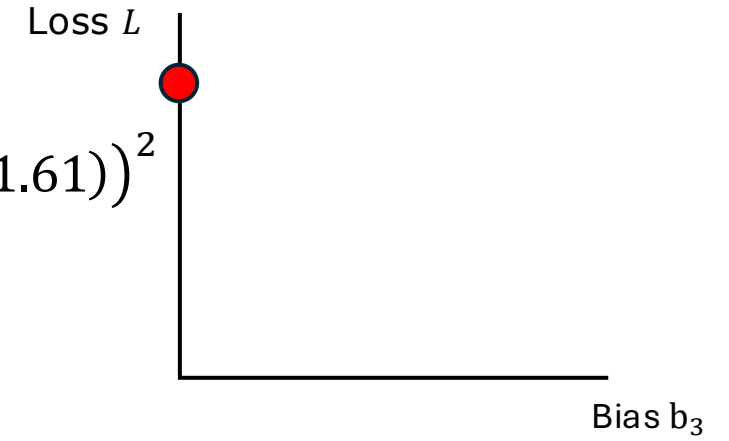
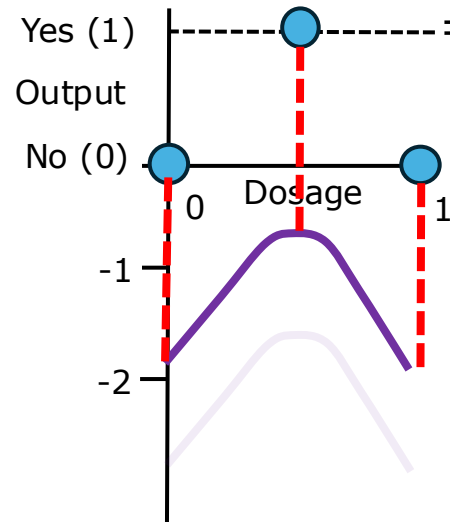


Backpropagation Example

... computing the Loss:

$$L = (0 - (-1.6))^2 + (1 - (-0.6))^2 + (0 - (-1.61))^2$$

$$= 7.8$$

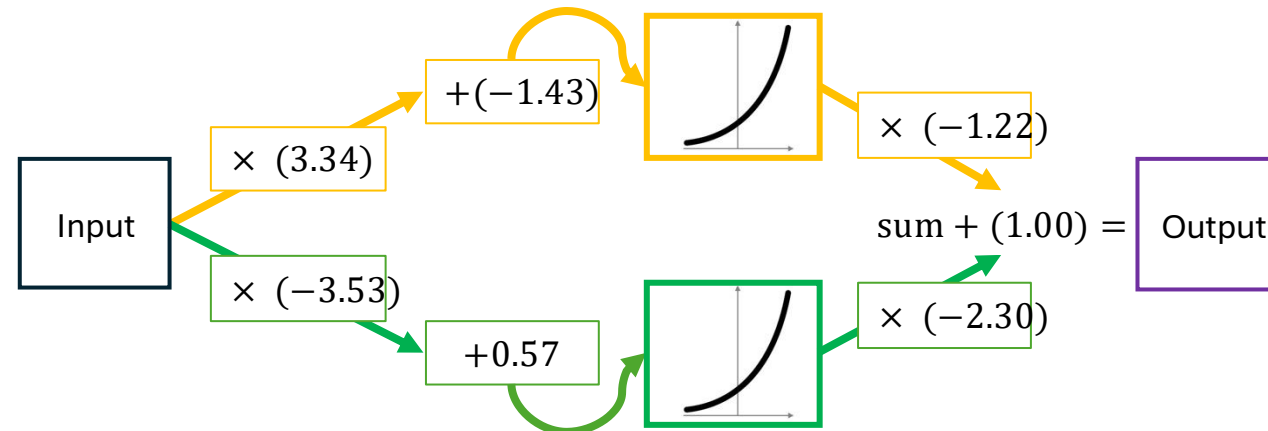
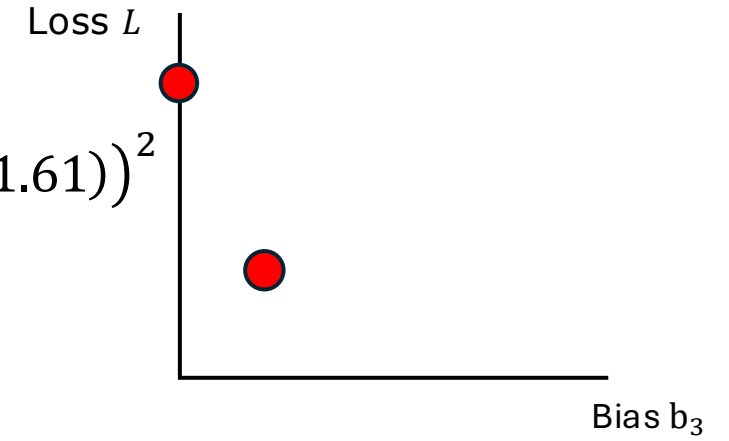
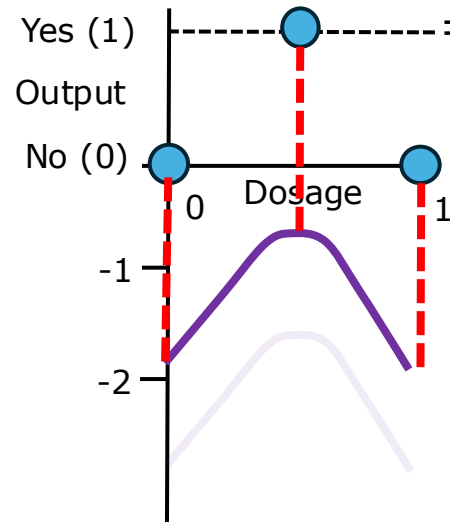


Backpropagation Example

... computing the Loss:

$$L = (0 - (-1.6))^2 + (1 - (-0.6))^2 + (0 - (-1.61))^2$$

$$= 7.8$$



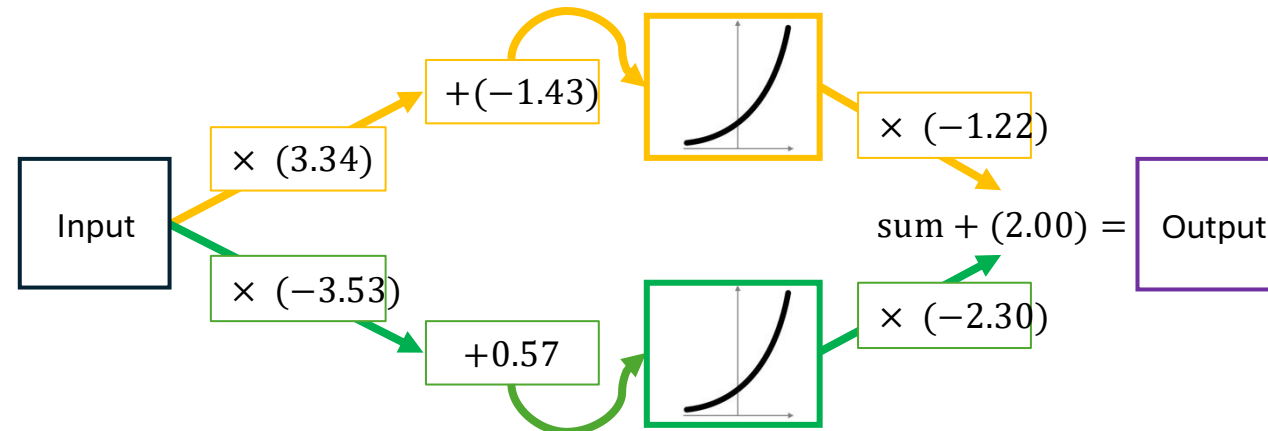
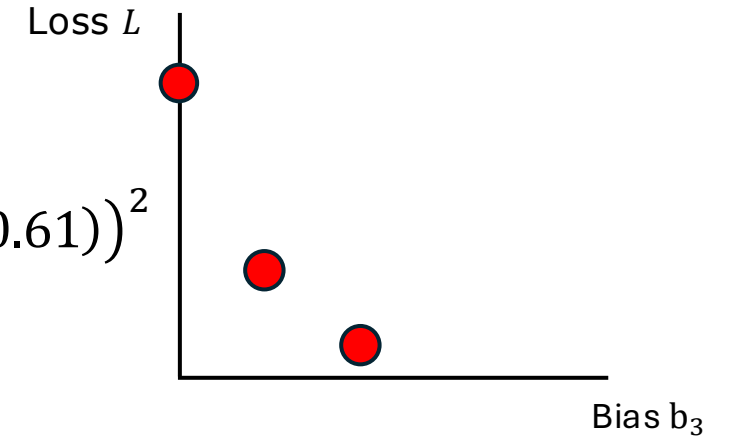
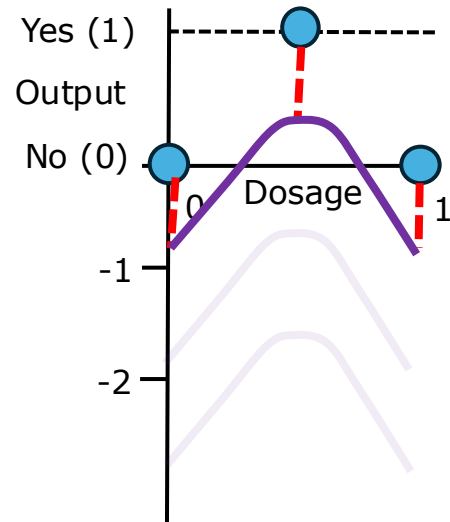
Backpropagation Example

... increase b_3 to 2:

L

$$= (0 - (-0.6))^2 + (1 - (-0.39))^2 + (0 - (-0.61))^2$$

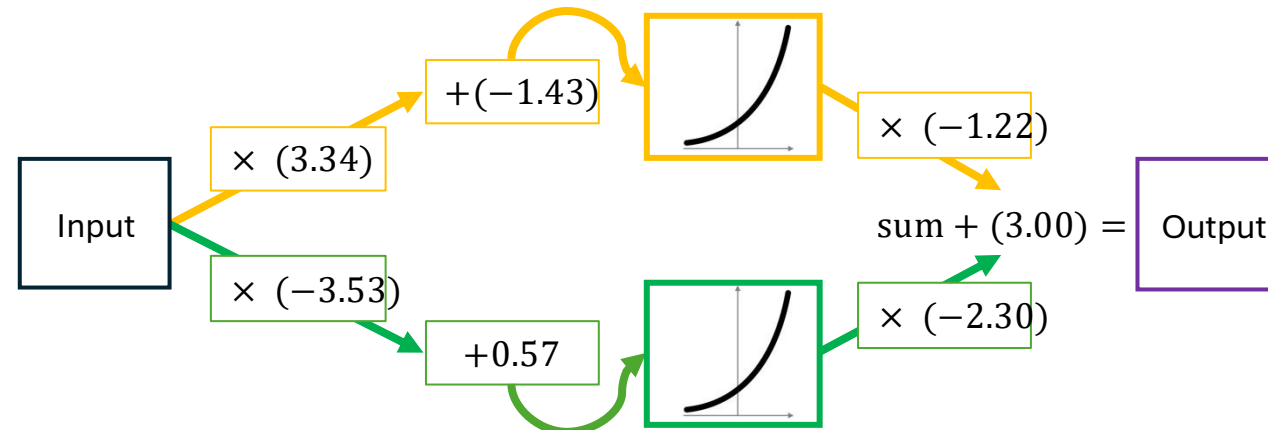
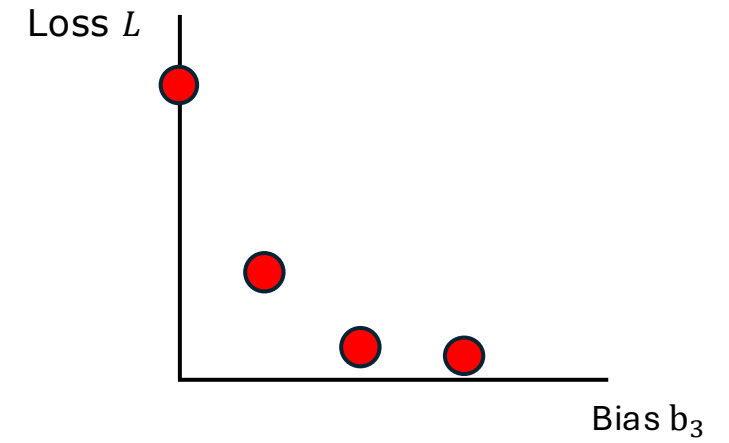
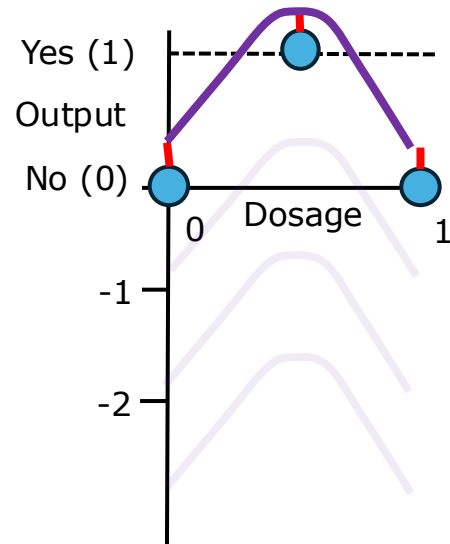
$$= 1.11$$



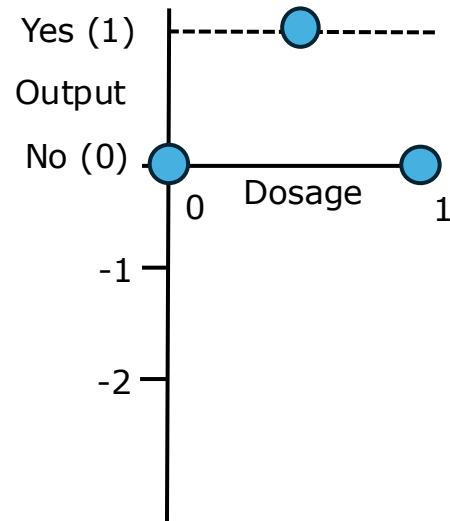
Backpropagation Example

... increase b_3 to 3:

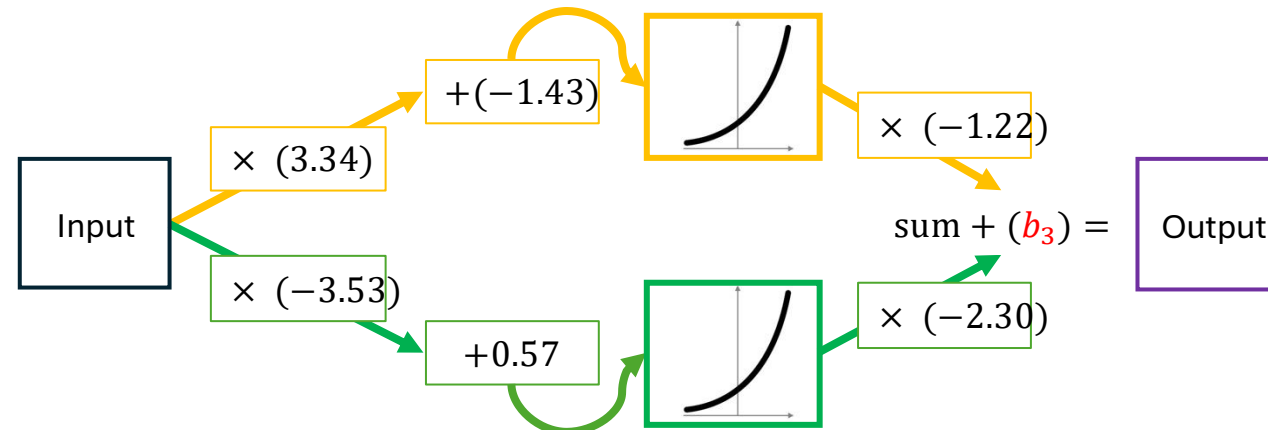
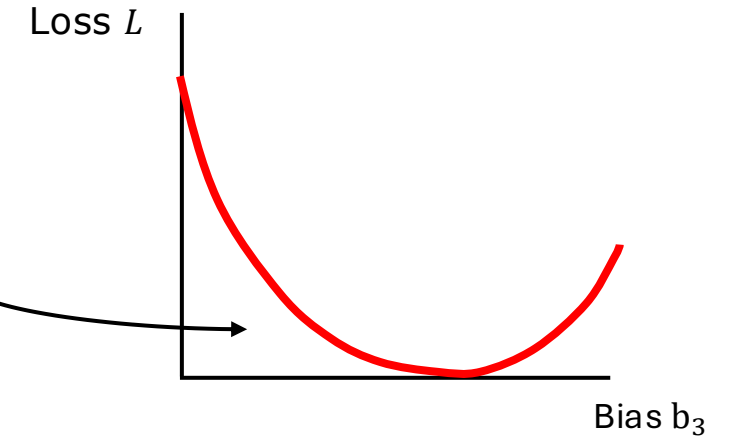
$$L = 0.46$$



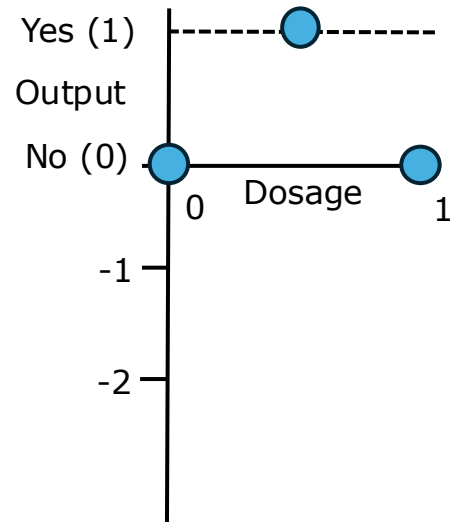
Backpropagation Example



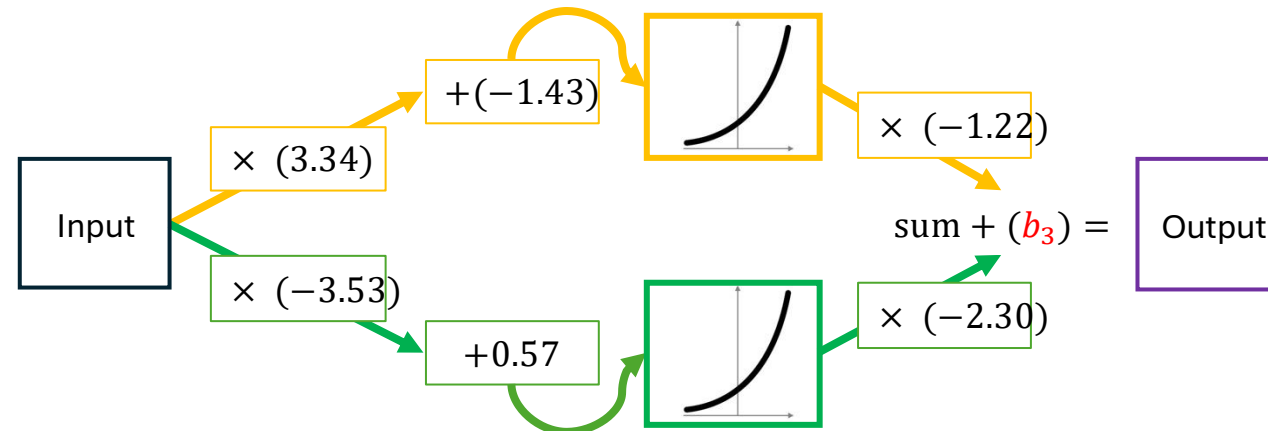
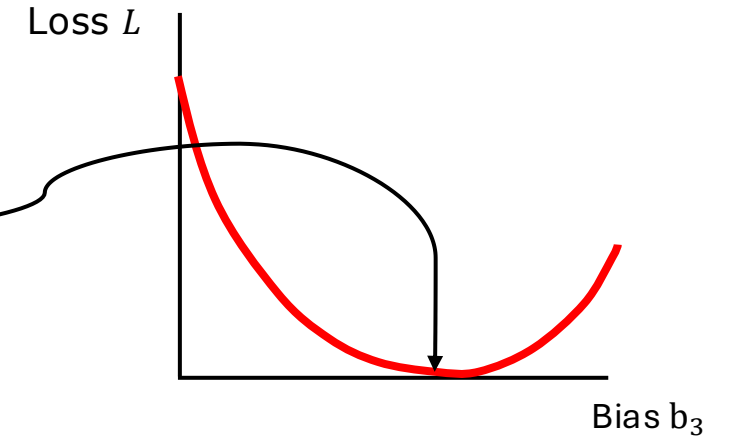
... if we had time to plug in tons of values for b_3 , we would get this curve



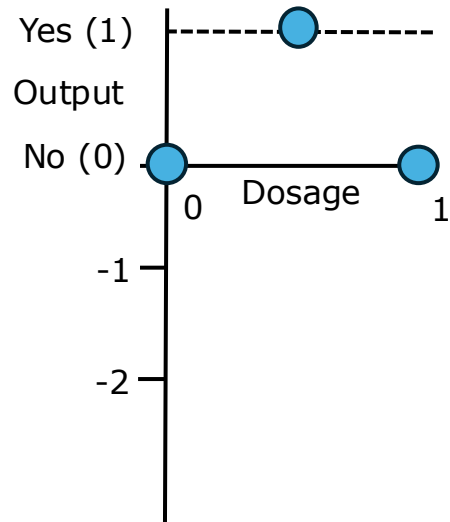
Backpropagation Example



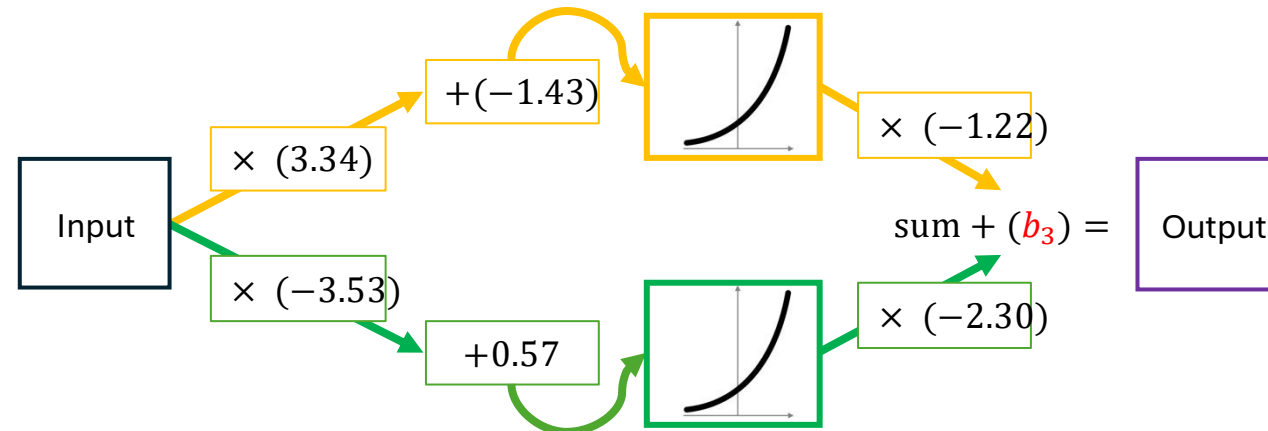
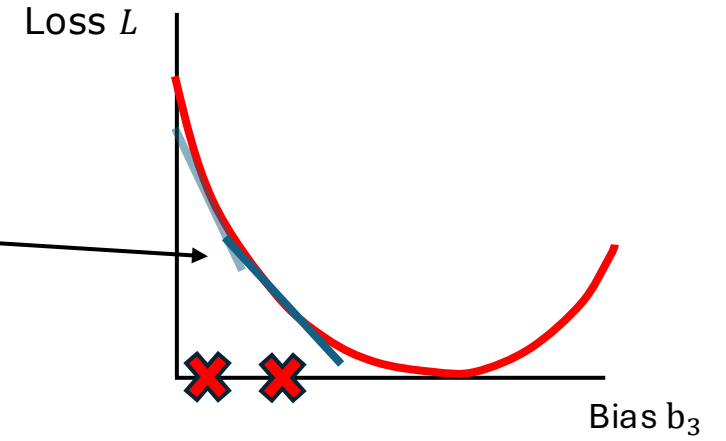
Aiming for the lowest Loss L means aiming for the lowest point in this graph



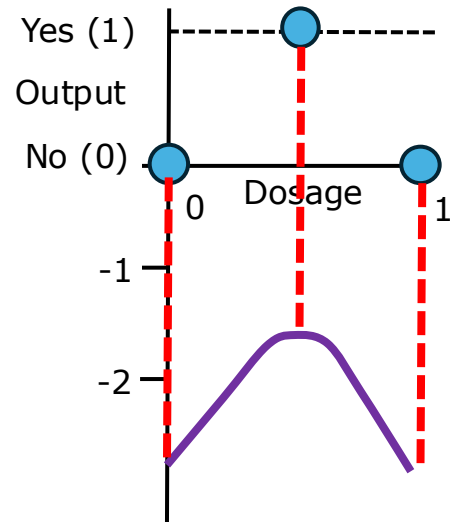
Backpropagation Example



Instead of plugging in tons of values to find the lowest point, we use **Gradient Descent**

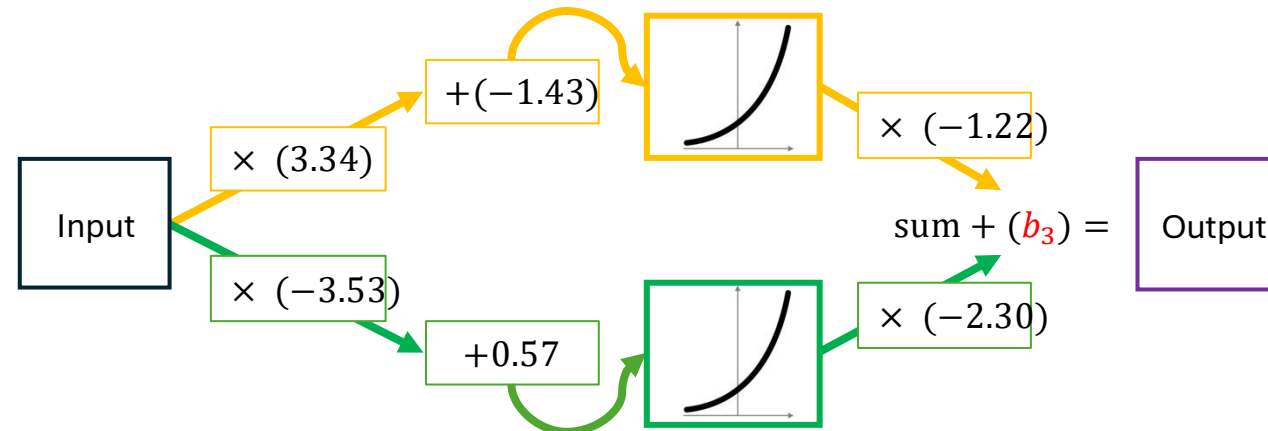
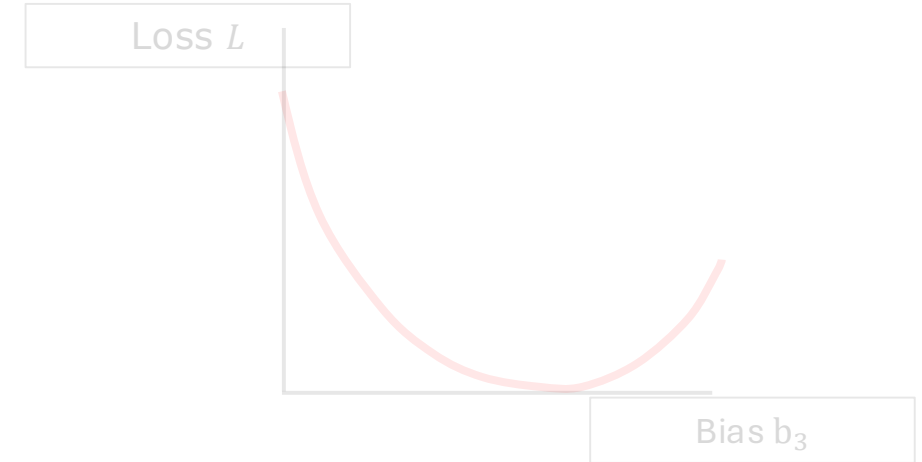


Backpropagation Example

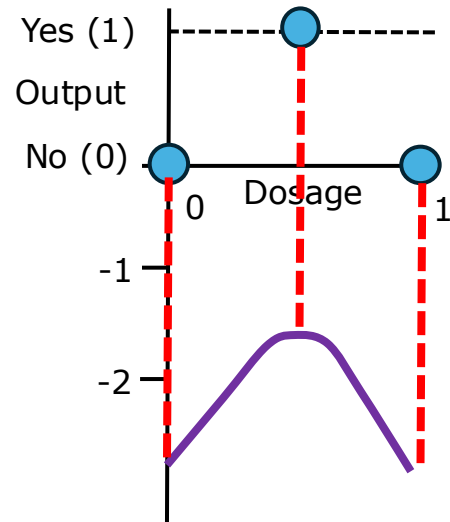


Remember the Loss:

$$L = (\text{Observed}_1 - \text{Predicted}_1)^2 + (\text{Observed}_2 - \text{Predicted}_2)^2 + (\text{Observed}_3 - \text{Predicted}_3)^2$$

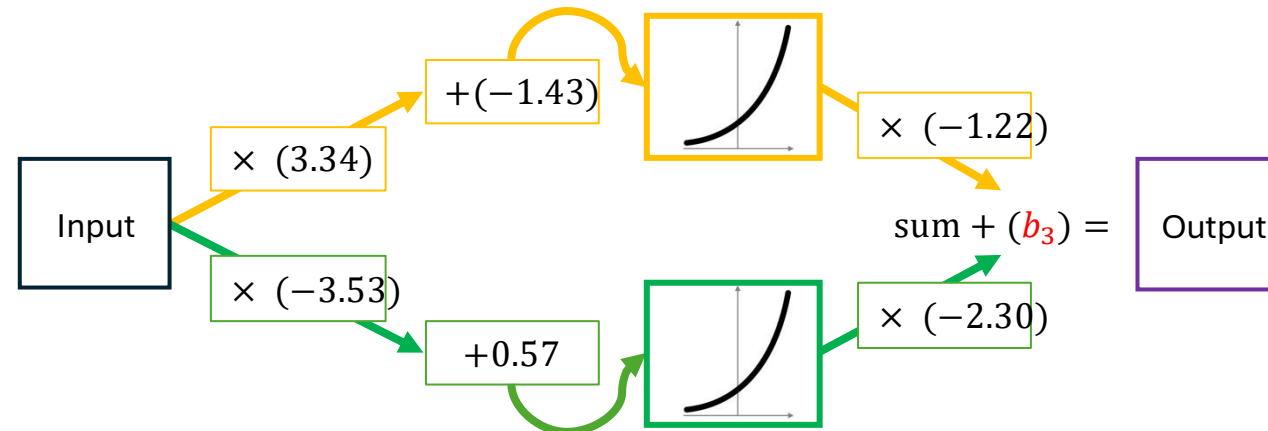
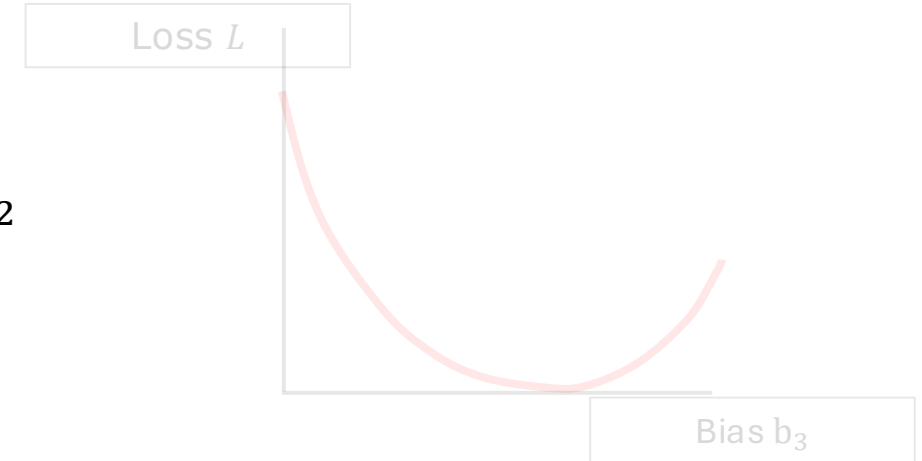


Backpropagation Example

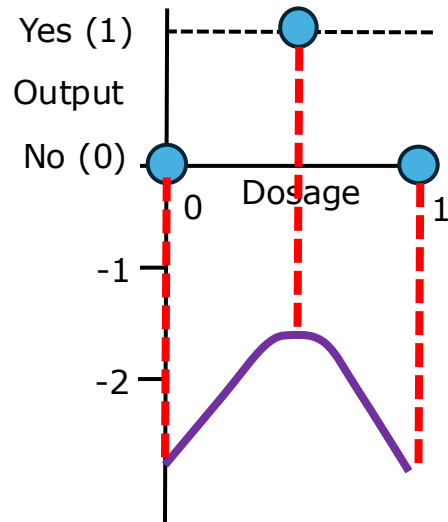


We could also write:

$$L = \sum_{i=1}^{n=3} (\text{Observed}_i - \text{Predicted}_i)^2$$



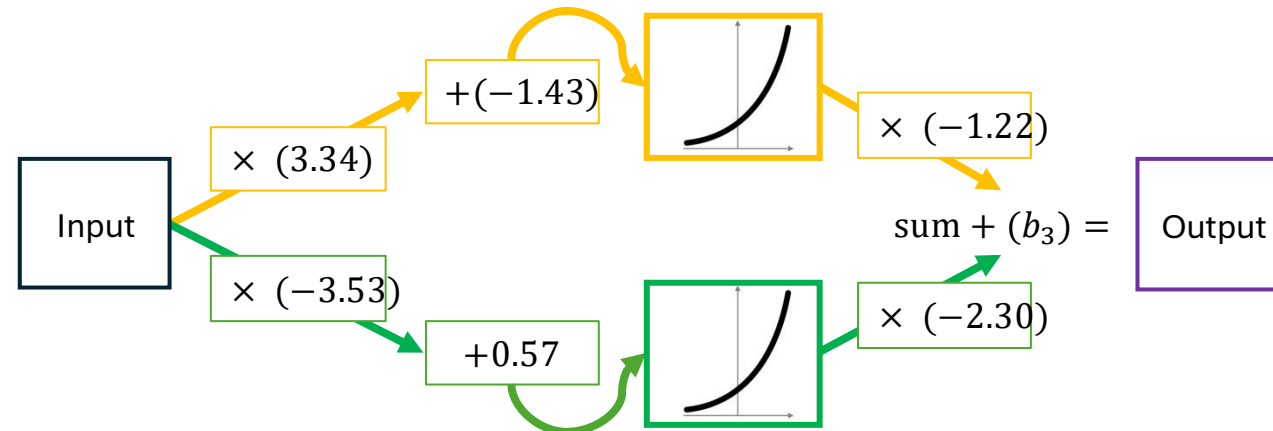
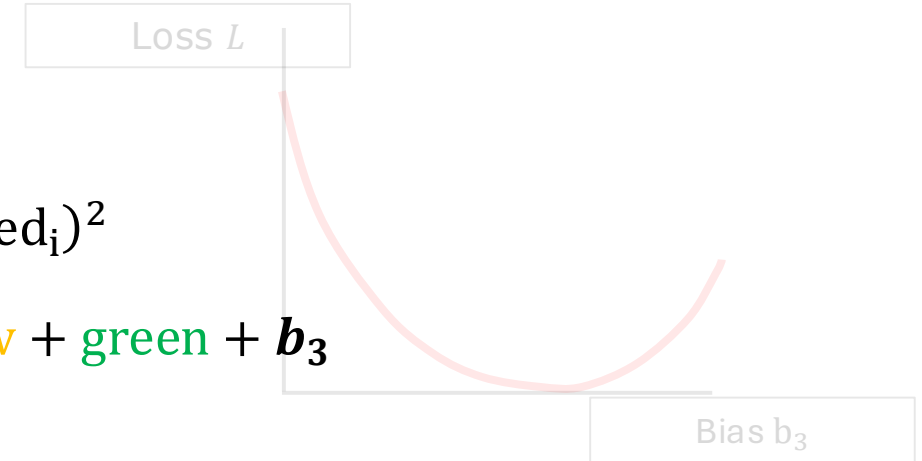
Backpropagation Example



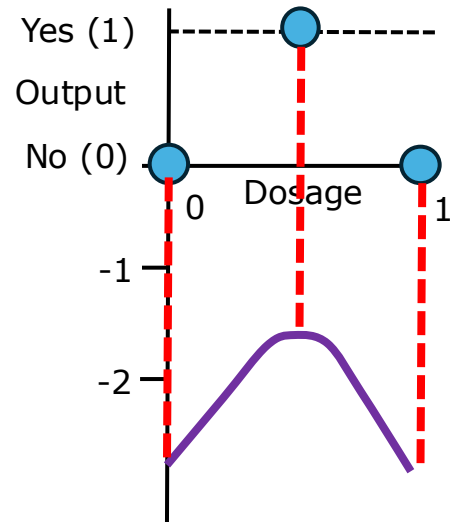
The Loss is linked to b_3 :

$$L = \sum_{i=1}^{n=3} (\text{Observed}_i - \text{Predicted}_i)^2$$

$$\text{Predicted}_i = \text{purple squiggle}_i = \text{yellow} + \text{green} + b_3$$



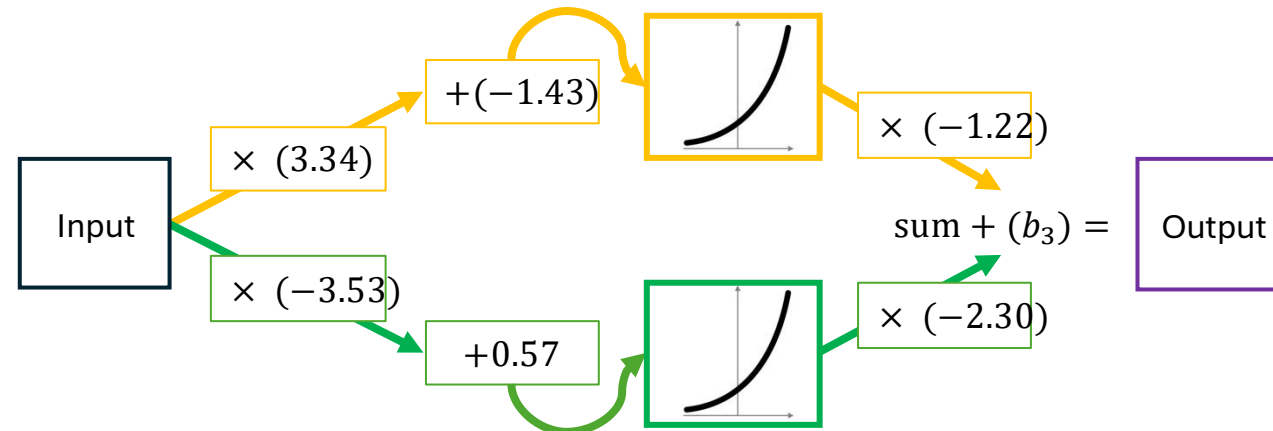
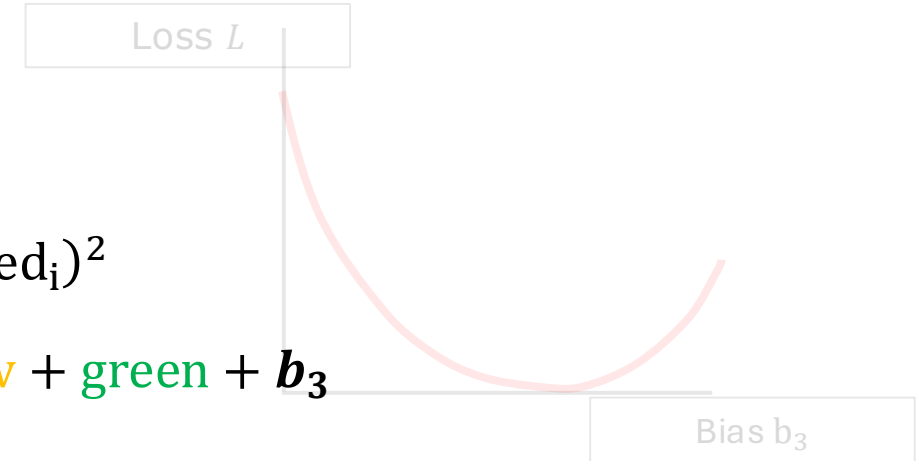
Backpropagation Example



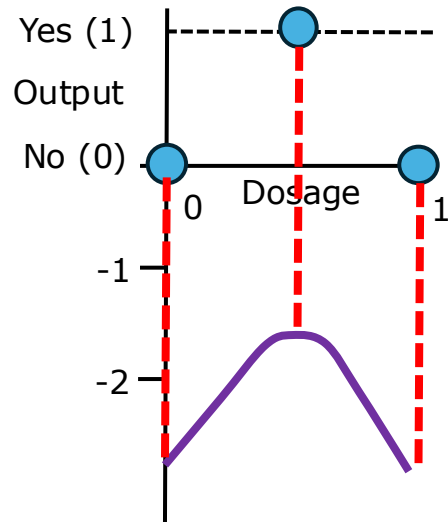
We want: $\frac{dL}{db_3}_{n=3}$

$$L = \sum_{i=1}^n (\text{Observed}_i - \text{Predicted}_i)^2$$

$$\text{Predicted}_i = \text{purple squiggle}_i = \text{yellow} + \text{green} + b_3$$



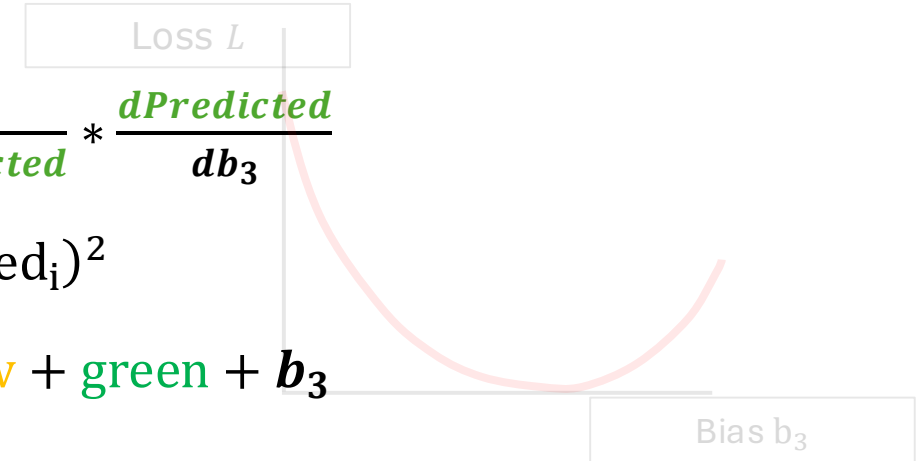
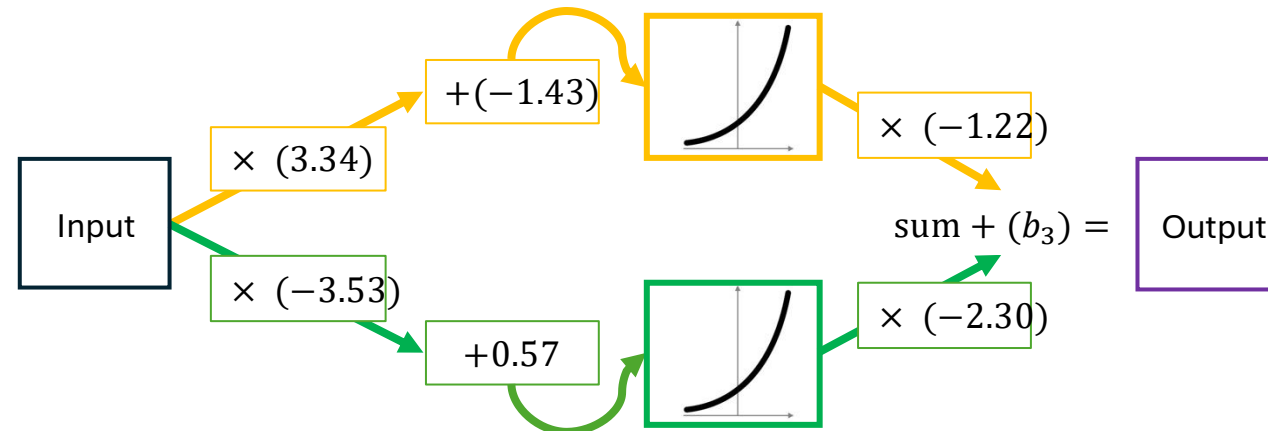
Backpropagation Example



The **chain rule** says: $\frac{dL}{db_3} = \frac{dL}{d\text{Predicted}} * \frac{d\text{Predicted}}{db_3}$

$$L = \sum_{i=1}^{n=3} (\text{Observed}_i - \text{Predicted}_i)^2$$

$\text{Predicted}_i = \text{purple squiggle}_i = \text{yellow} + \text{green} + b_3$



Backpropagation Example

$$\frac{dL}{db_3} = \frac{dL}{d\text{Predicted}} * \frac{d\text{Predicted}}{db_3}$$

Now, we can solve the derivative of L with respect to the **Predicted** values by first substituting in the equation and then the **chain rule**

$$\frac{dL}{d\text{Predicted}} = \frac{d}{d\text{Predicted}} \sum_{i=1}^{n=3} (\text{Observed}_i - \text{Predicted}_i)^2$$

$$L = \sum_{i=1}^{n=3} (\text{Observed}_i - \text{Predicted}_i)^2$$

$$\text{Predicted}_i = \text{purple squiggle}_i = \text{yellow} + \text{green} + b_3$$

Backpropagation Example

$$\frac{dL}{db_3} = \frac{dL}{d\text{Predicted}} * \frac{d\text{Predicted}}{db_3}$$

Now, we can solve the derivative of L with respect to the **Predicted** values by first substituting in the equation and then the **chain rule**

$$\frac{dL}{d\text{Predicted}} = \frac{d}{d\text{Predicted}} \sum_{i=1}^{n=3} (\text{Observed}_i - \text{Predicted}_i)^2 = \sum_{i=1}^{n=3} 2 * (\text{Observed}_i - \text{Predicted}_i) * (-1)$$

$$L = \sum_{i=1}^{n=3} (\text{Observed}_i - \text{Predicted}_i)^2$$

$$\text{Predicted}_i = \text{purple squiggle}_i = \text{yellow} + \text{green} + b_3$$

Backpropagation Example

$$\frac{dL}{db_3} = \frac{dL}{d\text{Predicted}} * \frac{d\text{Predicted}}{db_3}$$

Now, we can solve the derivative of L with respect to the **Predicted** values by first substituting in the equation and then the **chain rule**

$$\frac{dL}{d\text{Predicted}} = \frac{d}{d\text{Predicted}} \sum_{i=1}^{n=3} (\text{Observed}_i - \text{Predicted}_i)^2 = \sum_{i=1}^{n=3} -2 * (\text{Observed}_i - \text{Predicted}_i)$$

$$L = \sum_{i=1}^{n=3} (\text{Observed}_i - \text{Predicted}_i)^2$$

$$\text{Predicted}_i = \text{purple squiggle}_i = \text{yellow} + \text{green} + b_3$$

Backpropagation Example

$$\frac{dL}{db_3} = \underbrace{\frac{dL}{d\text{Predicted}}}_{n=3} * \frac{d\text{Predicted}}{db_3}$$

$$= \sum_{i=1}^{n=3} -2 * (\text{Observed}_i - \text{Predicted}_i)$$

We are done with the first part

$$L = \sum_{i=1}^{n=3} (\text{Observed}_i - \text{Predicted}_i)^2$$

$\text{Predicted}_i = \text{purple squiggle}_i = \text{yellow} + \text{green} + b_3$

Backpropagation Example

$$\frac{dL}{db_3} = \underbrace{\frac{dL}{d\text{Predicted}}}_{n=3} * \frac{d\text{Predicted}}{db_3}$$

$$= \sum_{i=1}^{n=3} -2 * (\text{Observed}_i - \text{Predicted}_i)$$

We are done with the first part

$$L = \sum_{i=1}^{n=3} (\text{Observed}_i - \text{Predicted}_i)^2$$

$\text{Predicted}_i = \text{purple squiggle}_i = \text{yellow} + \text{green} + b_3$

Backpropagation Example

$$\frac{dL}{db_3} = \frac{dL}{d\text{Predicted}} * \frac{d\text{Predicted}}{db_3}$$

Now, let's proceed with the second part, the **Predicted** values with respect to b_3

$$= \sum_{i=1}^{n=3} -2 * (\text{Observed}_i - \text{Predicted}_i)$$

$$L = \sum_{i=1}^{n=3} (\text{Observed}_i - \text{Predicted}_i)^2$$

$$\text{Predicted}_i = \text{purple squiggle}_i = \text{yellow} + \text{green} + b_3$$

Backpropagation Example

$$\frac{dL}{db_3} = \frac{dL}{d\text{Predicted}} * \frac{d\text{Predicted}}{db_3}$$

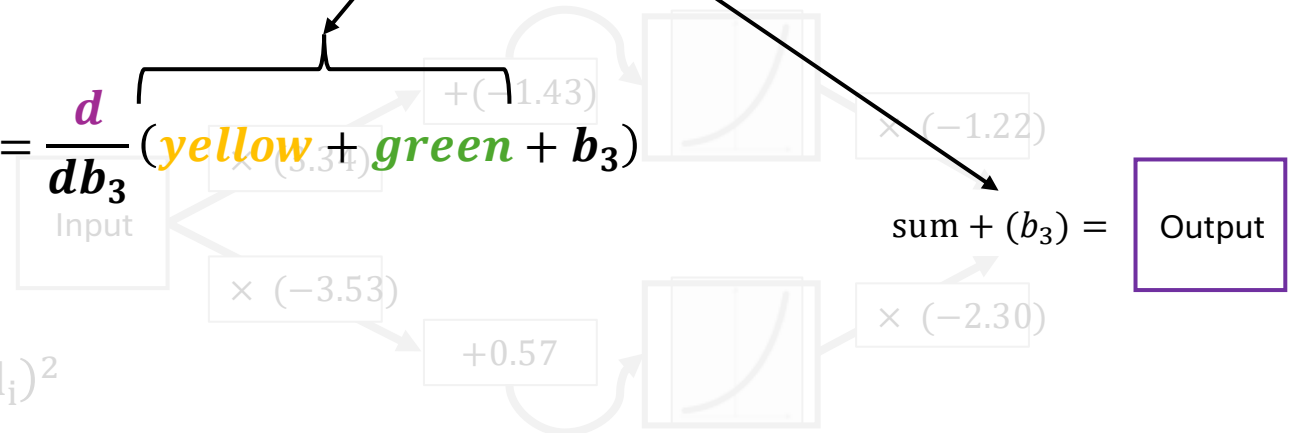
$$= \sum_{i=1}^{n=3} -2 * (\text{Observed}_i - \text{Predicted}_i)$$

Remember, the **yellow** and **green** curves were created before we got to b_3

$$\frac{d\text{Predicted}}{db_3} = \frac{d}{db_3} \text{purple squiggle} = \frac{d}{db_3} (\text{yellow} + \text{green} + b_3)$$

$$L = \sum_{i=1}^{n=3} (\text{Observed}_i - \text{Predicted}_i)^2$$

$$\text{Predicted}_i = \text{purple squiggle}_i = \text{yellow} + \text{green} + b_3$$



Backpropagation Example

$$\frac{dL}{db_3} = \frac{dL}{dPredicted} * \frac{dPredicted}{db_3}$$

... which leads us to ...

$$= \sum_{i=1}^{n=3} -2 * (Observed_i - Predicted_i)$$

$$\frac{dPredicted}{db_3} = \frac{d}{db_3} \text{purple squiggle} = \frac{d}{db_3} (\text{yellow} + \text{green} + b_3) = 0 + 0 + 1 = 1$$

$$L = \sum_{i=1}^{n=3} (Observed_i - Predicted_i)^2$$

$$Predicted_i = \text{purple squiggle}_i = \text{yellow} + \text{green} + b_3$$

Backpropagation Example

$$\frac{dL}{db_3} = \frac{dL}{dPredicted} * \frac{dPredicted}{db_3}$$

... which leads us to ...

$$= \sum_{i=1}^{n=3} -2 * (Observed_i - Predicted_i) * 1$$

$$\frac{dPredicted}{db_3} = \frac{d}{db_3} \text{purple squiggle} = \frac{d}{db_3} (\text{yellow} + \text{green} + b_3) = 1$$

$$L = \sum_{i=1}^{n=3} (Observed_i - Predicted_i)^2$$

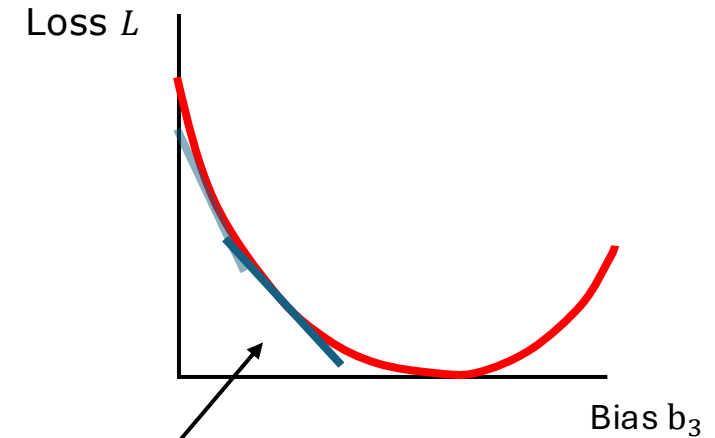
$$Predicted_i = \text{purple squiggle}_i = \text{yellow} + \text{green} + b_3$$

Backpropagation Example

$$\frac{dL}{db_3} = \frac{dL}{d\text{Predicted}} * \frac{d\text{Predicted}}{db_3}$$

$$= \sum_{i=1}^{n=3} -2 * (\text{Observed}_i - \text{Predicted}_i) * 1$$

Now, we have the derivative of the Loss L with respect to b_3



... and that means we can plug the derivative into **Gradient Descent** to find the optimal value for b_3

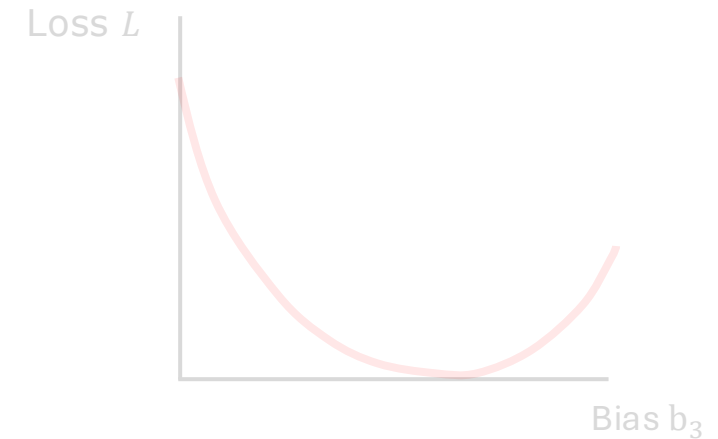
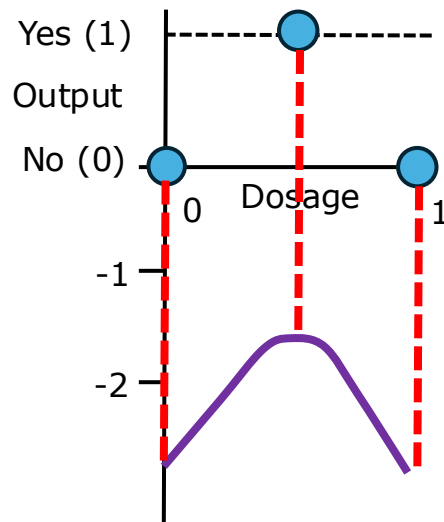
Backpropagation Example

$$\frac{dL}{db_3} = \frac{dL}{d\text{Predicted}} * \frac{d\text{Predicted}}{db_3}$$

$$= \sum_{i=1}^{n=3} -2 * (\text{Observed}_i - \text{Predicted}_i) * 1$$

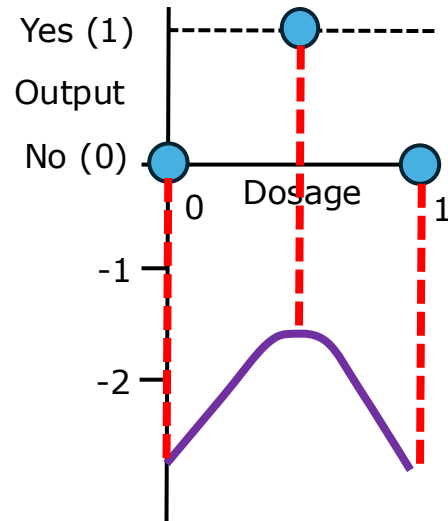
$$= -2 * (0 - \text{Predicted}_1) * 1 \\ + (-2 * (1 - \text{Predicted}_2) * 1) \\ + (-2 * (0 - \text{Predicted}_3) * 1)$$

$$= -2 * (0 - (-2.6)) * 1 + (-2 * (1 - (-1.6)) * 1) \\ + (-2 * (0 - (-2.6)) * 1) = -15.7$$

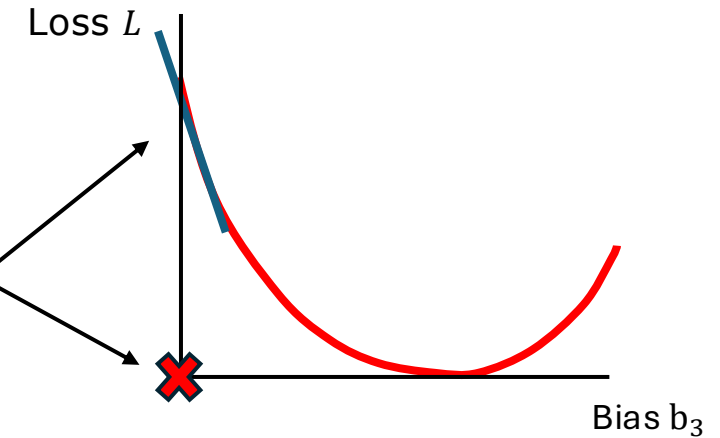


Backpropagation Example

$$\frac{dL}{db_3} = -15.7$$



That corresponds to
the slope for $b_3 = 0$



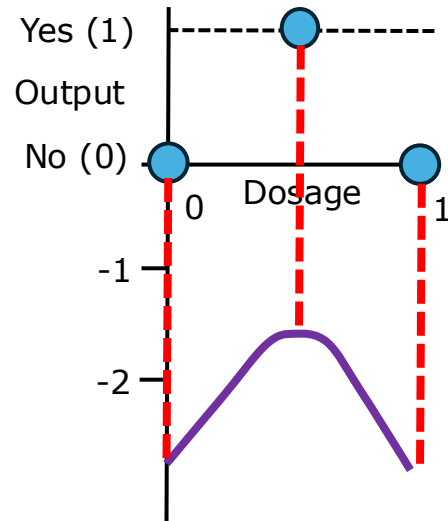
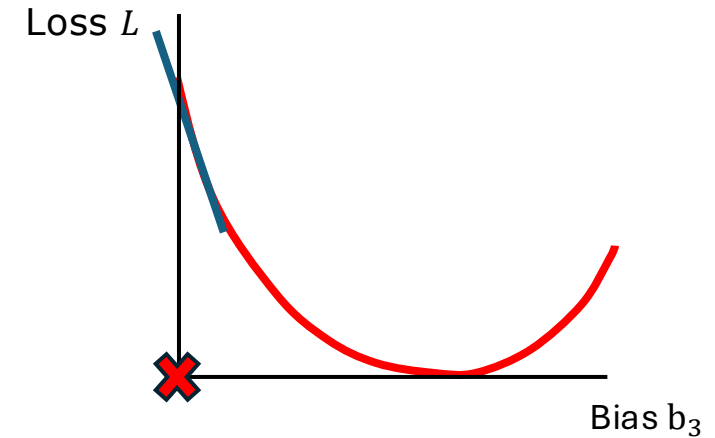
Backpropagation Example

$$\frac{dL}{db_3} = -15.7$$

Remember
Gradient Descent:

$$w := w - \eta \frac{\partial L}{\partial w}$$

$$\text{Step Size} = 0 - \text{Learning Rate} \cdot (-15.7)$$



Backpropagation Example

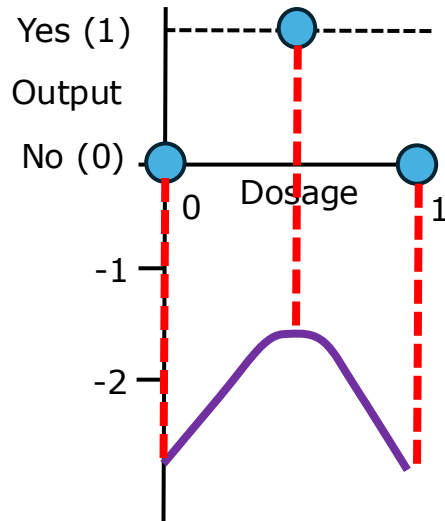
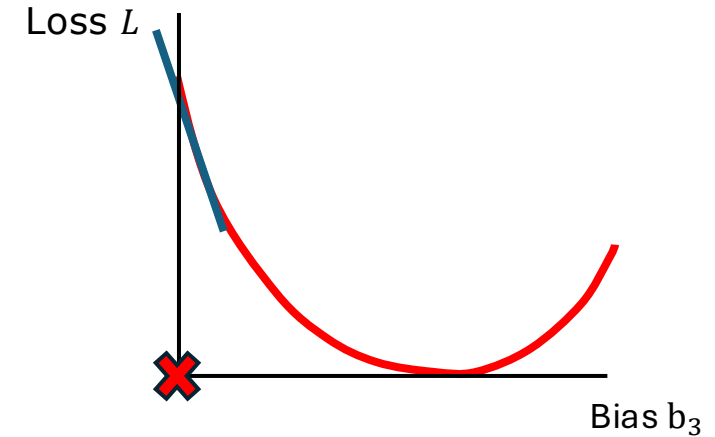
$$\frac{dL}{db_3} = -15.7$$

Remember
Gradient Descent:

$$w := w - \eta \frac{\partial L}{\partial w}$$

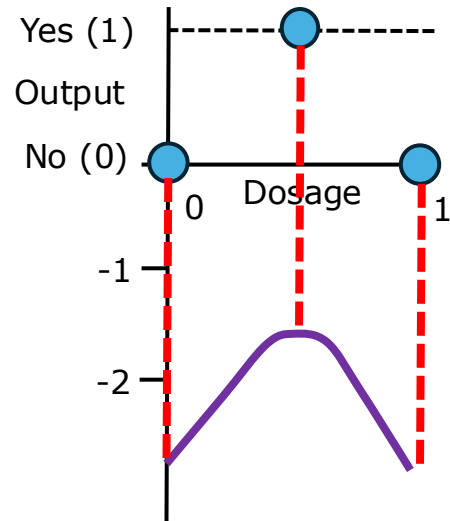
$$\text{Step Size} = 0 - 0.1 \cdot (-15.7) = 1.57$$

E.g. Learning Rate 0.1

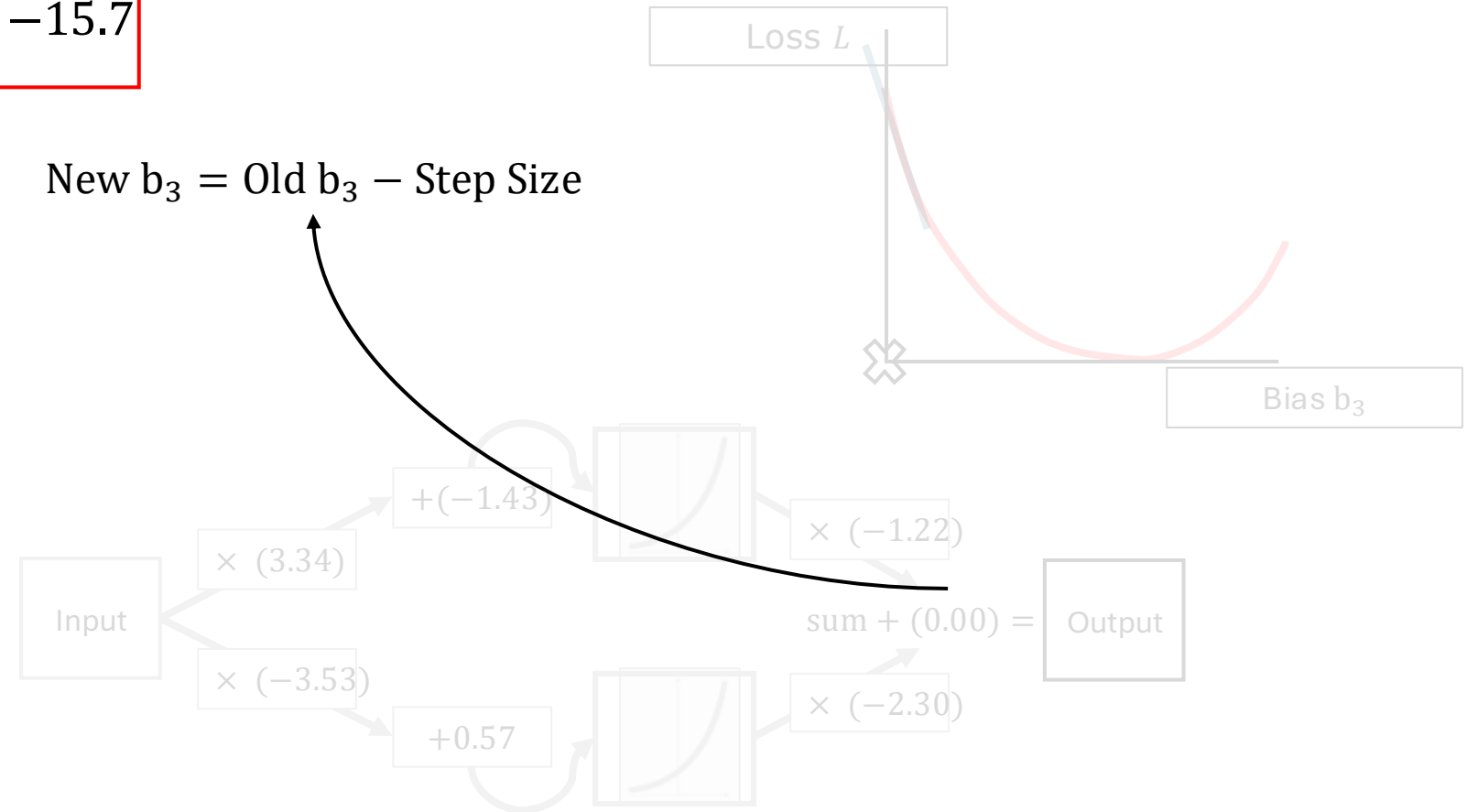


Backpropagation Example

$$\frac{dL}{db_3} = -15.7$$

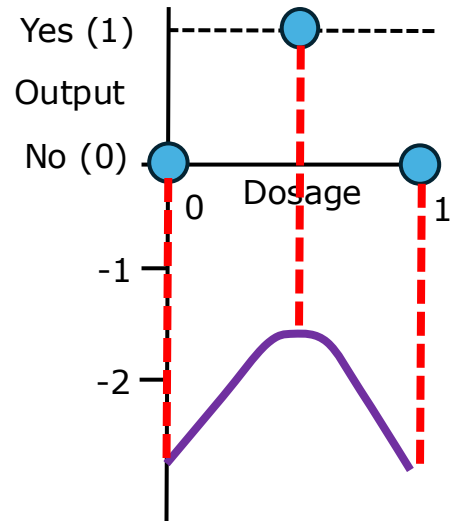


$$\text{New } b_3 = \text{Old } b_3 - \text{Step Size}$$

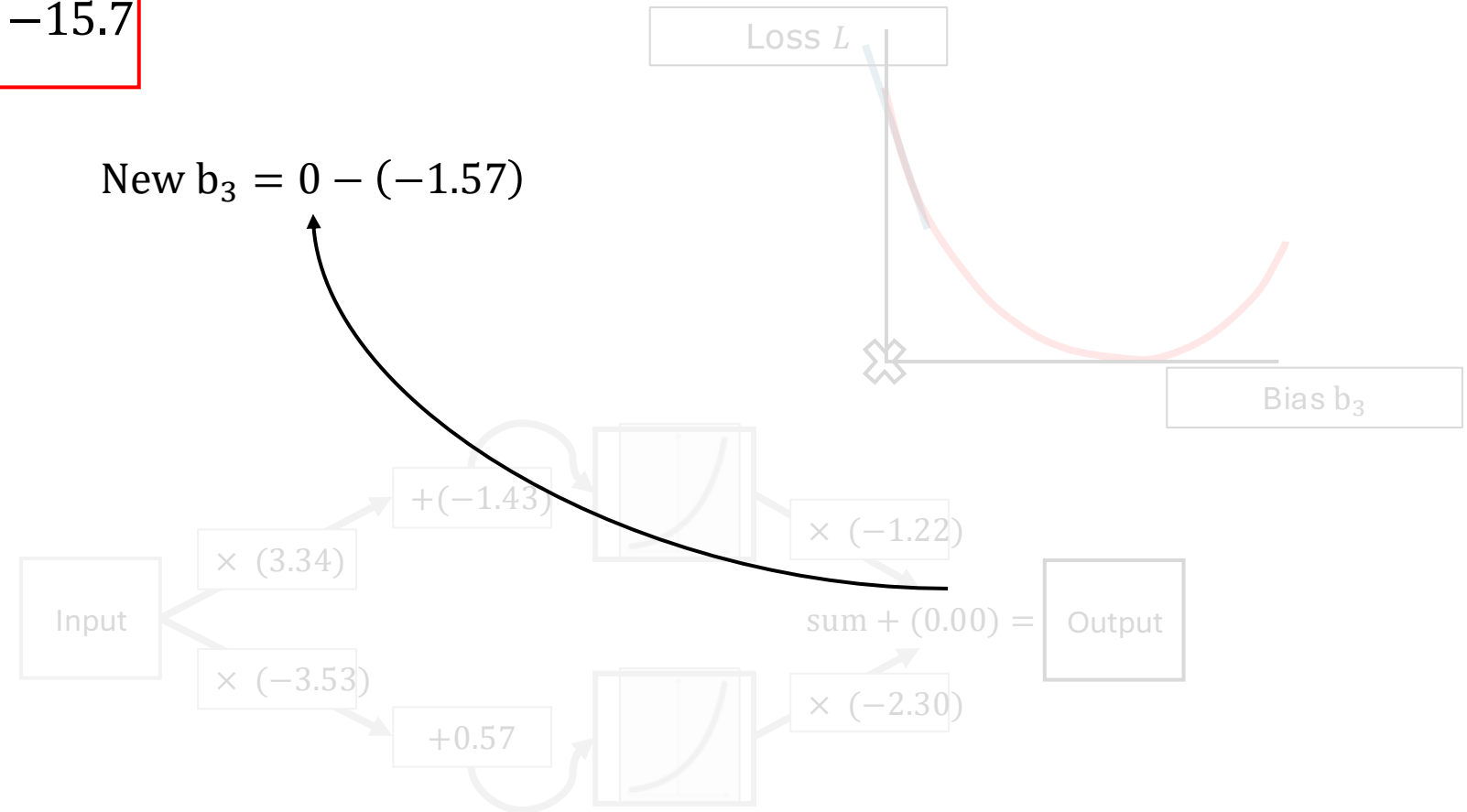


Backpropagation Example

$$\frac{dL}{db_3} = -15.7$$

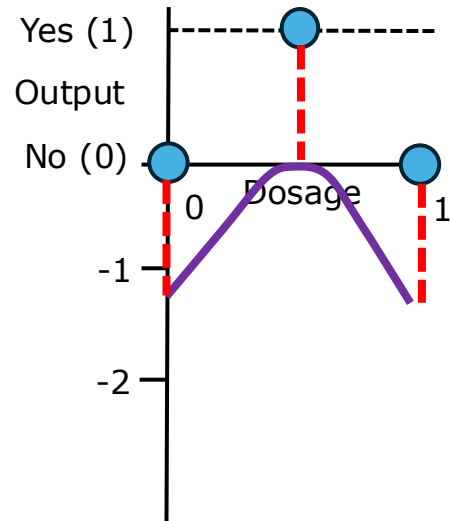


$$\text{New } b_3 = 0 - (-1.57)$$

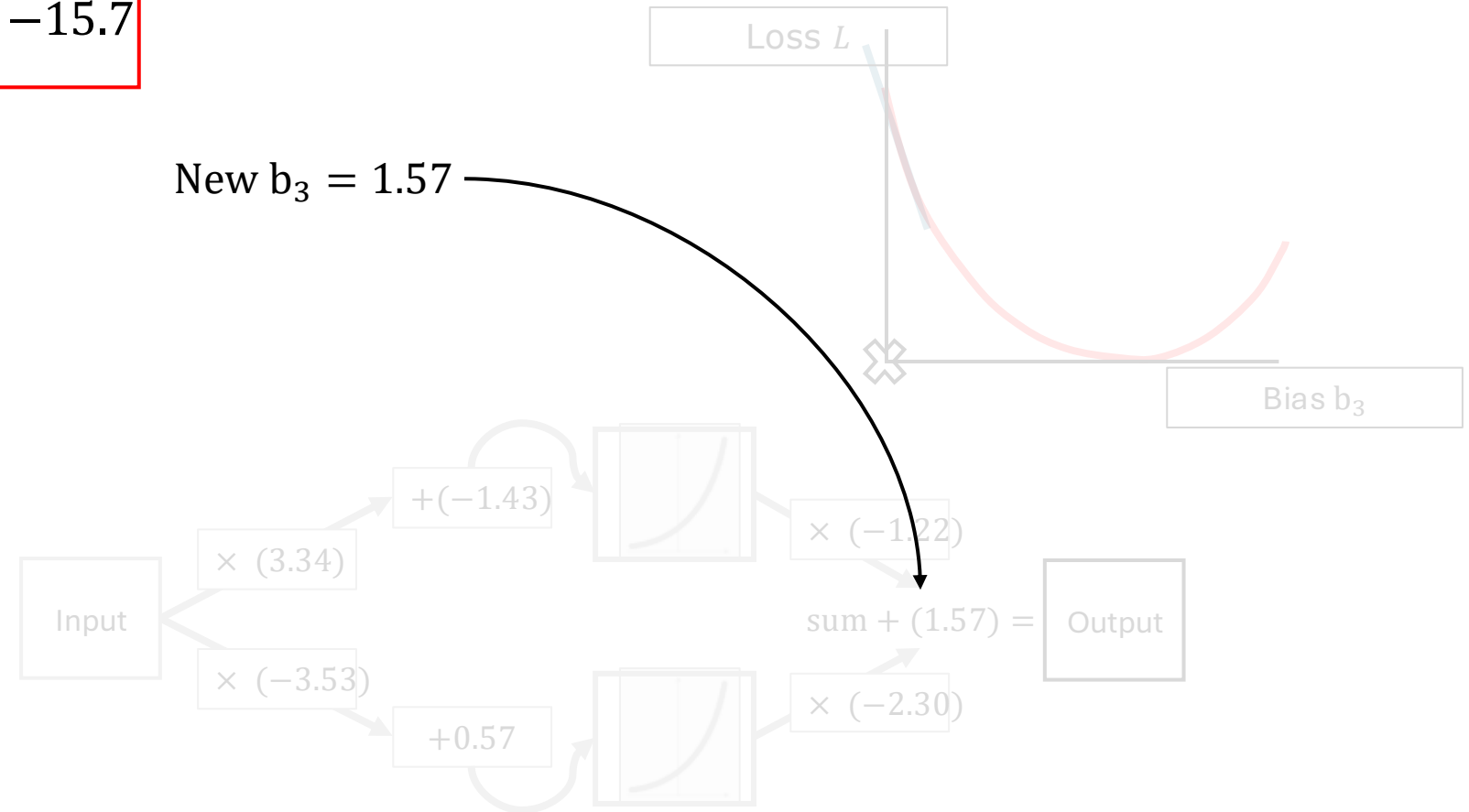


Backpropagation Example

$$\frac{dL}{db_3} = -15.7$$

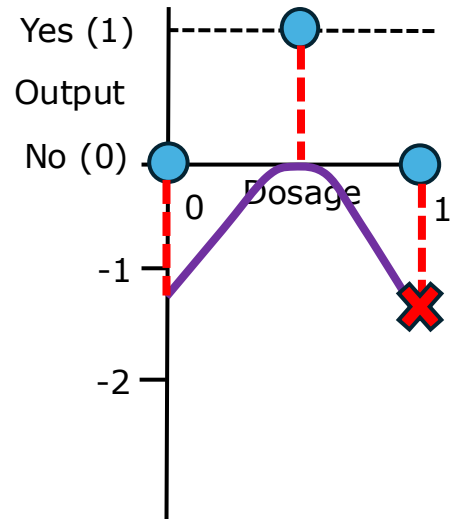


New $b_3 = 1.57$

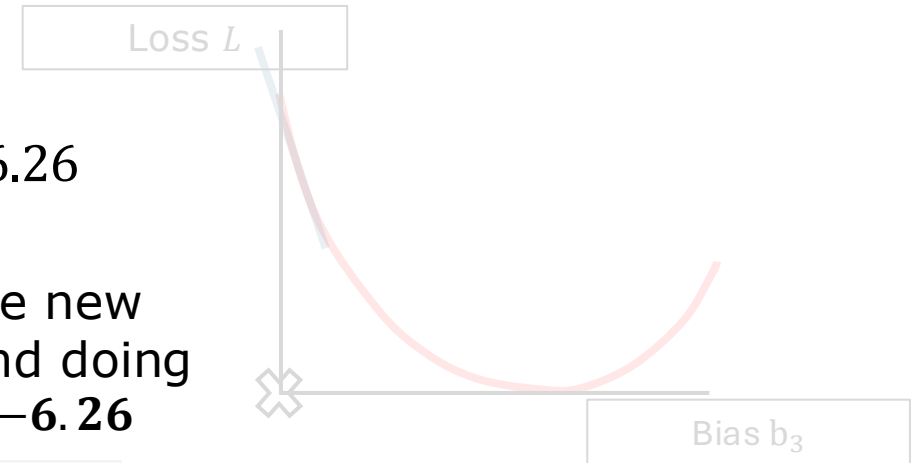


Backpropagation Example

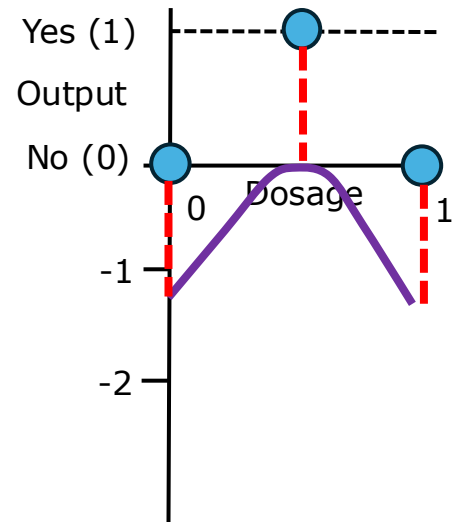
$$\begin{aligned} \frac{dL}{db_3} &= -2 * (0 - \text{Predicted}_1) * 1 \\ &+ (-2 * (1 - \text{Predicted}_2) * 1) \\ &+ (-2 * (0 - \text{Predicted}_3) * 1) = -6.26 \end{aligned}$$



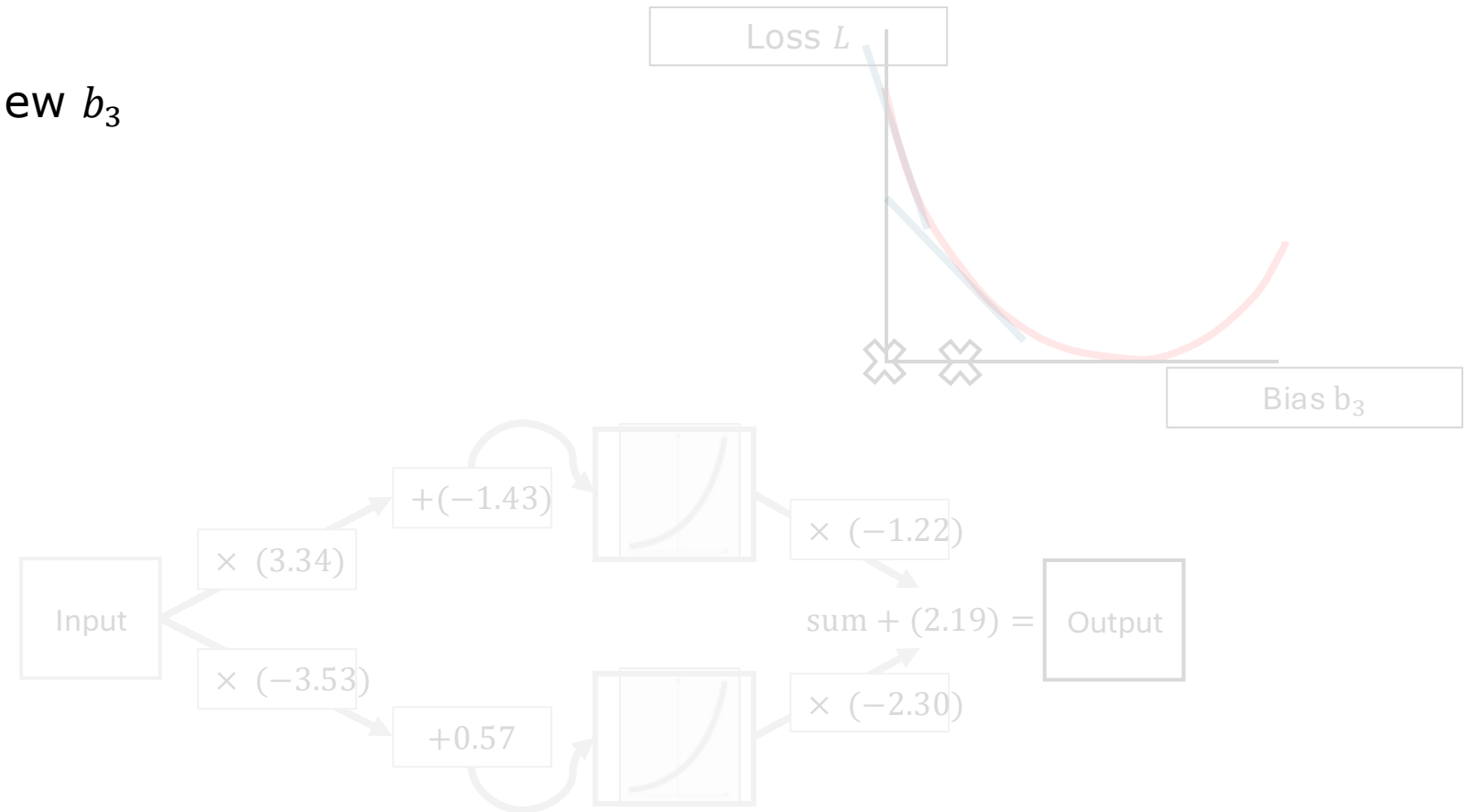
Now plugging in the new Predicted values and doing the math gives us **-6.26**



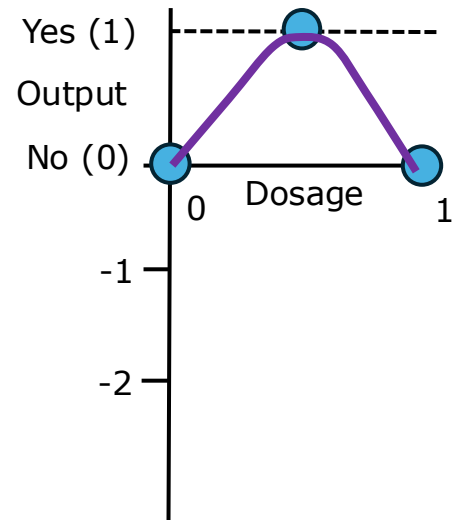
Backpropagation Example



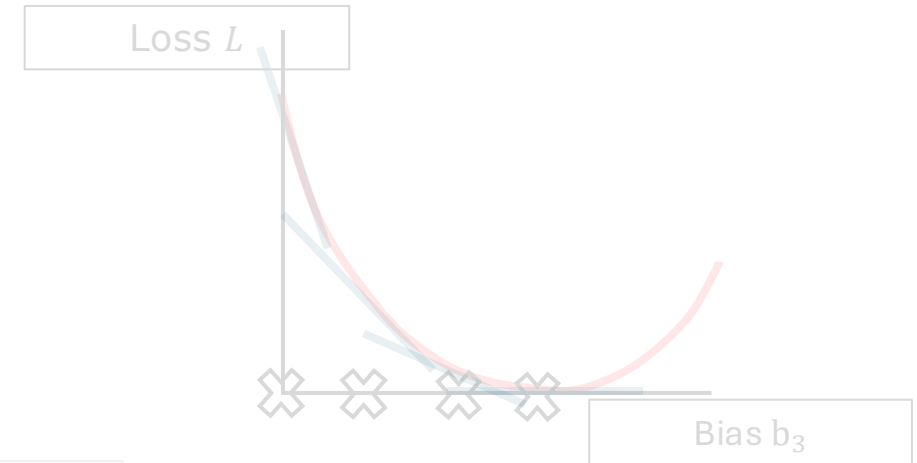
New b_3



Backpropagation Example

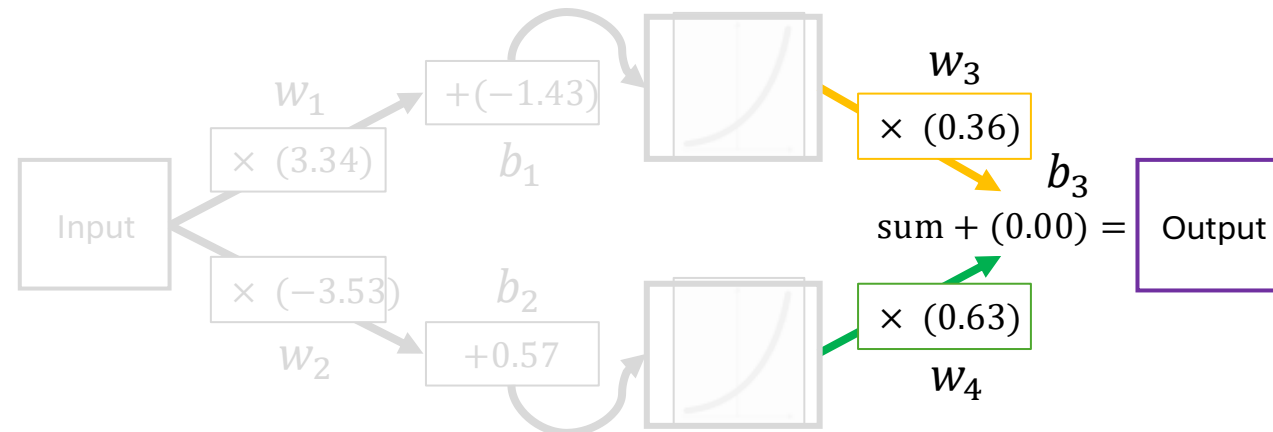
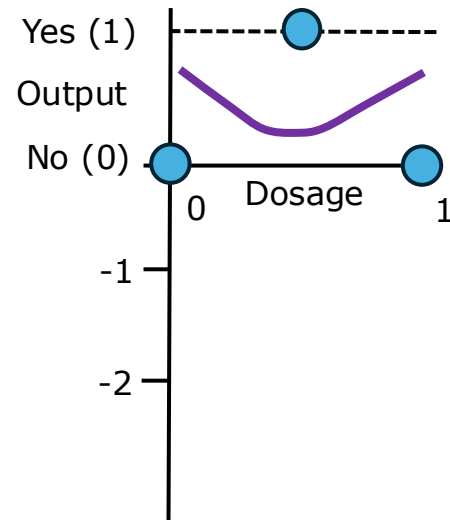


By proceeding we'll always get a new b_3 till convergence



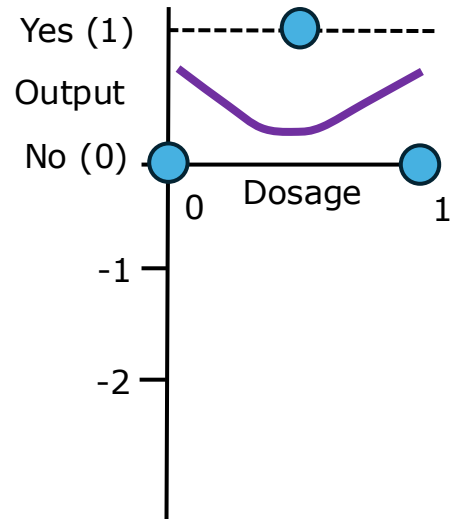
Backpropagation Example

Now, we would like to optimize all other parameters as well, e.g. w_3 and w_4

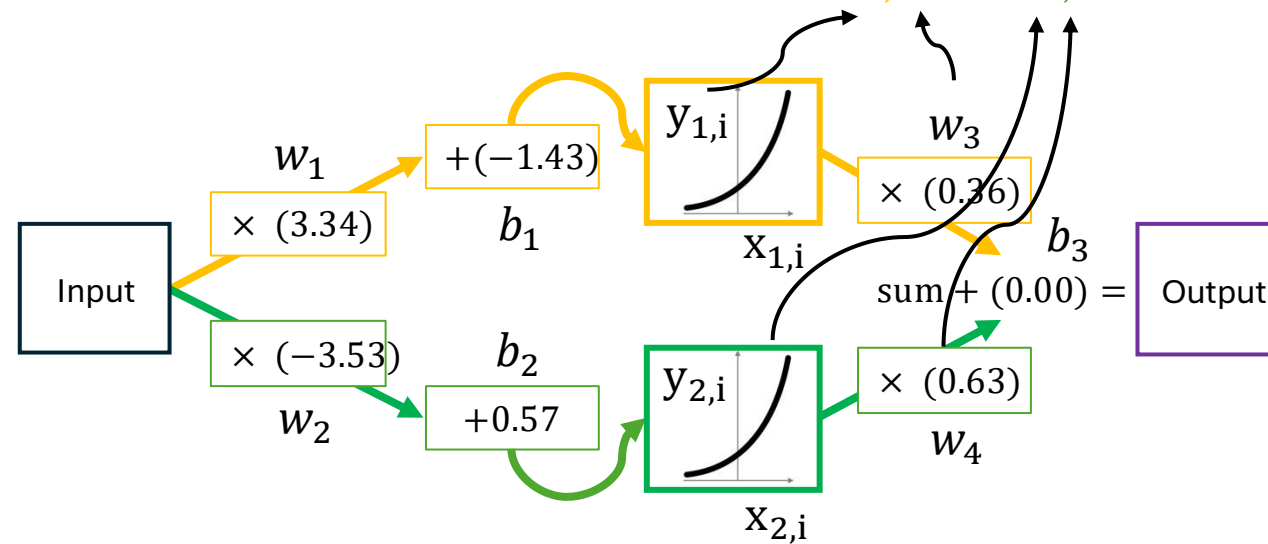


Backpropagation Example

Now, we would like to optimize all other parameters as well, e.g. w_3 and w_4



$$\text{Predicted}_i = \text{purple squiggle}_i = y_{1,i}w_3 + y_{2,i}w_4 + b_3$$



Backpropagation Example

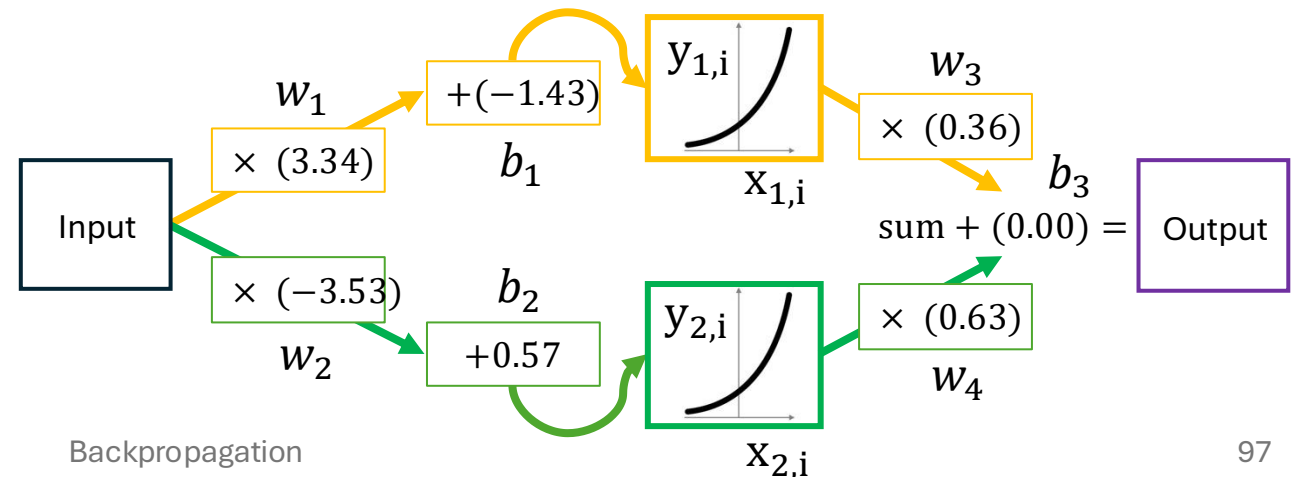
$$\frac{dL}{dw_3} = \frac{dL}{d\text{Predicted}} * \frac{d\text{Predicted}}{dw_3}$$

$$\frac{dL}{dw_4} = \frac{dL}{d\text{Predicted}} * \frac{d\text{Predicted}}{dw_4}$$

← Goal is to compute this

$$L = \sum_{i=1}^{n=3} (\text{Observed}_i - \text{Predicted}_i)^2$$

$$\text{Predicted}_i = \text{purple squiggle}_i = y_{1,i}w_3 + y_{2,i}w_4 + b_3$$



Backpropagation Example

$$\frac{dL}{dw_3} = \frac{dL}{d\text{Predicted}} * \frac{d\text{Predicted}}{dw_3}$$

$$\frac{dL}{dw_4} = \frac{dL}{d\text{Predicted}} * \frac{d\text{Predicted}}{dw_4}$$

How?

... is the exact same
as the derivative
used for b_3

$$\frac{dL}{db_3} = \frac{dL}{d\text{Predicted}} * \frac{d\text{Predicted}}{db_3}$$

Backpropagation Example

$$\frac{dL}{dw_3} = \frac{dL}{dPredicted} * \frac{dPredicted}{dw_3}$$

$$\frac{dL}{dw_4} = \frac{dL}{dPredicted} * \frac{dPredicted}{dw_4}$$

How?

$$\frac{dL}{db_3} = \frac{dL}{dPredicted} * \frac{dPredicted}{db_3} = \sum_{i=1}^{n=3} -2 * (Observed_i - Predicted_i) * 1$$

Backpropagation Example

$$\frac{dL}{dw_3} = \frac{dL}{d\text{Predicted}} * \frac{d\text{Predicted}}{dw_3} = \sum_{i=1}^{n=3} -2 * (\text{Observed}_i - \text{Predicted}_i)$$

$$\frac{dL}{dw_4} = \frac{dL}{d\text{Predicted}} * \frac{d\text{Predicted}}{dw_4} = \sum_{i=1}^{n=3} -2 * (\text{Observed}_i - \text{Predicted}_i)$$

$$\frac{d\text{Predicted}}{dw_3} = \frac{d}{dw_3} \text{purple squiggle} = \frac{d}{dw_3} (y_{1,i}w_3 + y_{2,i}w_4 + b_3) = y_{1,i}$$

$$\frac{d\text{Predicted}}{dw_4} = \frac{d}{dw_4} \text{purple squiggle} = \frac{d}{dw_4} (y_{1,i}w_3 + y_{2,i}w_4 + b_3) = y_{2,i}$$

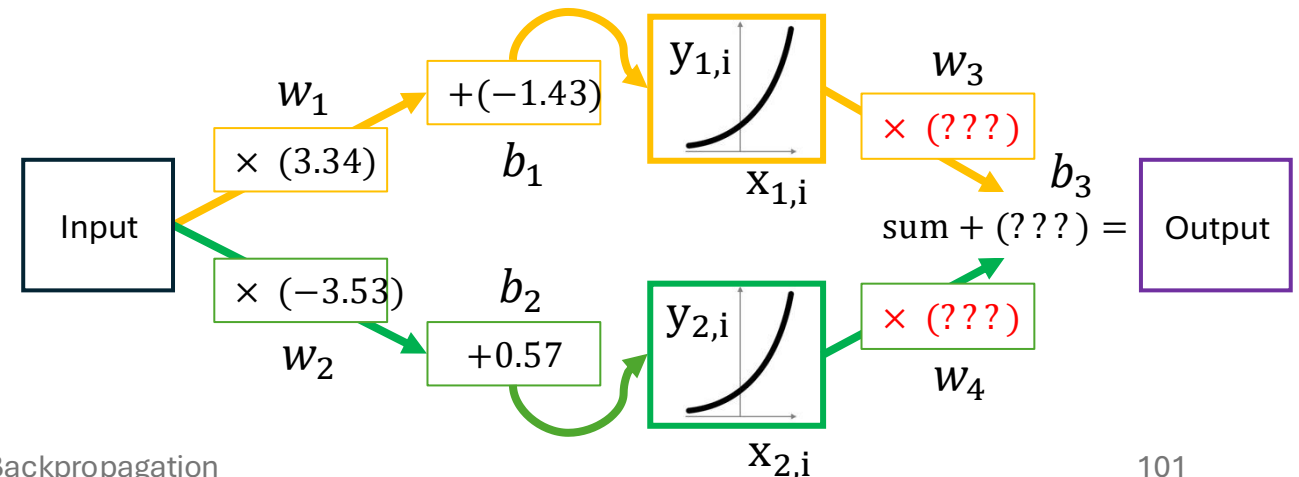
Backpropagation Example

$$\frac{dL}{dw_3} = \frac{dL}{d\text{Predicted}} * \frac{d\text{Predicted}}{dw_3} = \sum_{i=1}^{n=3} -2 * (\text{Observed}_i - \text{Predicted}_i) * y_{1,i}$$

$$\frac{dL}{dw_4} = \frac{dL}{d\text{Predicted}} * \frac{d\text{Predicted}}{dw_4} = \sum_{i=1}^{n=3} -2 * (\text{Observed}_i - \text{Predicted}_i) * y_{2,i}$$

$$\frac{dL}{db_3} = \frac{dL}{d\text{Predicted}} * \frac{d\text{Predicted}}{db_3} = \sum_{i=1}^{n=3} -2 * (\text{Observed}_i - \text{Predicted}_i) * 1$$

... we can plug them into Gradient Descent to optimize w_3 , w_4 , and b_3



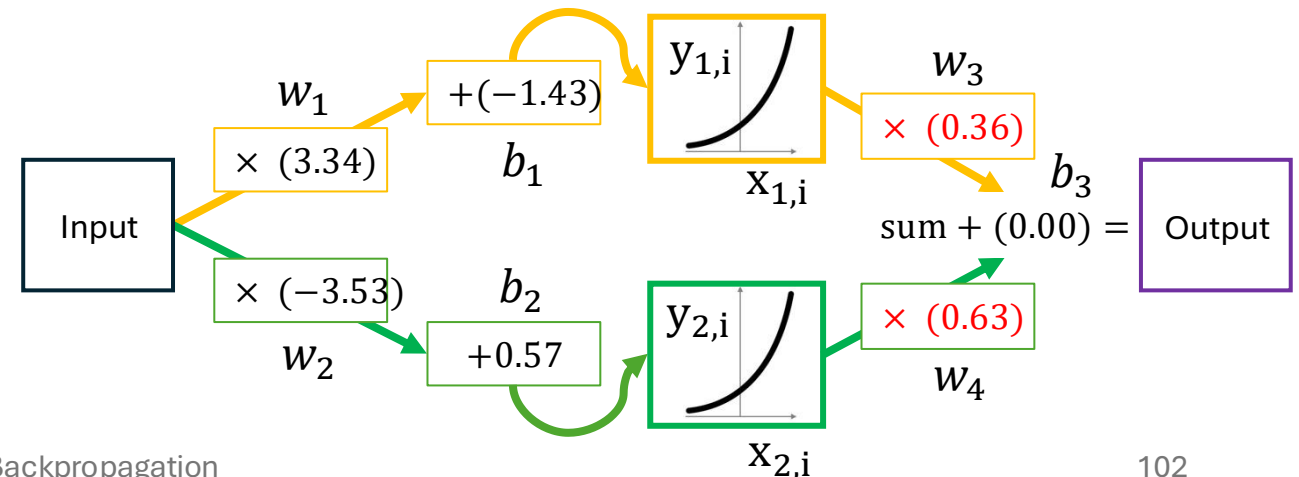
Backpropagation Example

$$\frac{dL}{dw_3} = \frac{dL}{d\text{Predicted}} * \frac{d\text{Predicted}}{dw_3} = \sum_{i=1}^{n=3} -2 * (\text{Observed}_i - \text{Predicted}_i) * y_{1,i}$$

$$\frac{dL}{dw_4} = \frac{dL}{d\text{Predicted}} * \frac{d\text{Predicted}}{dw_4} = \sum_{i=1}^{n=3} -2 * (\text{Observed}_i - \text{Predicted}_i) * y_{2,i}$$

$$\frac{dL}{db_3} = \frac{dL}{d\text{Predicted}} * \frac{d\text{Predicted}}{db_3} = \sum_{i=1}^{n=3} -2 * (\text{Observed}_i - \text{Predicted}_i) * 1$$

... we can plug them into Gradient Descent to optimize w_3 , w_4 , and b_3

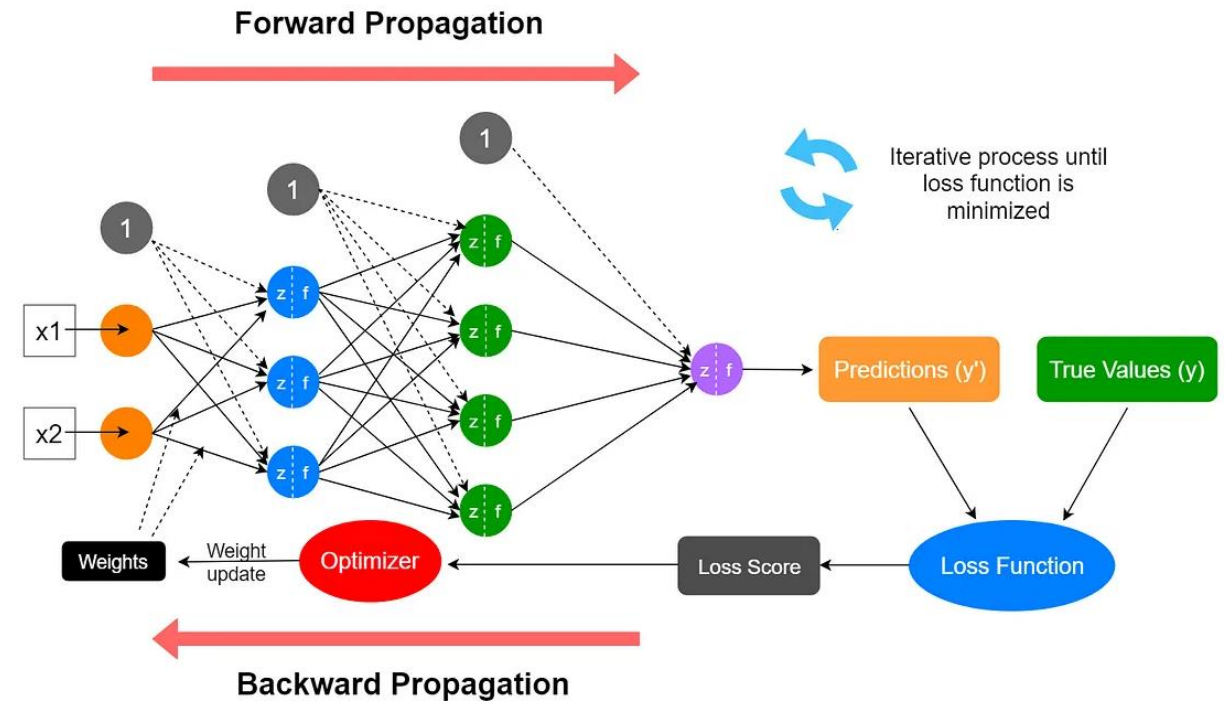


Zusammenfassung



Backpropagation is:

- Efficient gradient computation
- Based on chain rule
- Uses stored forward values
- Enables gradient descent in deep networks



<https://wandb.ai/mostafaibrahim17/ml-articles/reports/Decoding-Backpropagation-and-Its-Role-in-Neural-Network-Learning--Vmlldzo3MTY5MzM1>

Weiterführende Quellen



Videos

- [Gradient Descent, Step-by-Step](#) ([StatQuest with Josh Starmer](#) ~ 24 min)
- [Gradient descent, how neural networks learn | Deep Learning Chapter 2](#) ([3Blue1Brown](#) ~ 20 min)
- [Backpropagation, Gradientenabstieg, Fehlersignal, Aktivität, Hyperparameter, Loss](#) ([Professor Joe](#) ~ 28 min)

Folien

- [2. Unconstrained Optimization / 2.1. Gradient Descent](#) (Lars Schmidt-Thieme)

Bücher

- [Hands-On Machine Learning with Scikit-Learn and PyTorch](#) (Aurélien Géron)
- [Math for Deep Learning](#) (Ronald T. Kneusel)

Kontakt



Mail:
konak@uni-potsdam.de

Folien von heute



<https://github.com/konakion/backpropagation>